

**ThoughtWorks®**

***Jeroen Soeters***

---

# NEUROEVOLUTION

---

*The Hitchhiker's Guide to Neuroevolution in  
Erlang.*

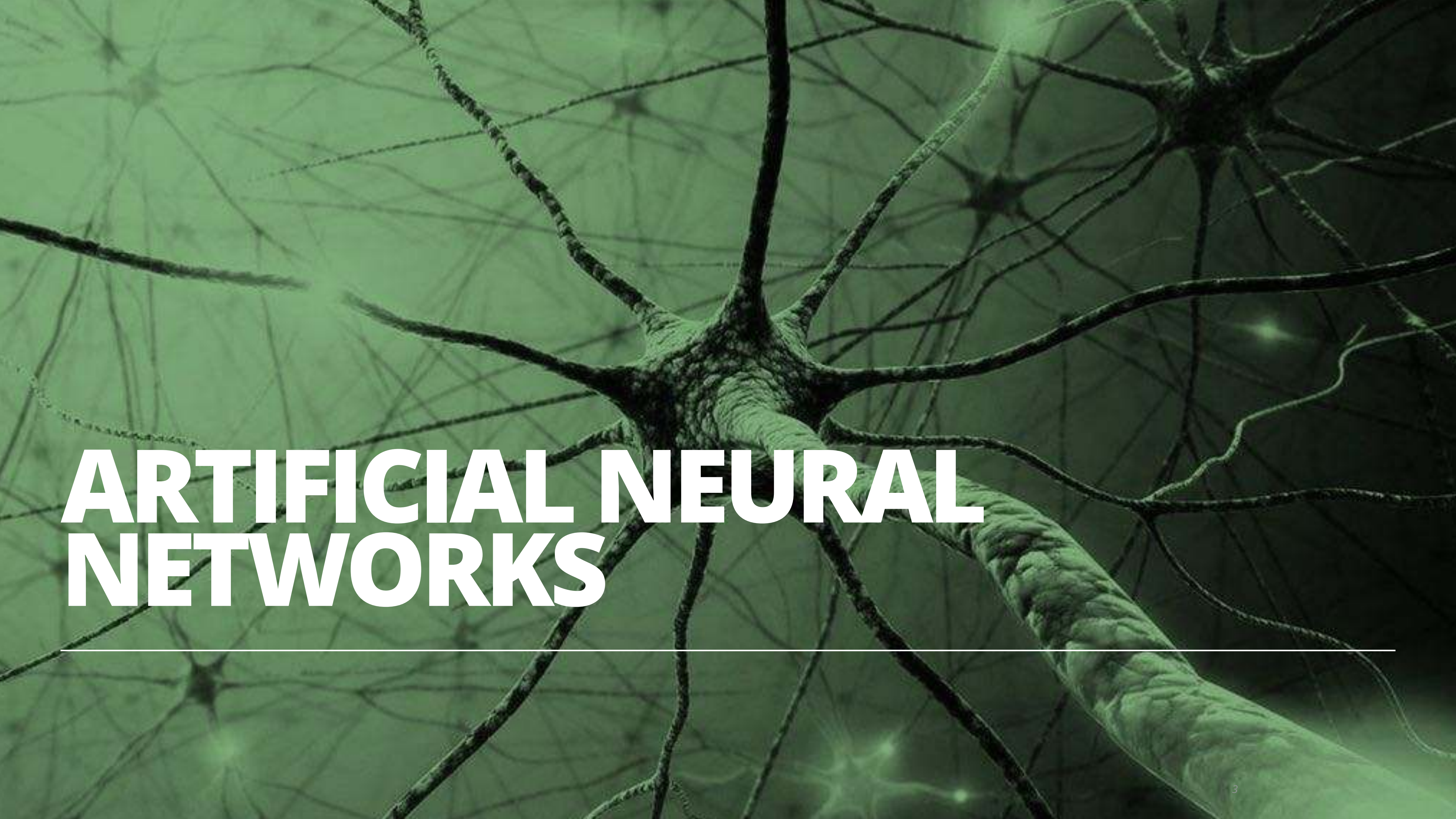
# JEROEN... HOW DO YOU PRONOUNCE THAT?

---

Well, it goes like this:

YUH-ROON





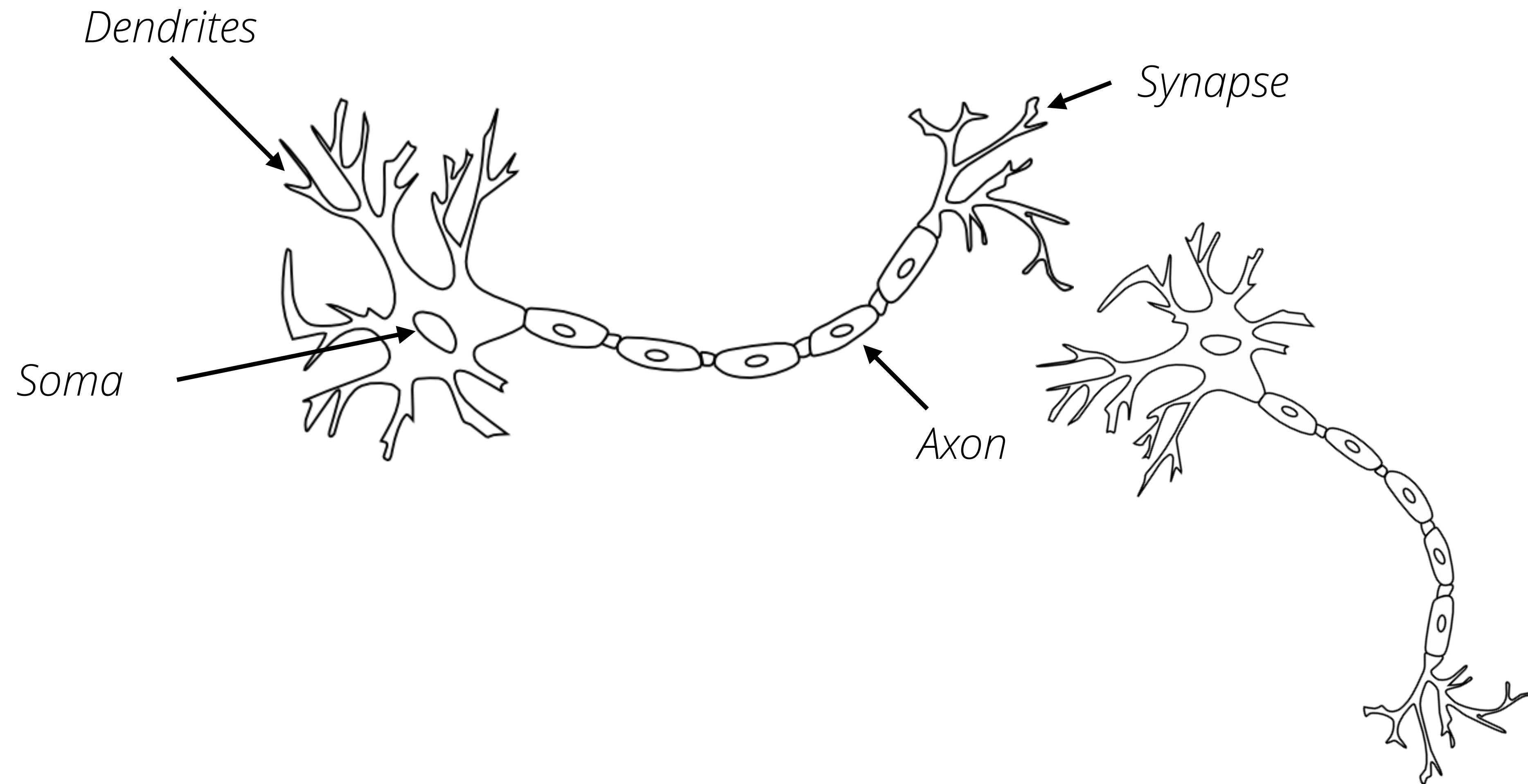
# ARTIFICIAL NEURAL NETWORKS

---



# BIOLOGICAL NEURAL NETWORKS

---



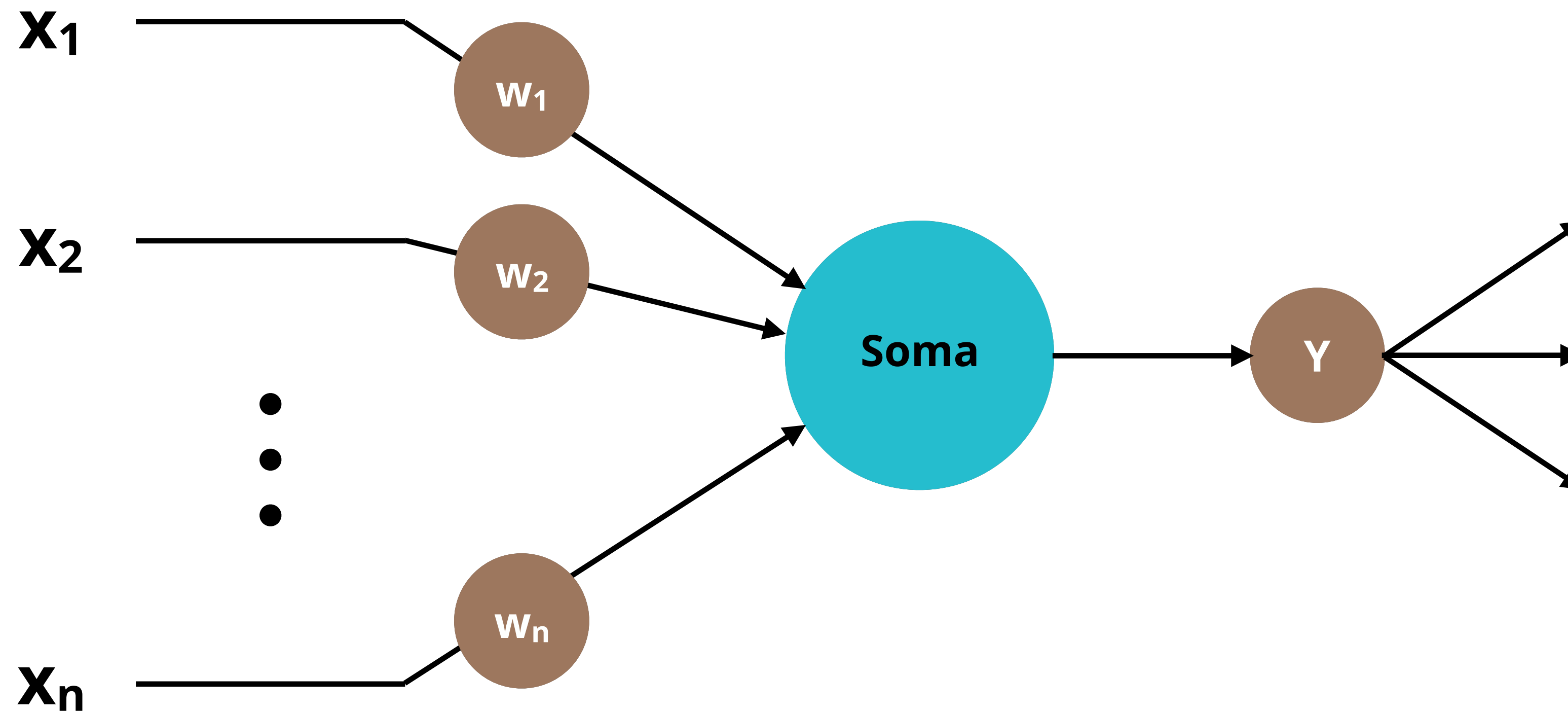
# A COMPUTER MODEL FOR A NEURON

---

*Dendrites*

*Synapses*

*Axon*



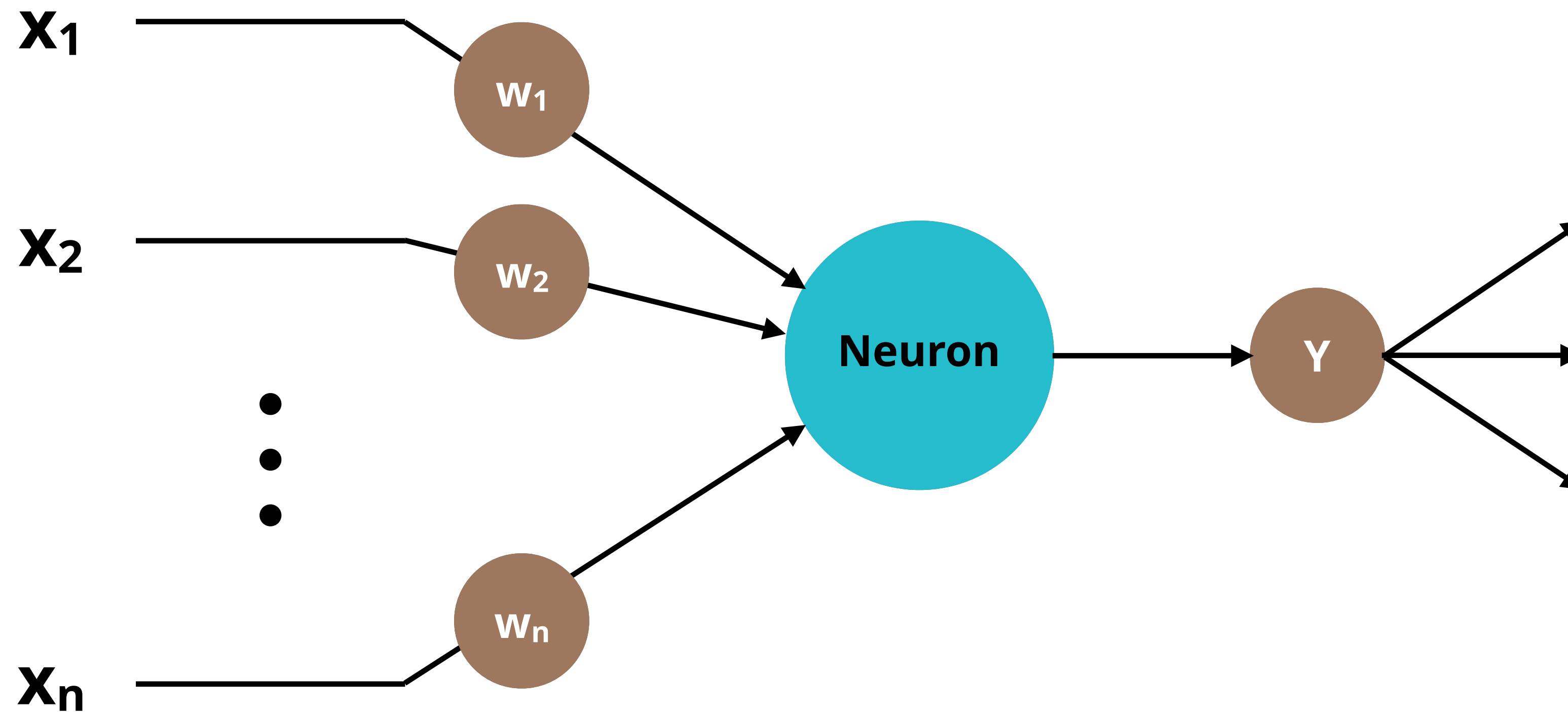
# A COMPUTER MODEL FOR A NEURON

---

*Input signals*

*Weights*

*Output signal*





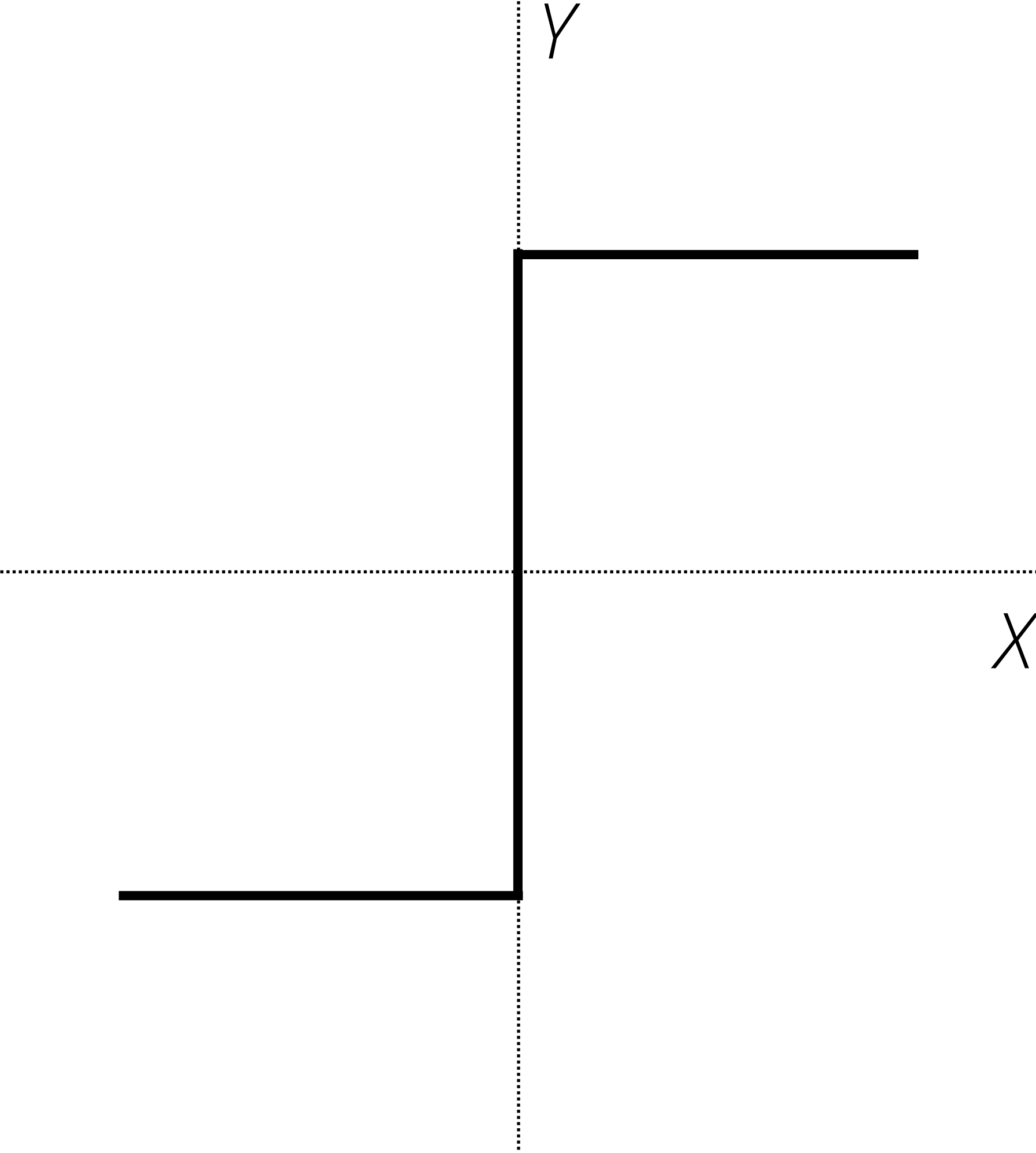
# HOW DOES THE NEURON DETERMINE IT'S OUTPUT?

---

$$Y = \textit{sign} \left[ \sum_{n=1}^n X_i W_i - \theta \right]$$

# ACTIVATION FUNCTION

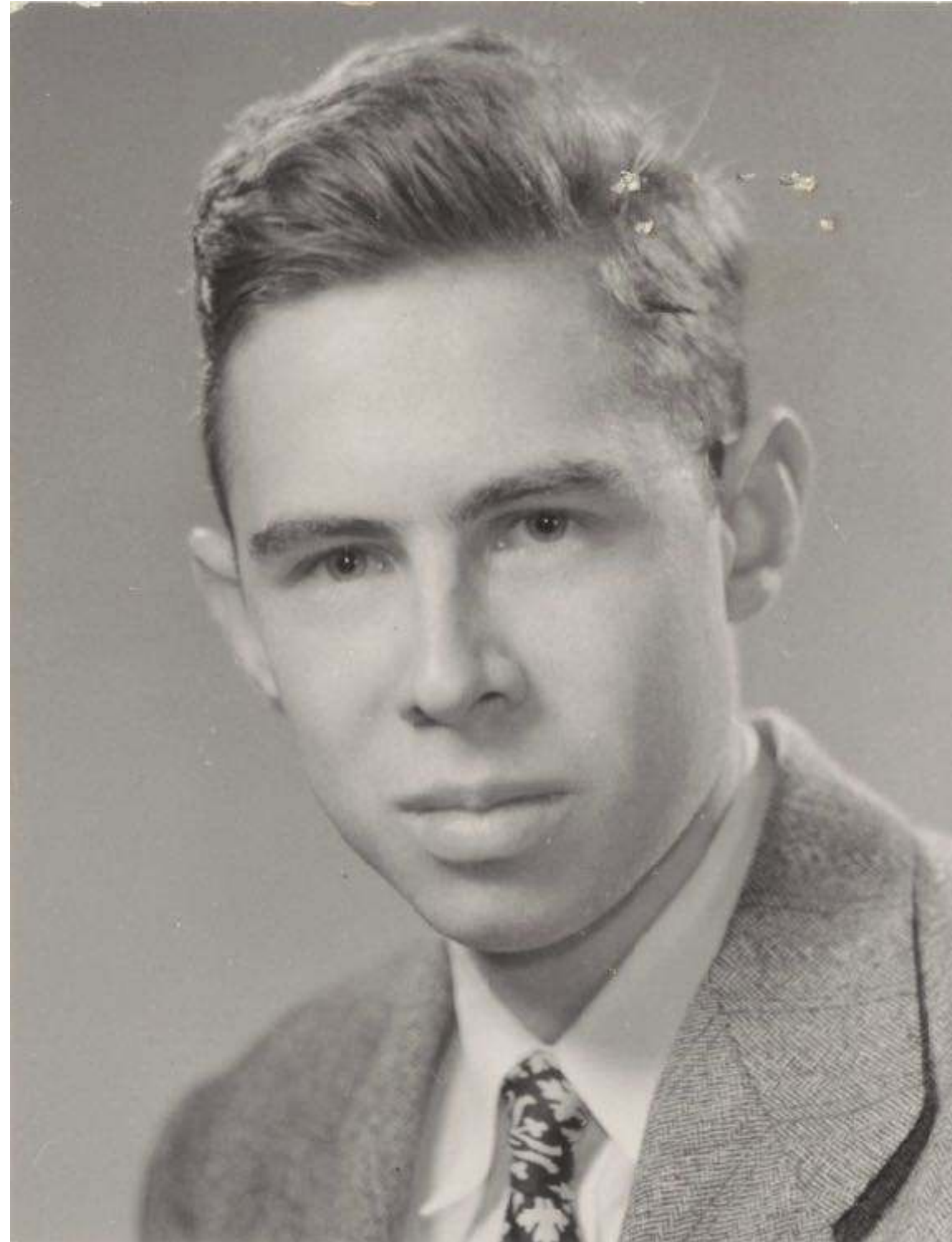
---





# MEET FRANK ROSENBLATT

---



# PERCEPTRON LEARNING RULE

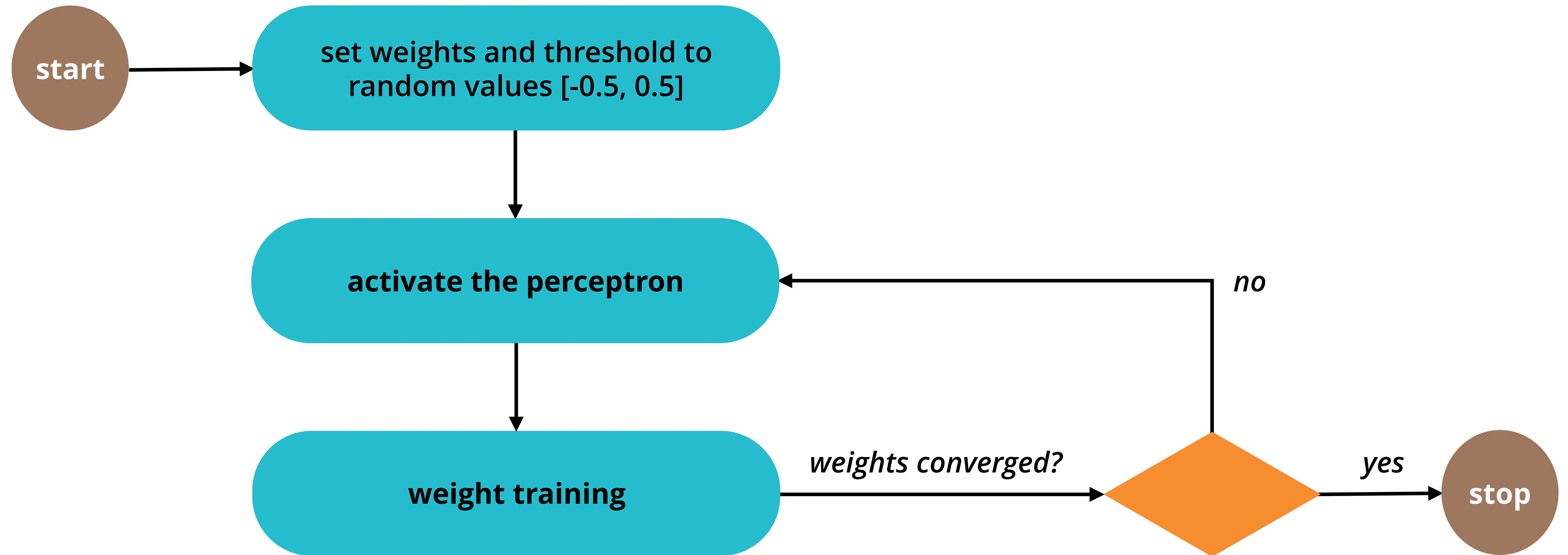
---

$$e(p) = Y_d(p) - Y(p)$$

$$w_i(p + 1) = w_i(p) + \alpha \cdot w_i(p) \cdot e(p)$$

# PERCEPTRON TRAINING ALGORITHM

---





# LOGIC GATES

---

| input values |    |           |          |           |
|--------------|----|-----------|----------|-----------|
| x1           | x2 | x1 AND x2 | x1 OR x2 | x1 XOR x2 |
| 0            | 0  | 0         | 0        | 0         |
| 0            | 1  | 0         | 1        | 1         |
| 1            | 0  | 0         | 1        | 1         |
| 1            | 1  | 1         | 1        | 0         |



# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 1     | 0      | 0     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 0      | 1     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 1      | 0     | 0                       | 0.3             | -0.1  | 1                    | -1           | 0.2           | -0.1  |
|       | 1      | 1     | 1                       | 0.2             | -0.1  | 0                    | 1            | 0.3           | 0.0   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$

# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 1     | 0      | 0     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 0      | 1     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 1      | 0     | 0                       | 0.3             | -0.1  | 1                    | -1           | 0.2           | -0.1  |
|       | 1      | 1     | 1                       | 0.2             | -0.1  | 0                    | 1            | 0.3           | 0.0   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$

# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 1     | 0      | 0     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 0      | 1     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 1      | 0     | 0                       | 0.3             | -0.1  | 1                    | -1           | 0.2           | -0.1  |
|       | 1      | 1     | 1                       | 0.2             | -0.1  | 0                    | 1            | 0.3           | 0.0   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$



# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 1     | 0      | 0     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 0      | 1     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 1      | 0     | 0                       | 0.3             | -0.1  | 1                    | -1           | 0.2           | -0.1  |
|       | 1      | 1     | 1                       | 0.2             | -0.1  | 0                    | 1            | 0.3           | 0.0   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$

# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 1     | 0      | 0     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 0      | 1     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 1      | 0     | 0                       | 0.3             | -0.1  | 1                    | -1           | 0.2           | -0.1  |
|       | 1      | 1     | 1                       | 0.2             | -0.1  | 0                    | 1            | 0.3           | 0.0   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$



# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 1     | 0      | 0     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 0      | 1     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 1      | 0     | 0                       | 0.3             | -0.1  | 1                    | -1           | 0.2           | -0.1  |
|       | 1      | 1     | 1                       | 0.2             | -0.1  | 0                    | 1            | 0.3           | 0.0   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$

# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 1     | 0      | 0     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 0      | 1     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 1      | 0     | 0                       | 0.3             | -0.1  | 1                    | -1           | 0.2           | -0.1  |
|       | 1      | 1     | 1                       | 0.2             | -0.1  | 0                    | 1            | 0.3           | 0.0   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$

# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 1     | 0      | 0     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 0      | 1     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 1      | 0     | 0                       | 0.3             | -0.1  | 1                    | -1           | 0.2           | -0.1  |
|       | 1      | 1     | 1                       | 0.2             | -0.1  | 0                    | 1            | 0.3           | 0.0   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$



# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 1     | 0      | 0     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 0      | 1     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 1      | 0     | 0                       | 0.3             | -0.1  | 1                    | -1           | 0.2           | -0.1  |
|       | 1      | 1     | 1                       | 0.2             | -0.1  | 0                    | 1            | 0.3           | 0.0   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$

# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 1     | 0      | 0     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 0      | 1     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 1      | 0     | 0                       | 0.3             | -0.1  | 1                    | -1           | 0.2           | -0.1  |
|       | 1      | 1     | 1                       | 0.2             | -0.1  | 0                    | 1            | 0.3           | 0.0   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$

# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 1     | 0      | 0     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 0      | 1     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 1      | 0     | 0                       | 0.3             | -0.1  | 1                    | -1           | 0.2           | -0.1  |
|       | 1      | 1     | 1                       | 0.2             | -0.1  | 0                    | 1            | 0.3           | 0.0   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$



# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 1     | 0      | 0     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 0      | 1     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 1      | 0     | 0                       | 0.3             | -0.1  | 1                    | -1           | 0.2           | -0.1  |
|       | 1      | 1     | 1                       | 0.2             | -0.1  | 0                    | 1            | 0.3           | 0.0   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$

# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 1     | 0      | 0     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 0      | 1     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 1      | 0     | 0                       | 0.3             | -0.1  | 1                    | -1           | 0.2           | -0.1  |
|       | 1      | 1     | 1                       | 0.2             | -0.1  | 0                    | 1            | 0.3           | 0.0   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$

# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 1     | 0      | 0     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 0      | 1     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 1      | 0     | 0                       | 0.3             | -0.1  | 1                    | -1           | 0.2           | -0.1  |
|       | 1      | 1     | 1                       | 0.2             | -0.1  | 0                    | 1            | 0.3           | 0.0   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$



# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 1     | 0      | 0     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 0      | 1     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 1      | 0     | 0                       | 0.3             | -0.1  | 1                    | -1           | 0.2           | -0.1  |
|       | 1      | 1     | 1                       | 0.2             | -0.1  | 0                    | 1            | 0.3           | 0.0   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$

# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 1     | 0      | 0     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 0      | 1     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 1      | 0     | 0                       | 0.3             | -0.1  | 1                    | -1           | 0.2           | -0.1  |
|       | 1      | 1     | 1                       | 0.2             | -0.1  | 0                    | 1            | 0.3           | 0.0   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$

# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 1     | 0      | 0     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 0      | 1     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 1      | 0     | 0                       | 0.3             | -0.1  | 1                    | -1           | 0.2           | -0.1  |
|       | 1      | 1     | 1                       | 0.2             | -0.1  | 0                    | 1            | 0.3           | 0.0   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$



# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 1     | 0      | 0     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 0      | 1     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 1      | 0     | 0                       | 0.3             | -0.1  | 1                    | -1           | 0.2           | -0.1  |
|       | 1      | 1     | 1                       | 0.2             | -0.1  | 0                    | 1            | 0.3           | 0.0   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$



# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 1     | 0      | 0     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 0      | 1     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 1      | 0     | 0                       | 0.3             | -0.1  | 1                    | -1           | 0.2           | -0.1  |
|       | 1      | 1     | 1                       | 0.2             | -0.1  | 0                    | 1            | 0.3           | 0.0   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$

# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 1     | 0      | 0     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 0      | 1     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 1      | 0     | 0                       | 0.3             | -0.1  | 1                    | -1           | 0.2           | -0.1  |
|       | 1      | 1     | 1                       | 0.2             | -0.1  | 0                    | 1            | 0.3           | 0.0   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$

# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 1     | 0      | 0     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 0      | 1     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 1      | 0     | 0                       | 0.3             | -0.1  | 1                    | -1           | 0.2           | -0.1  |
|       | 1      | 1     | 1                       | 0.2             | -0.1  | 0                    | 1            | 0.3           | 0.0   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$



# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 1     | 0      | 0     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 0      | 1     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 1      | 0     | 0                       | 0.3             | -0.1  | 1                    | -1           | 0.2           | -0.1  |
|       | 1      | 1     | 1                       | 0.2             | -0.1  | 0                    | 1            | 0.3           | 0.0   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$

# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 1     | 0      | 0     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 0      | 1     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 1      | 0     | 0                       | 0.3             | -0.1  | 1                    | -1           | 0.2           | -0.1  |
|       | 1      | 1     | 1                       | 0.2             | -0.1  | 0                    | 1            | 0.3           | 0.0   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$

# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 1     | 0      | 0     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 0      | 1     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 1      | 0     | 0                       | 0.3             | -0.1  | 1                    | -1           | 0.2           | -0.1  |
|       | 1      | 1     | 1                       | 0.2             | -0.1  | 0                    | 1            | 0.3           | 0.0   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$



# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 1     | 0      | 0     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 0      | 1     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 1      | 0     | 0                       | 0.3             | -0.1  | 1                    | -1           | 0.2           | -0.1  |
|       | 1      | 1     | 1                       | 0.2             | -0.1  | 0                    | 1            | 0.3           | 0.0   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$

# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 1     | 0      | 0     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 0      | 1     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 1      | 0     | 0                       | 0.3             | -0.1  | 1                    | -1           | 0.2           | -0.1  |
|       | 1      | 1     | 1                       | 0.2             | -0.1  | 0                    | 1            | 0.3           | 0.0   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$



# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 1     | 0      | 0     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 0      | 1     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 1      | 0     | 0                       | 0.3             | -0.1  | 1                    | -1           | 0.2           | -0.1  |
|       | 1      | 1     | 1                       | 0.2             | -0.1  | 0                    | 1            | 0.3           | 0.0   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$

# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 1     | 0      | 0     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 0      | 1     | 0                       | 0.3             | -0.1  | 0                    | 0            | 0.3           | -0.1  |
|       | 1      | 0     | 0                       | 0.3             | -0.1  | 1                    | -1           | 0.2           | -0.1  |
|       | 1      | 1     | 1                       | 0.2             | -0.1  | 0                    | 1            | 0.3           | 0.0   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$

# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

---

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 2     | 0      | 0     | 0                       | 0.3             | 0.0   | 0                    | 0            | 0.3           | 0.0   |
|       | 0      | 1     | 0                       | 0.3             | 0.0   | 0                    | 0            | 0.3           | 0.0   |
|       | 1      | 0     | 0                       | 0.3             | 0.0   | 1                    | -1           | 0.2           | 0.0   |
|       | 1      | 1     | 1                       | 0.2             | 0.0   | 1                    | 0            | 0.2           | 0.0   |

*Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$*



# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 2     | 0      | 0     | 0                       | 0.3             | 0.0   | 0                    | 0            | 0.3           | 0.0   |
|       | 0      | 1     | 0                       | 0.3             | 0.0   | 0                    | 0            | 0.3           | 0.0   |
|       | 1      | 0     | 0                       | 0.3             | 0.0   | 1                    | -1           | 0.2           | 0.0   |
|       | 1      | 1     | 1                       | 0.2             | 0.0   | 1                    | 0            | 0.2           | 0.0   |

*Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$*

# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 2     | 0      | 0     | 0                       | 0.3             | 0.0   | 0                    | 0            | 0.3           | 0.0   |
|       | 0      | 1     | 0                       | 0.3             | 0.0   | 0                    | 0            | 0.3           | 0.0   |
|       | 1      | 0     | 0                       | 0.3             | 0.0   | 1                    | -1           | 0.2           | 0.0   |
|       | 1      | 1     | 1                       | 0.2             | 0.0   | 1                    | 0            | 0.2           | 0.0   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$

# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 2     | 0      | 0     | 0                       | 0.3             | 0.0   | 0                    | 0            | 0.3           | 0.0   |
|       | 0      | 1     | 0                       | 0.3             | 0.0   | 0                    | 0            | 0.3           | 0.0   |
|       | 1      | 0     | 0                       | 0.3             | 0.0   | 1                    | -1           | 0.2           | 0.0   |
|       | 1      | 1     | 1                       | 0.2             | 0.0   | 1                    | 0            | 0.2           | 0.0   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$



# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 3     | 0      | 0     | 0                       | 0.2             | 0.0   | 0                    | 0            | 0.2           | 0.0   |
|       | 0      | 1     | 0                       | 0.2             | 0.0   | 0                    | 0            | 0.2           | 0.0   |
|       | 1      | 0     | 0                       | 0.2             | 0.0   | 1                    | -1           | 0.1           | 0.0   |
|       | 1      | 1     | 1                       | 0.1             | 0.0   | 0                    | 1            | 0.2           | 0.1   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$

# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 3     | 0      | 0     | 0                       | 0.2             | 0.0   | 0                    | 0            | 0.2           | 0.0   |
|       | 0      | 1     | 0                       | 0.2             | 0.0   | 0                    | 0            | 0.2           | 0.0   |
|       | 1      | 0     | 0                       | 0.2             | 0.0   | 1                    | -1           | 0.1           | 0.0   |
|       | 1      | 1     | 1                       | 0.1             | 0.0   | 0                    | 1            | 0.2           | 0.1   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$



# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 3     | 0      | 0     | 0                       | 0.2             | 0.0   | 0                    | 0            | 0.2           | 0.0   |
|       | 0      | 1     | 0                       | 0.2             | 0.0   | 0                    | 0            | 0.2           | 0.0   |
|       | 1      | 0     | 0                       | 0.2             | 0.0   | 1                    | -1           | 0.1           | 0.0   |
|       | 1      | 1     | 1                       | 0.1             | 0.0   | 0                    | 1            | 0.2           | 0.1   |

*Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$*

# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 3     | 0      | 0     | 0                       | 0.2             | 0.0   | 0                    | 0            | 0.2           | 0.0   |
|       | 0      | 1     | 0                       | 0.2             | 0.0   | 0                    | 0            | 0.2           | 0.0   |
|       | 1      | 0     | 0                       | 0.2             | 0.0   | 1                    | -1           | 0.1           | 0.0   |
|       | 1      | 1     | 1                       | 0.1             | 0.0   | 0                    | 1            | 0.2           | 0.1   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$

# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 4     | 0      | 0     | 0                       | 0.2             | 0.1   | 0                    | 0            | 0.2           | 0.1   |
|       | 0      | 1     | 0                       | 0.2             | 0.1   | 0                    | 0            | 0.2           | 0.1   |
|       | 1      | 0     | 0                       | 0.2             | 0.1   | 1                    | -1           | 0.1           | 0.1   |
|       | 1      | 1     | 1                       | 0.1             | 0.1   | 1                    | 0            | 0.1           | 0.1   |

*Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$*



# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 4     | 0      | 0     | 0                       | 0.2             | 0.1   | 0                    | 0            | 0.2           | 0.1   |
|       | 0      | 1     | 0                       | 0.2             | 0.1   | 0                    | 0            | 0.2           | 0.1   |
|       | 1      | 0     | 0                       | 0.2             | 0.1   | 1                    | -1           | 0.1           | 0.1   |
|       | 1      | 1     | 1                       | 0.1             | 0.1   | 1                    | 0            | 0.1           | 0.1   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$

# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 4     | 0      | 0     | 0                       | 0.2             | 0.1   | 0                    | 0            | 0.2           | 0.1   |
|       | 0      | 1     | 0                       | 0.2             | 0.1   | 0                    | 0            | 0.2           | 0.1   |
|       | 1      | 0     | 0                       | 0.2             | 0.1   | 1                    | -1           | 0.1           | 0.1   |
|       | 1      | 1     | 1                       | 0.1             | 0.1   | 1                    | 0            | 0.1           | 0.1   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$

# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 4     | 0      | 0     | 0                       | 0.2             | 0.1   | 0                    | 0            | 0.2           | 0.1   |
|       | 0      | 1     | 0                       | 0.2             | 0.1   | 0                    | 0            | 0.2           | 0.1   |
|       | 1      | 0     | 0                       | 0.2             | 0.1   | 1                    | -1           | 0.1           | 0.1   |
|       | 1      | 1     | 1                       | 0.1             | 0.1   | 1                    | 0            | 0.1           | 0.1   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$



# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 5     | 0      | 0     | 0                       | 0.1             | 0.1   | 0                    | 0            | 0.1           | 0.1   |
|       | 0      | 1     | 0                       | 0.1             | 0.1   | 0                    | 0            | 0.1           | 0.1   |
|       | 1      | 0     | 0                       | 0.1             | 0.1   | 0                    | 0            | 0.1           | 0.1   |
|       | 1      | 1     | 1                       | 0.1             | 0.1   | 1                    | 0            | 0.1           | 0.1   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$

# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 5     | 0      | 0     | 0                       | 0.1             | 0.1   | 0                    | 0            | 0.1           | 0.1   |
|       | 0      | 1     | 0                       | 0.1             | 0.1   | 0                    | 0            | 0.1           | 0.1   |
|       | 1      | 0     | 0                       | 0.1             | 0.1   | 0                    | 0            | 0.1           | 0.1   |
|       | 1      | 1     | 1                       | 0.1             | 0.1   | 1                    | 0            | 0.1           | 0.1   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$



# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 5     | 0      | 0     | 0                       | 0.1             | 0.1   | 0                    | 0            | 0.1           | 0.1   |
|       | 0      | 1     | 0                       | 0.1             | 0.1   | 0                    | 0            | 0.1           | 0.1   |
|       | 1      | 0     | 0                       | 0.1             | 0.1   | 0                    | 0            | 0.1           | 0.1   |
|       | 1      | 1     | 1                       | 0.1             | 0.1   | 1                    | 0            | 0.1           | 0.1   |

Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$

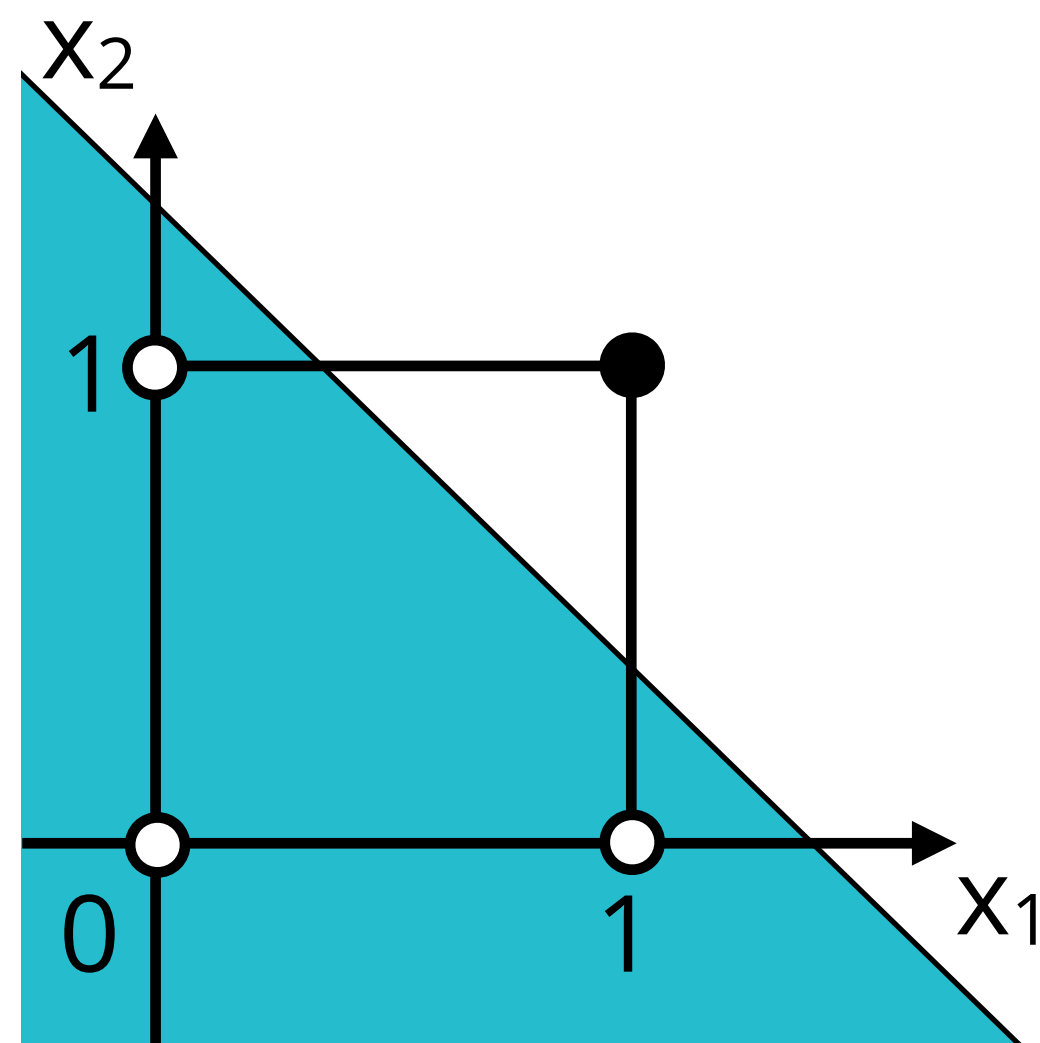
# TRAINING A PERCEPTRON TO PERFORM THE AND OPERATION

| epoch | inputs |       | desired output<br>$Y_d$ | initial weights |       | actual output<br>$Y$ | error<br>$e$ | final weights |       |
|-------|--------|-------|-------------------------|-----------------|-------|----------------------|--------------|---------------|-------|
|       | $x_1$  | $x_2$ |                         | $w_1$           | $w_2$ |                      |              | $w_1$         | $w_2$ |
| 5     | 0      | 0     | 0                       | 0.1             | 0.1   | 0                    | 0            | 0.1           | 0.1   |
|       | 0      | 1     | 0                       | 0.1             | 0.1   | 0                    | 0            | 0.1           | 0.1   |
|       | 1      | 0     | 0                       | 0.1             | 0.1   | 0                    | 0            | 0.1           | 0.1   |
|       | 1      | 1     | 1                       | 0.1             | 0.1   | 1                    | 0            | 0.1           | 0.1   |

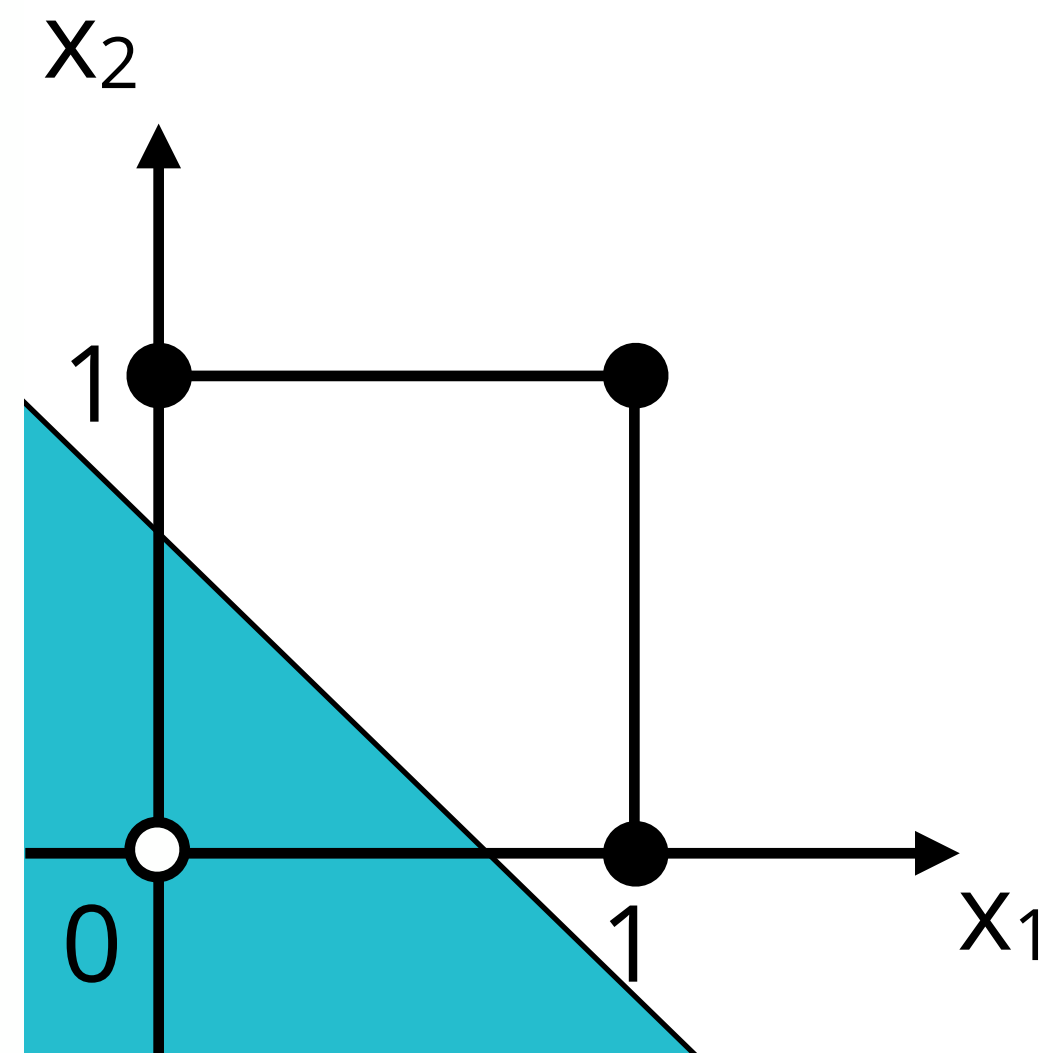
Threshold:  $\theta = 0.2$  ; learning rate:  $\alpha = 0.1$

# A LITTLE GEOMETRY...

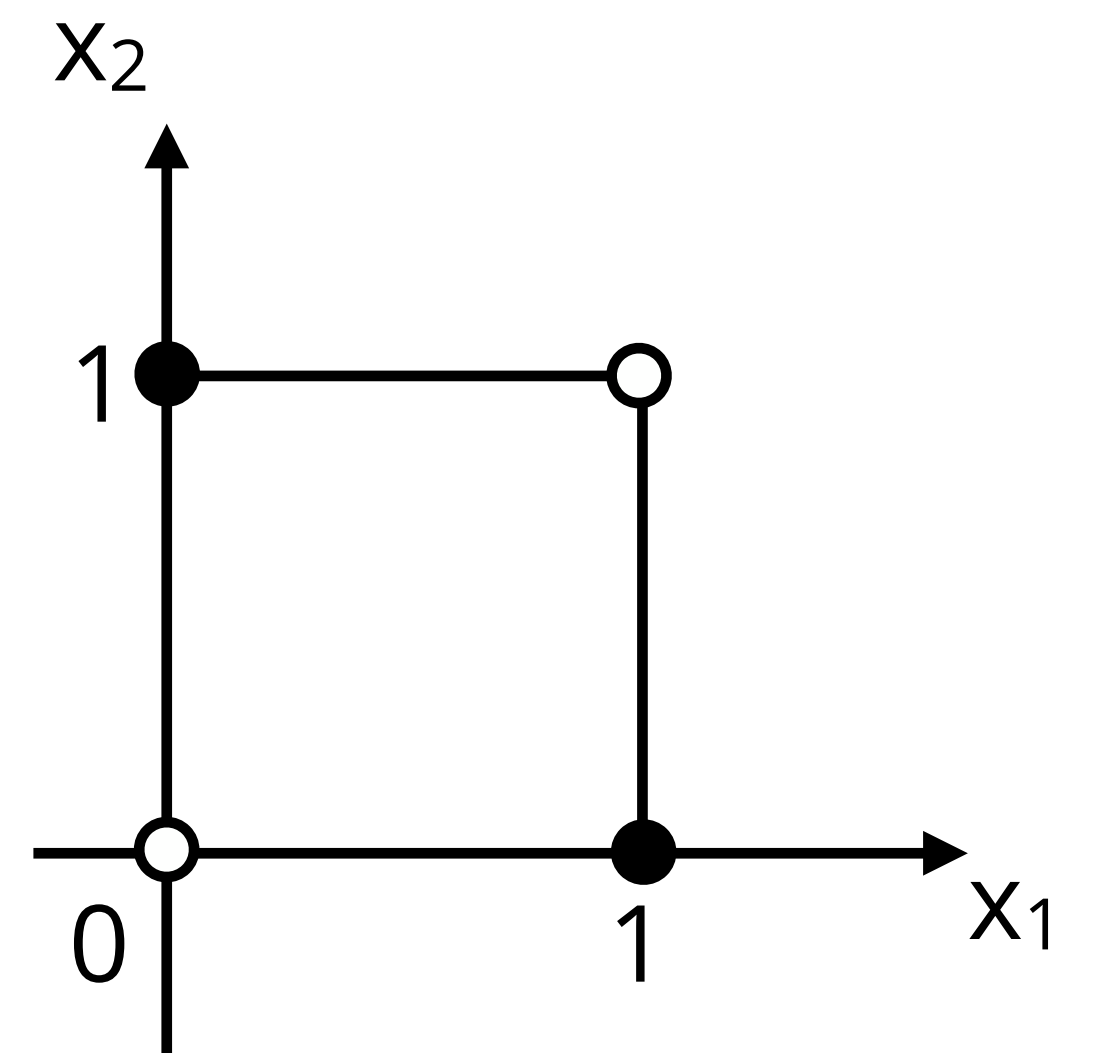
---



$x_1 \text{ AND } x_2$



$x_1 \text{ OR } x_2$

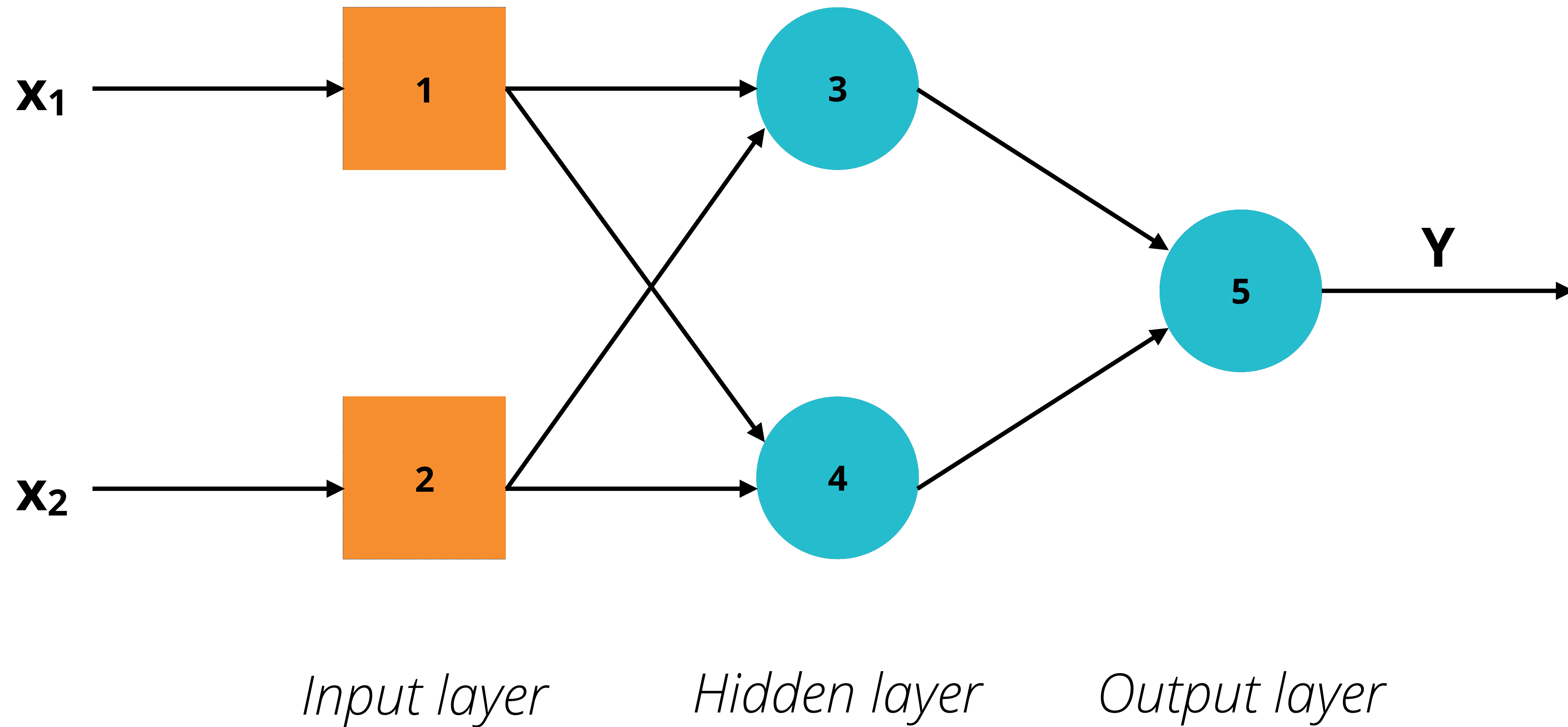


$x_1 \text{ XOR } x_2$



# WE NEED MORE LAYERS

---



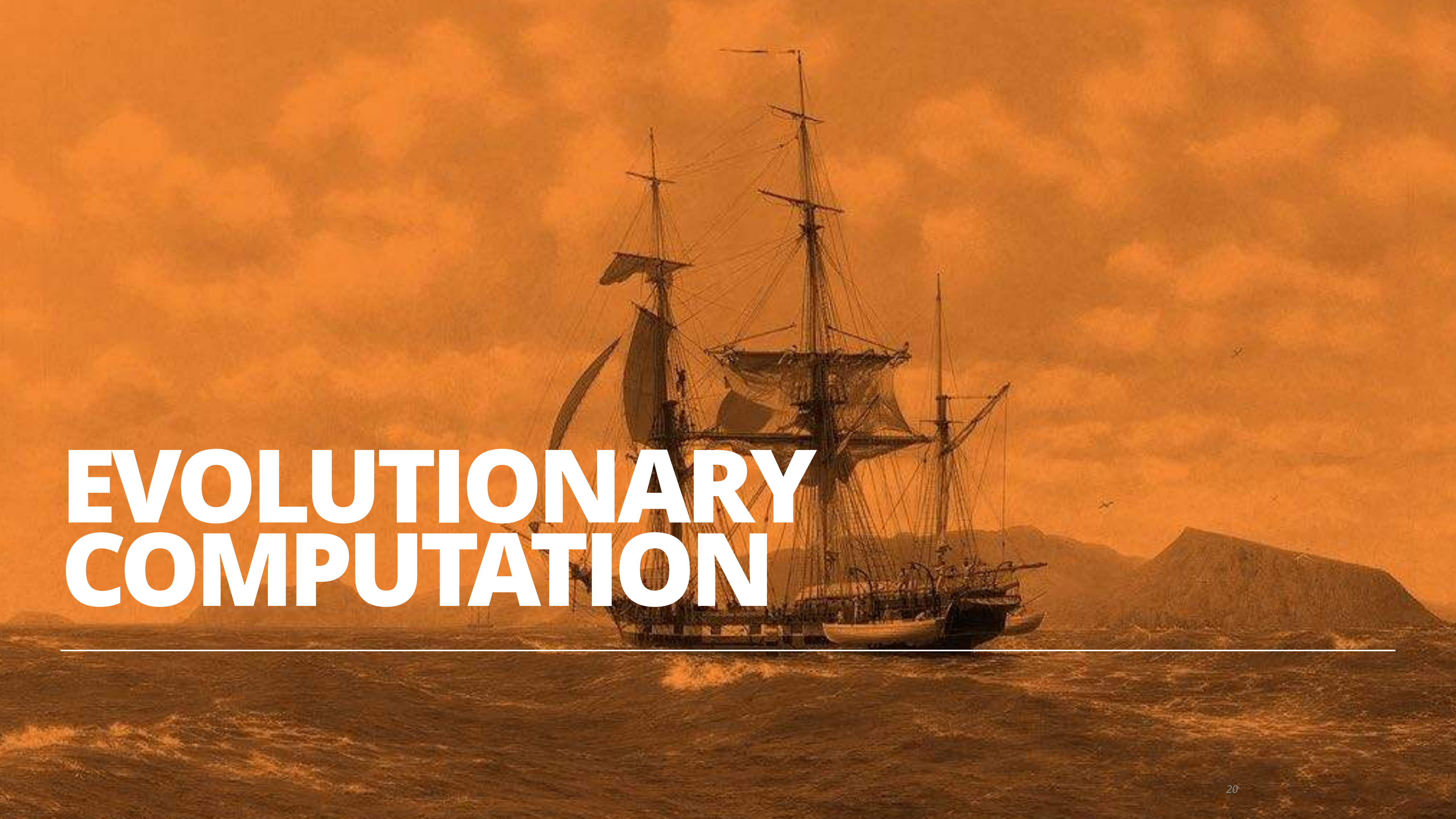
# **LIMITATIONS OF BACK PROPAGATION**

---

- a training set of sufficient size is required
- topology of the network needs to be known in advance
- no recurrent connections are allowed
- activation function must be differentiable

Only works well for supervised learning!





# EVOLUTIONARY COMPUTATION

---



# MEET JOHN HOLLAND

---



# THE CHROMOSOME

---

1 0 0 1 1 1 0 1



# CROSSOVER

---

*parents*

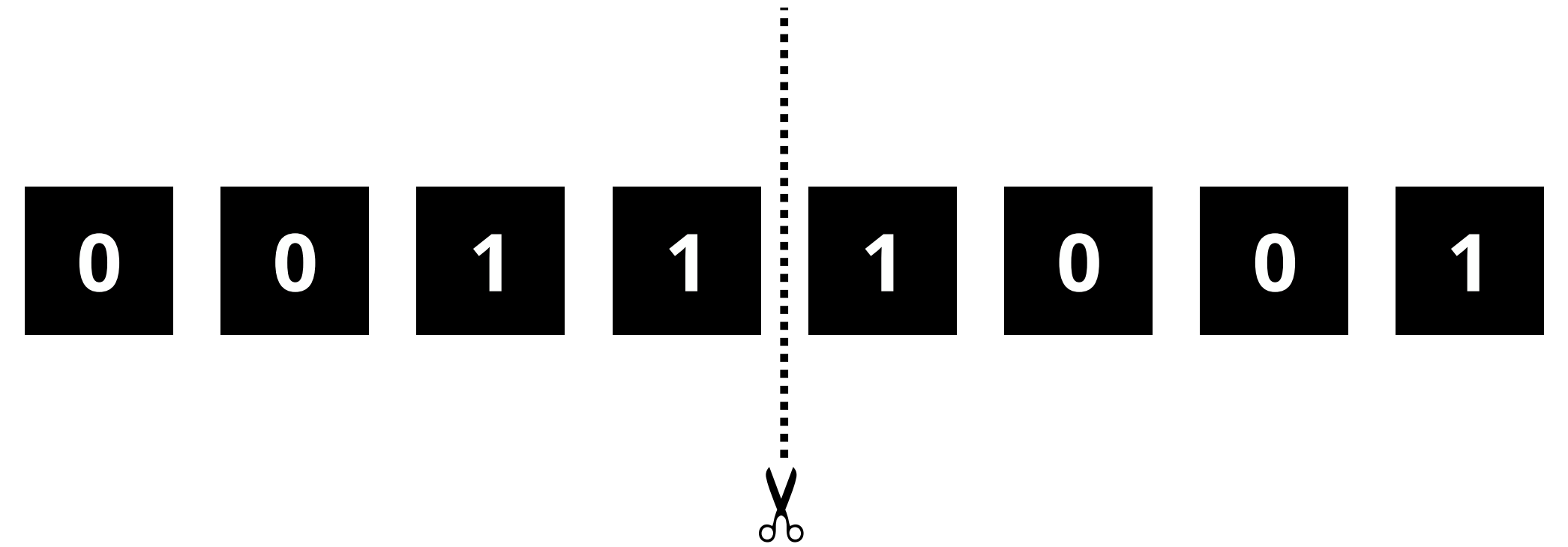
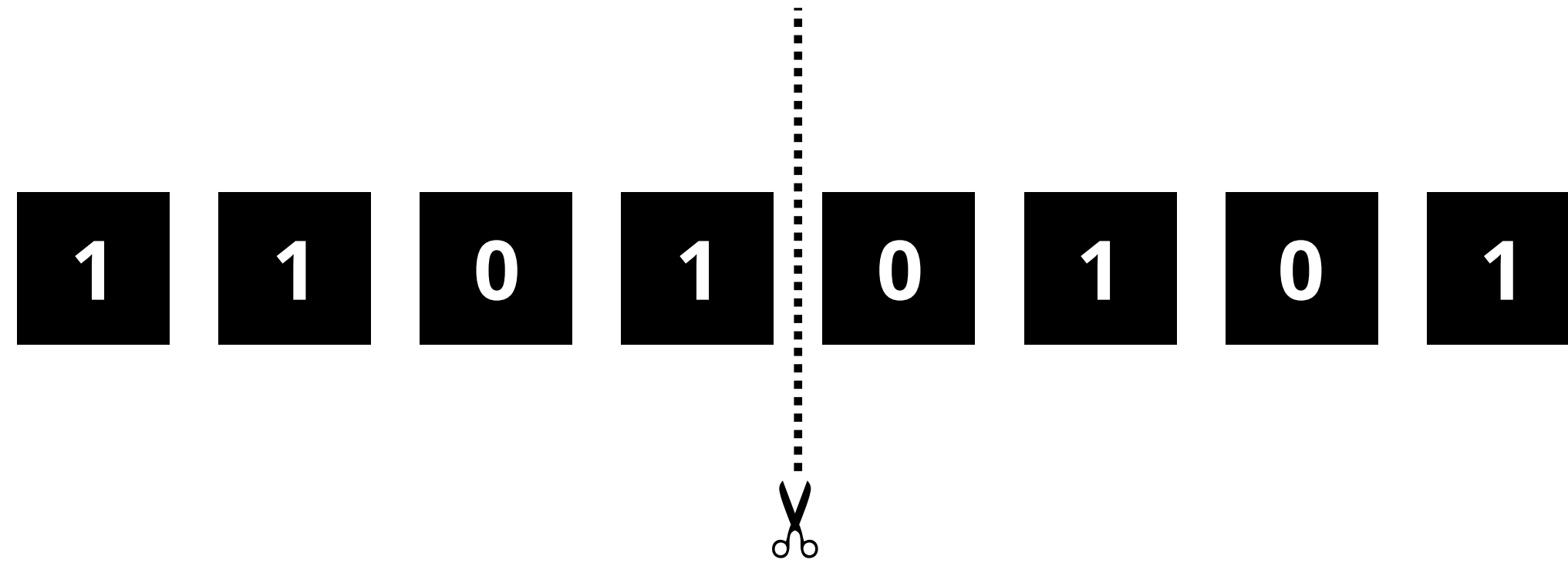
1 1 0 1 0 1 0 1

0 0 1 1 1 0 0 1

# CROSSOVER

---

*parents*



# CROSSOVER

---

*parents*



*offspring*





# MUTATION

---



# MUTATION

---



# MUTATION

---



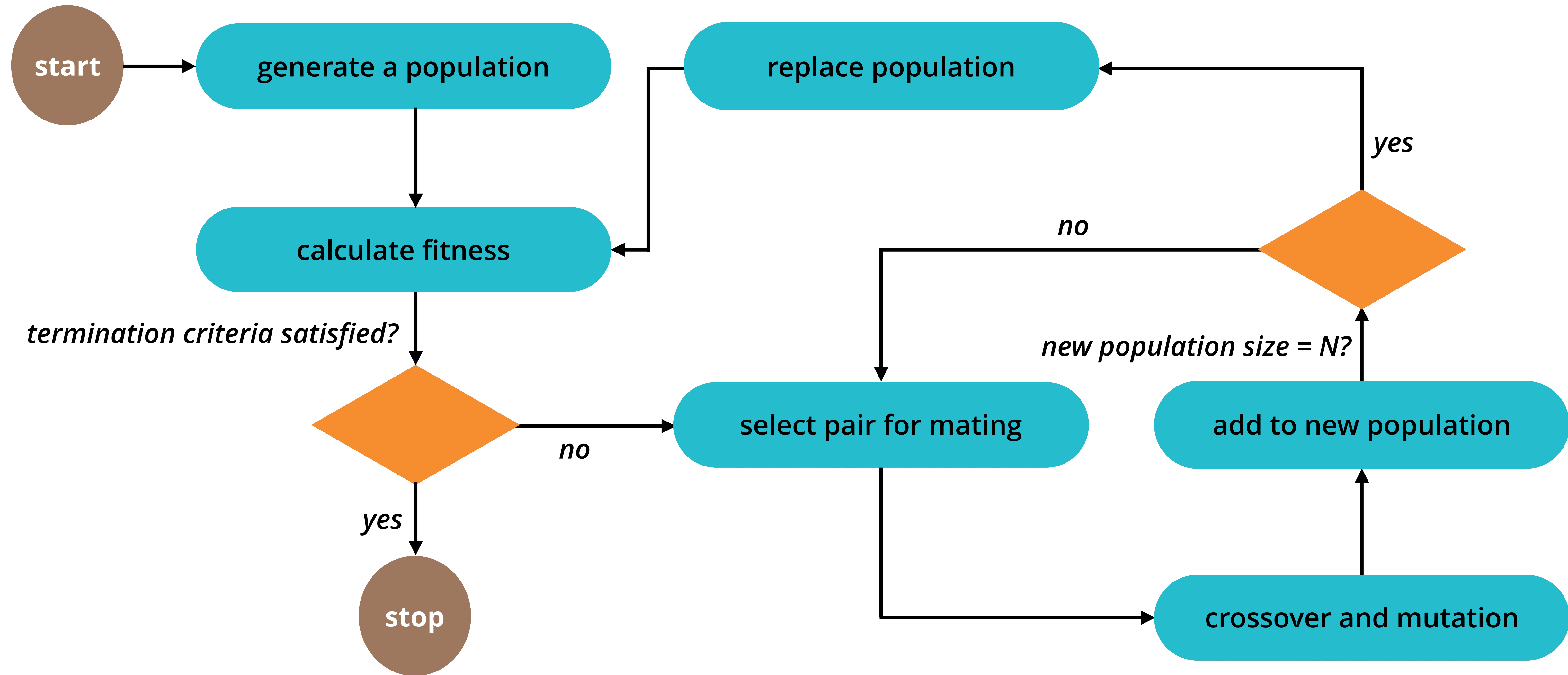


# EVOLUTIONARY ALGORITHM

---

- represent the candidate solution as a chromosome
- define the genetic operators for the chromosome
- define a fitness function to measure the performance of the chromosome
- chose the initial population size  $N$ , crossover probability ( $P_c$ ) and mutation probability ( $P_m$ )

# EVOLUTIONARY ALGORITHM





# TRAVELING SALESMAN

---



A

B

D

C

E

F

G

H

# EVOLUTIONARY ALGORITHM

---

- **represent the candidate solution as a chromosome**
- define the genetic operators for the chromosome
- define a fitness function to measure the performance of the chromosome
- chose the initial population size  $N$ , crossover probability  $P_c$  and mutation probability  $P_m$

# THE CHROMOSOME

---





# EVOLUTIONARY ALGORITHM

---

- represent the candidate solution as a chromosome
- **define the genetic operators for the chromosome**
- define a fitness function to measure the performance of the chromosome
- chose the initial population size  $N$ , crossover probability  $P_c$  and mutation probability  $P_m$

# CROSSOVER

---

*parents*

C B G E H A D F

B E A F H C G D

# CROSSOVER

---

*parents*





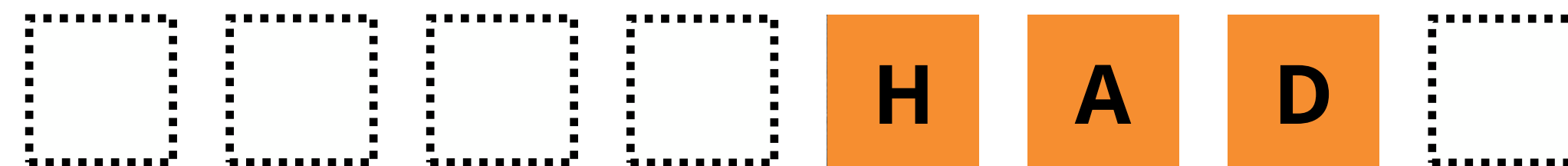
# CROSSOVER

---

*parents*



*offspring*

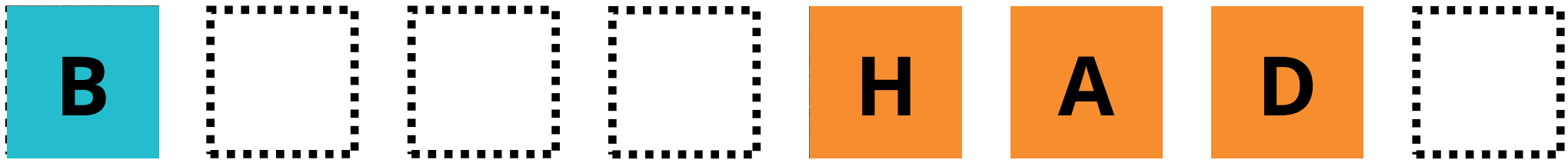


# CROSSOVER

parents



offspring





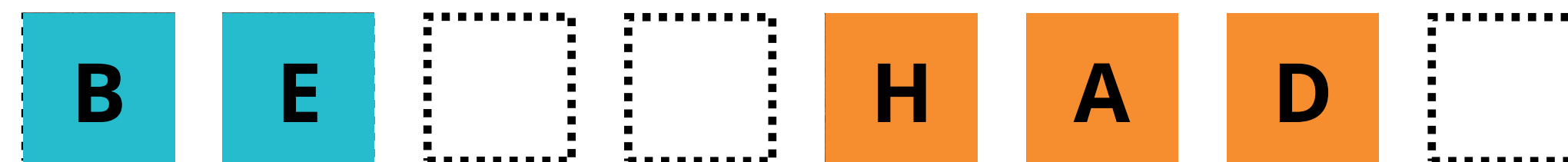
# CROSSOVER

---

*parents*



*offspring*

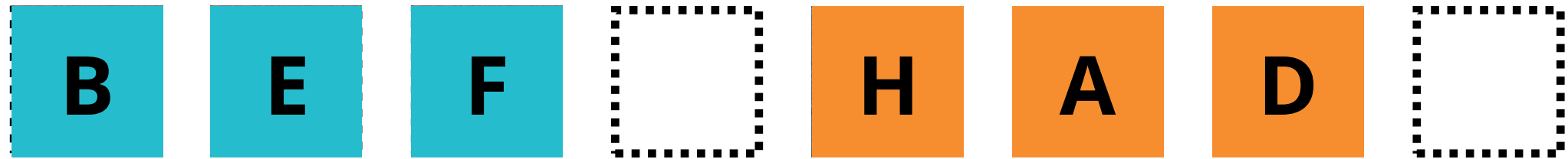


# CROSSOVER

*parents*



*offspring*



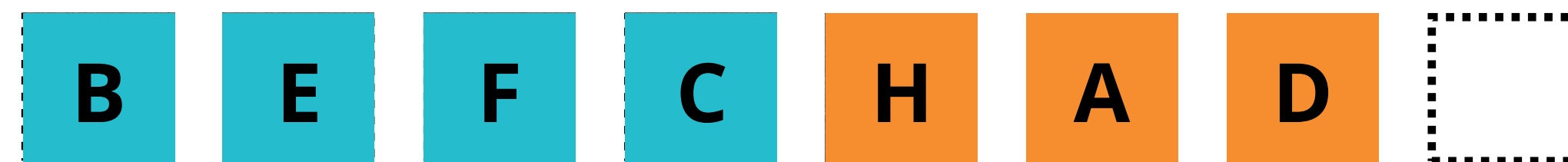
# CROSSOVER

---

*parents*



*offspring*





# CROSSOVER

---

*parents*

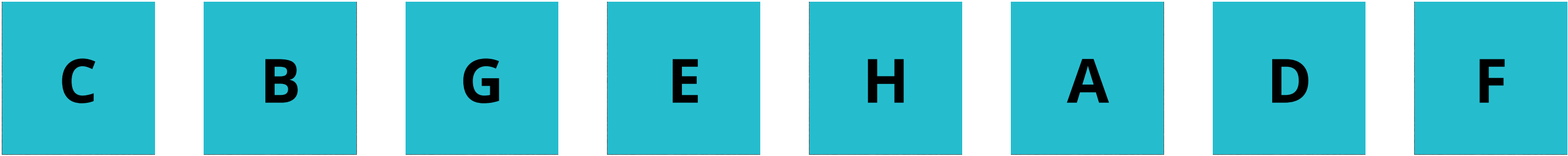


*offspring*



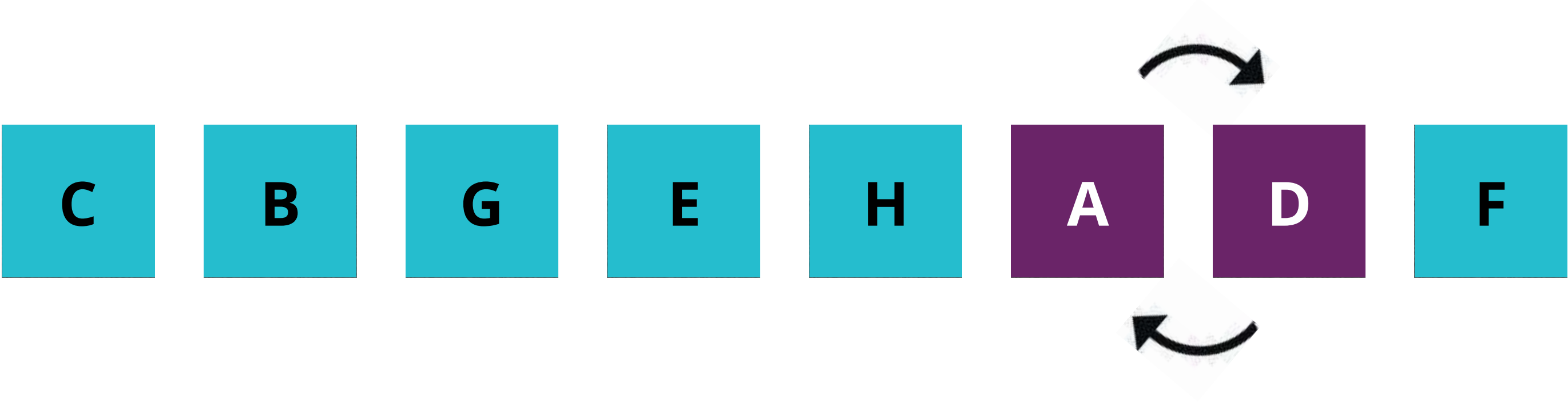
# MUTATION

---



# MUTATION

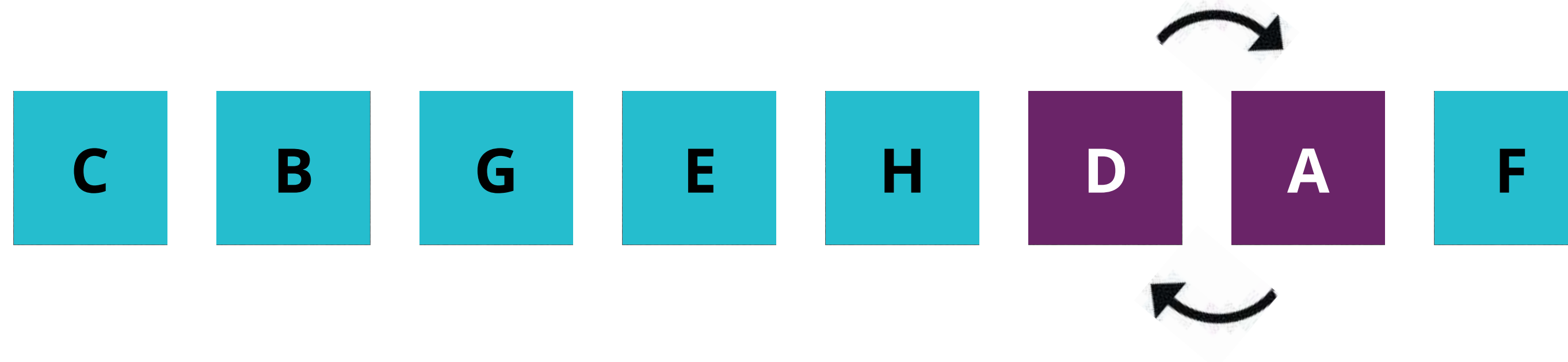
---





# MUTATION

---



# EVOLUTIONARY ALGORITHM

---

- represent the candidate solution as a chromosome
- define the genetic operators for the chromosome
- **define a fitness function to measure the performance of the chromosome**
- chose the initial population size  $N$ , crossover probability  $P_c$  and mutation probability  $P_m$



# FITNESS FUNCTION

---

$$\text{Fitness} = 1 / \text{total distance}$$

# EVOLUTIONARY ALGORITHM

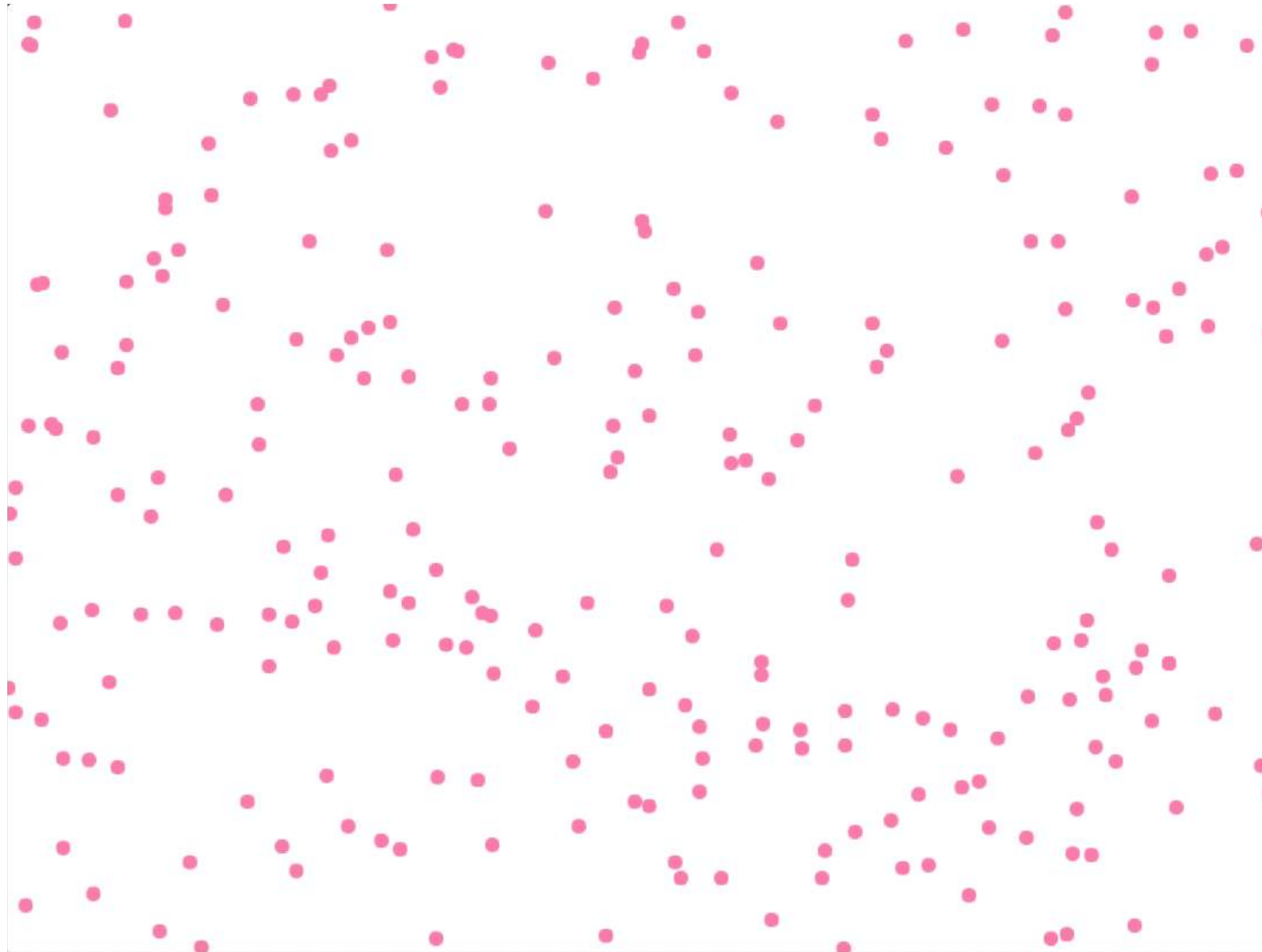
---

- represent the candidate solution as a chromosome
- define a fitness function to measure the performance of the chromosome
- define the genetic operators for the chromosome
- **chose the initial population size  $N$ , crossover probability  $P_c$  and mutation probability  $P_m$**

# DEMO

---

There are 250 Cities in the map.

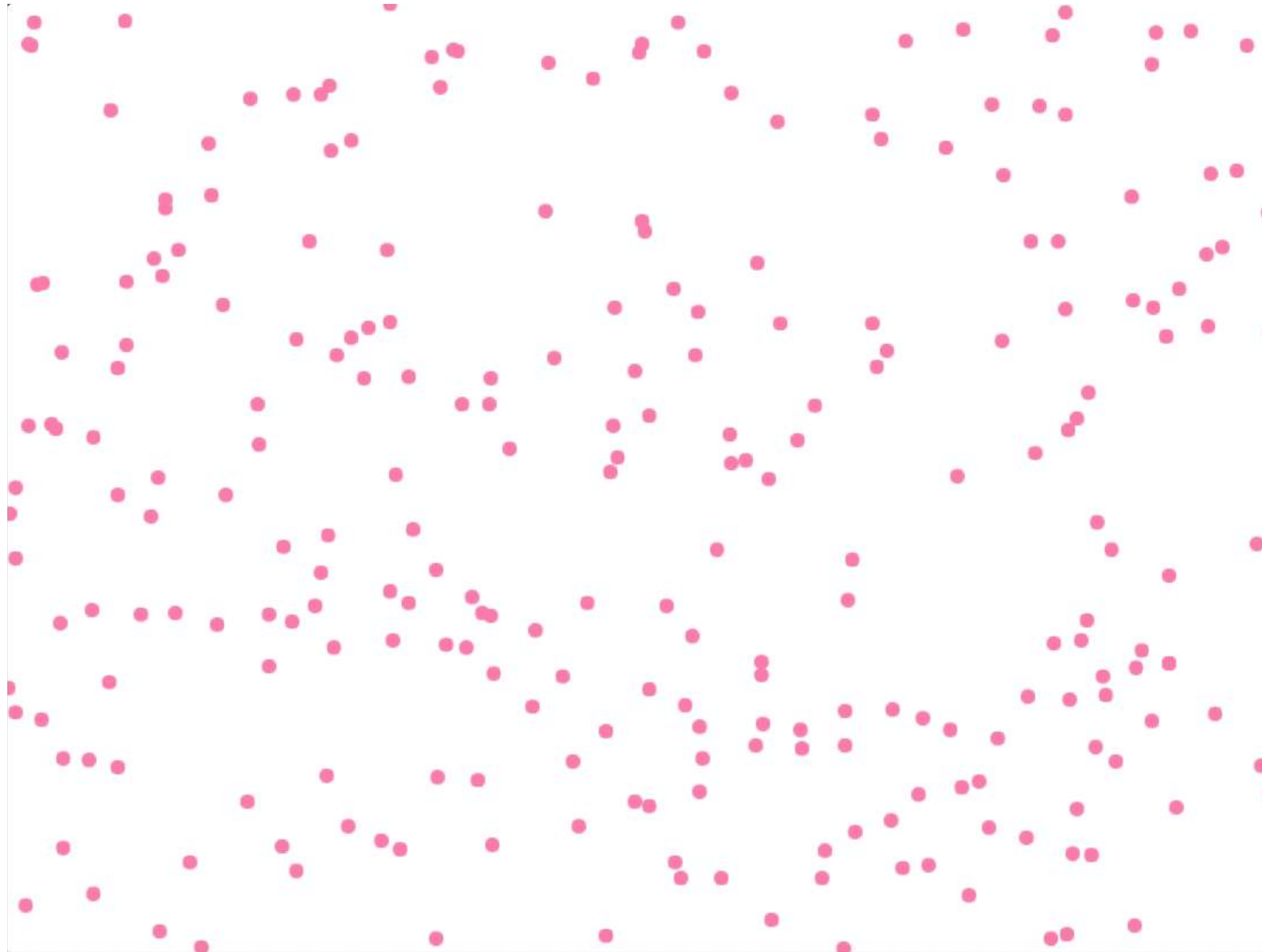




# DEMO

---

There are 250 Cities in the map.







# NEUROEVOLUTION

---



# MEET GENE SHER

---



# SIMULATION

---

- sensors (provide input for the network)
- actuators (act upon output from the network)
- fitness function



# CLEANING ROBOT

---







## AND LOTS MORE...

---

- data compression
- training NPCs in a video game
- cyber warfare

# EVOLUTIONARY ALGORITHM

---

- represent the candidate solution as a chromosome
- define the genetic operators for the chromosome
- define a fitness function to measure the performance of the chromosome
- chose the initial population size  $N$ , crossover probability ( $P_c$ ) and mutation probability ( $P_m$ )

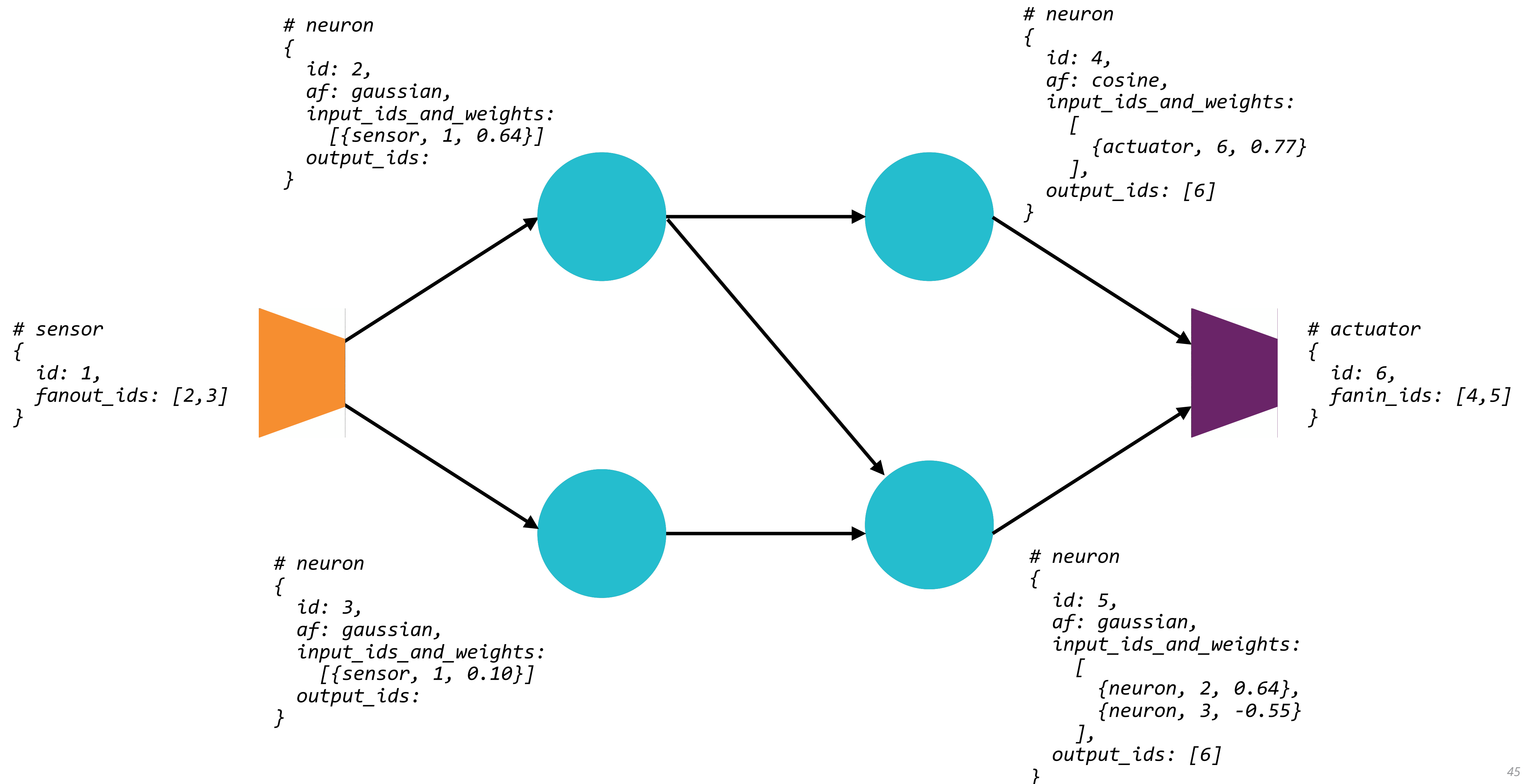


# EVOLUTIONARY ALGORITHM

---

- **represent the candidate solution as a chromosome**
- define the genetic operators for the chromosome
- define a fitness function to measure the performance of the chromosome
- chose the initial population size  $N$ , crossover probability  $P_c$  and mutation probability  $P_m$

# THE CHROMOSOME



# EVOLUTIONARY ALGORITHM

---

- represent the candidate solution as a chromosome
- **define the genetic operators for the chromosome**
- define a fitness function to measure the performance of the chromosome
- chose the initial population size  $N$ , crossover probability  $P_c$  and mutation probability  $P_m$



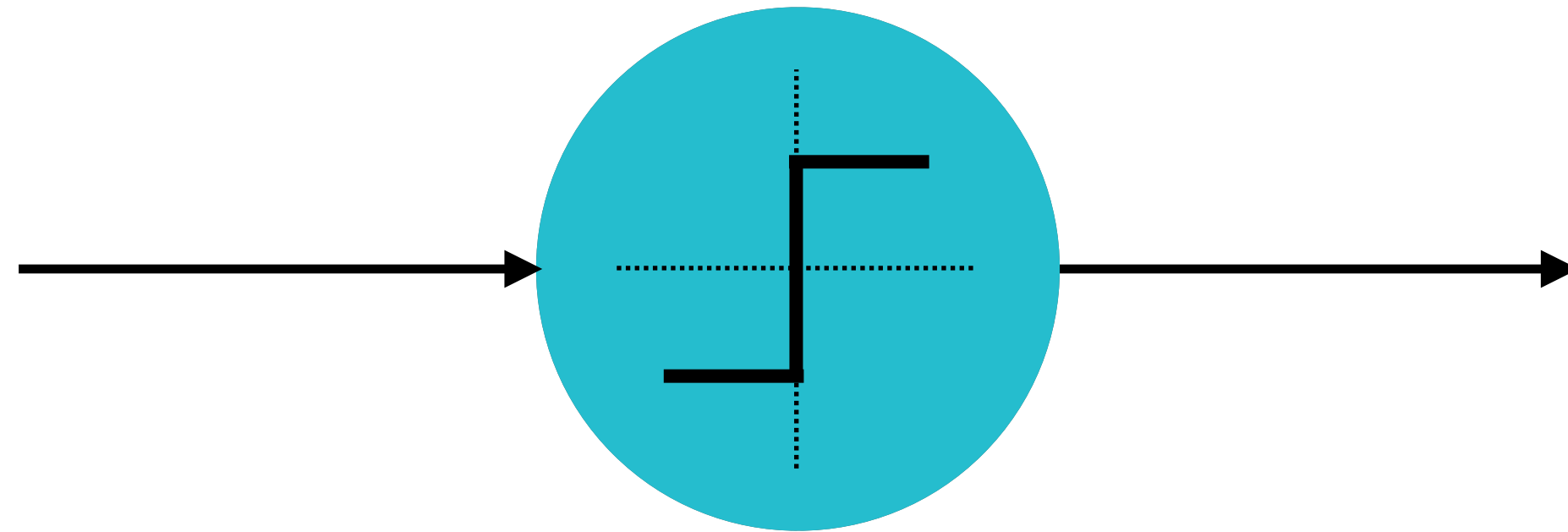
# CROSSOVER

---

Crossover doesn't work for large neural nets!

# MUTATE ACTIVATION FUNCTION

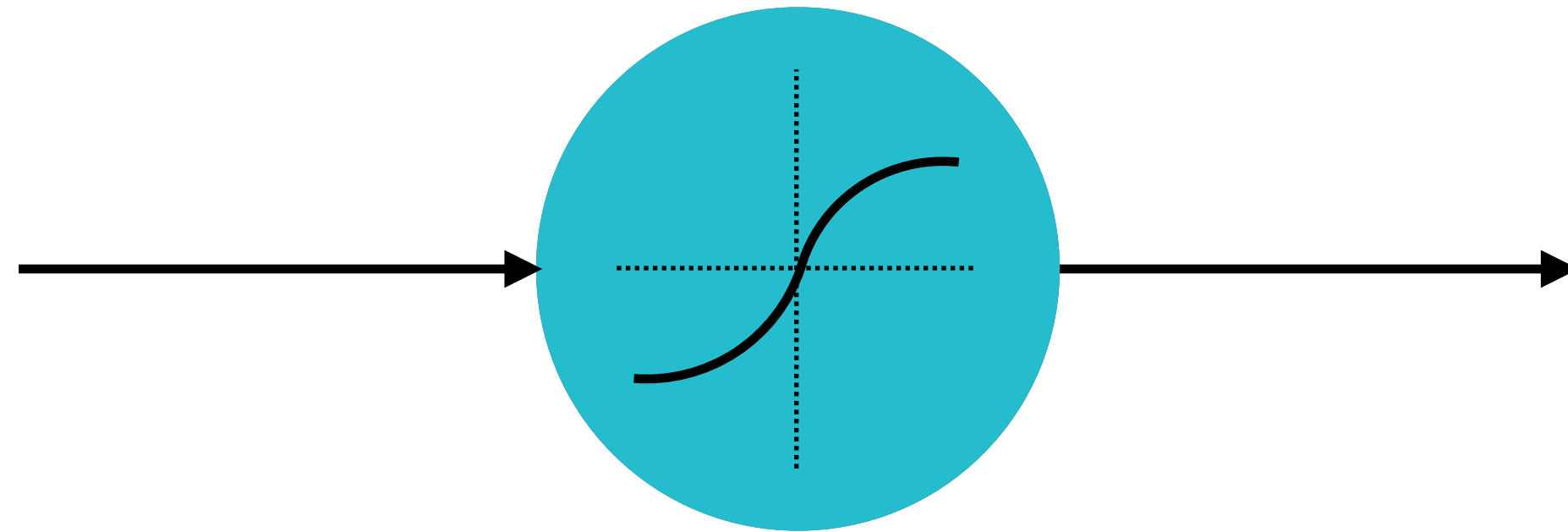
---





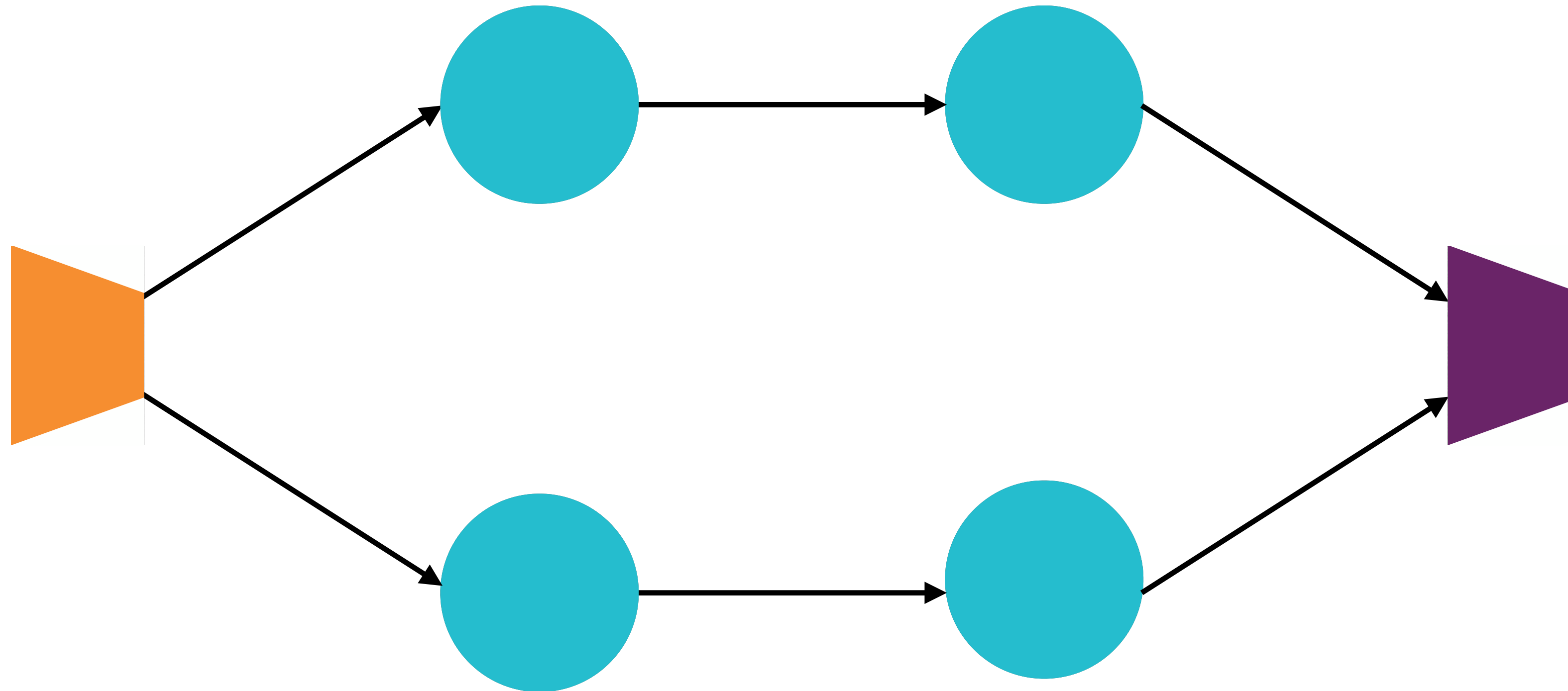
# MUTATE ACTIVATION FUNCTION

---



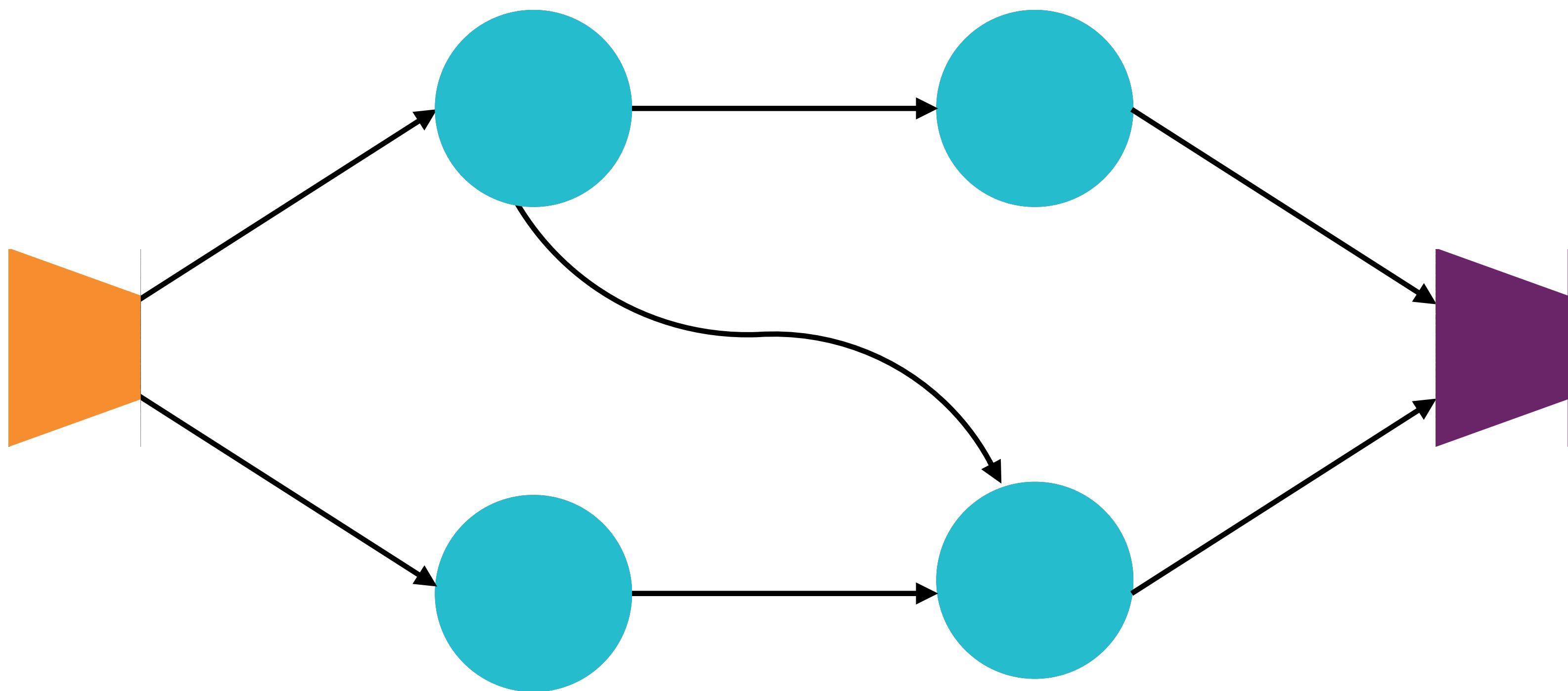
# ADD CONNECTION

---



# ADD CONNECTION

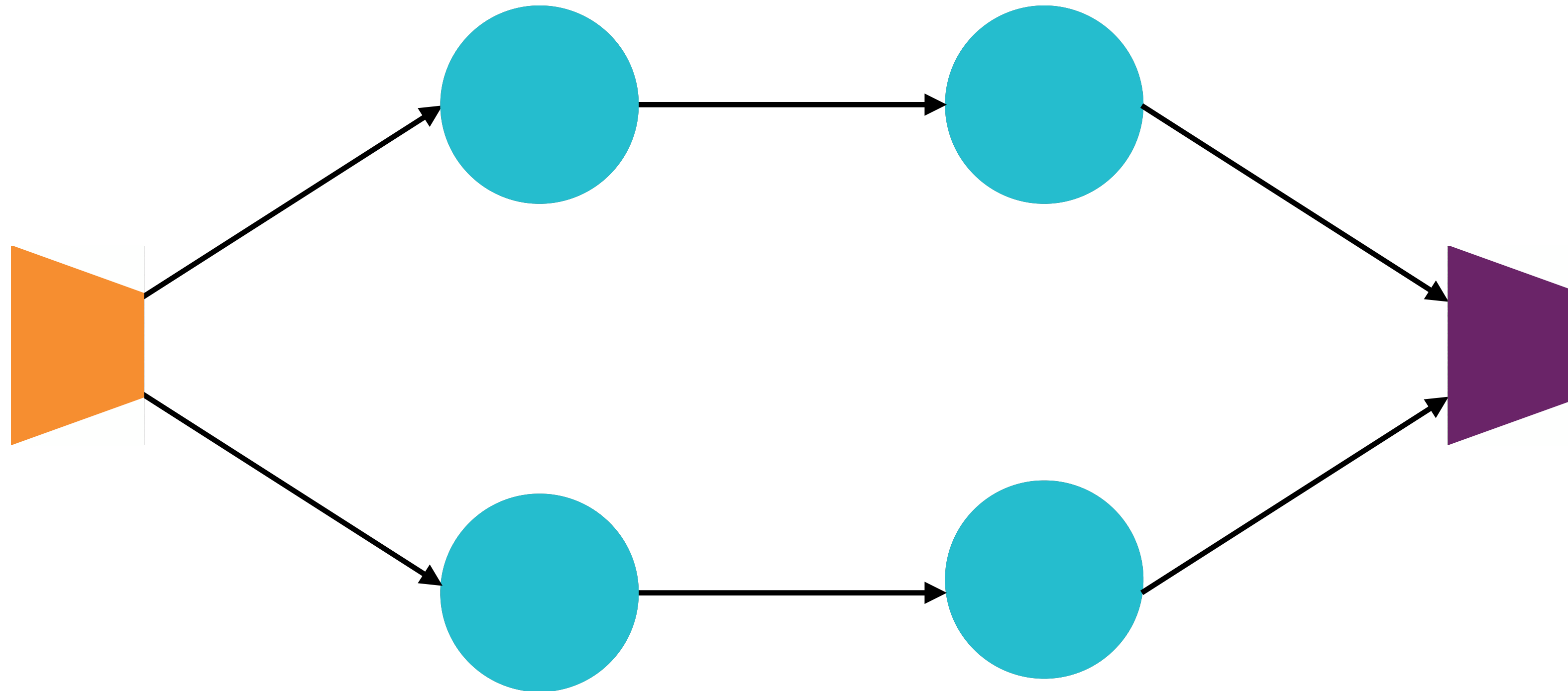
---





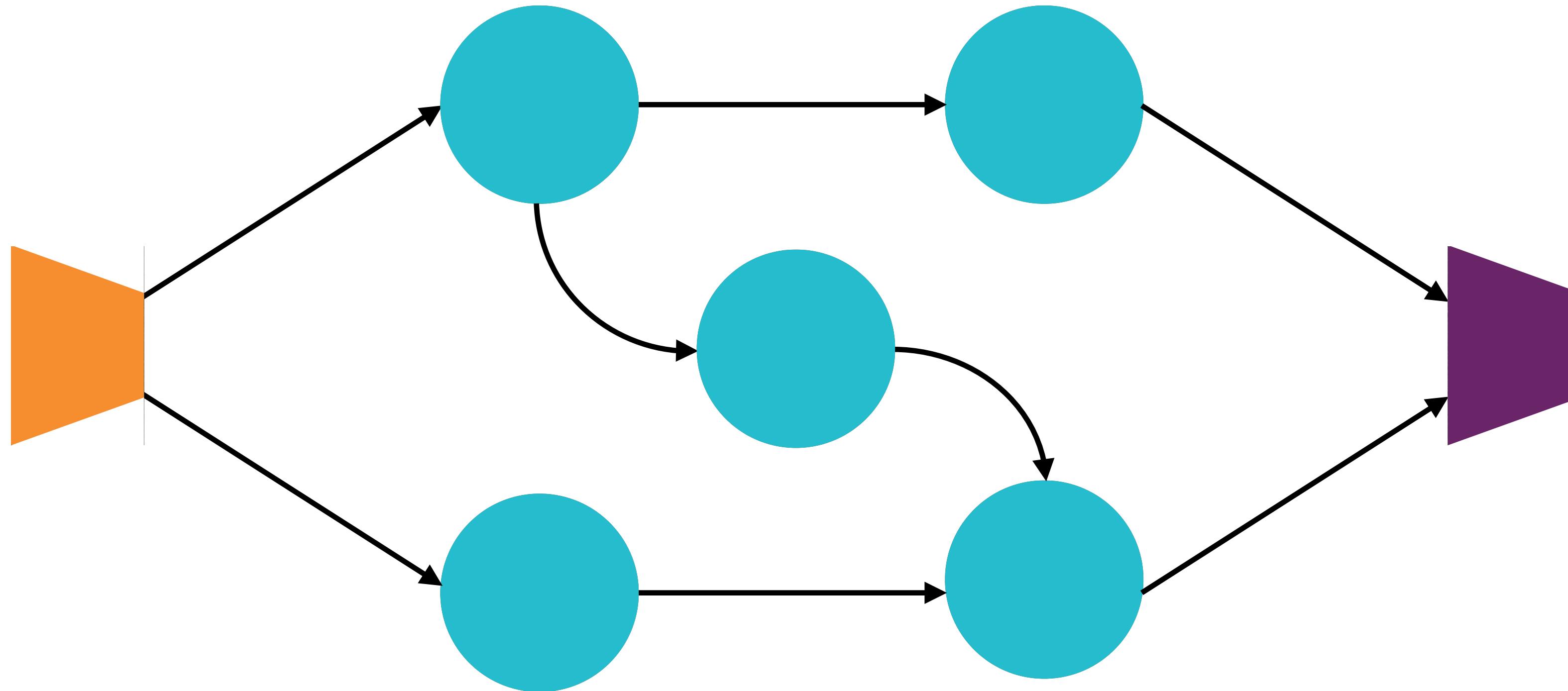
# ADD NEURON

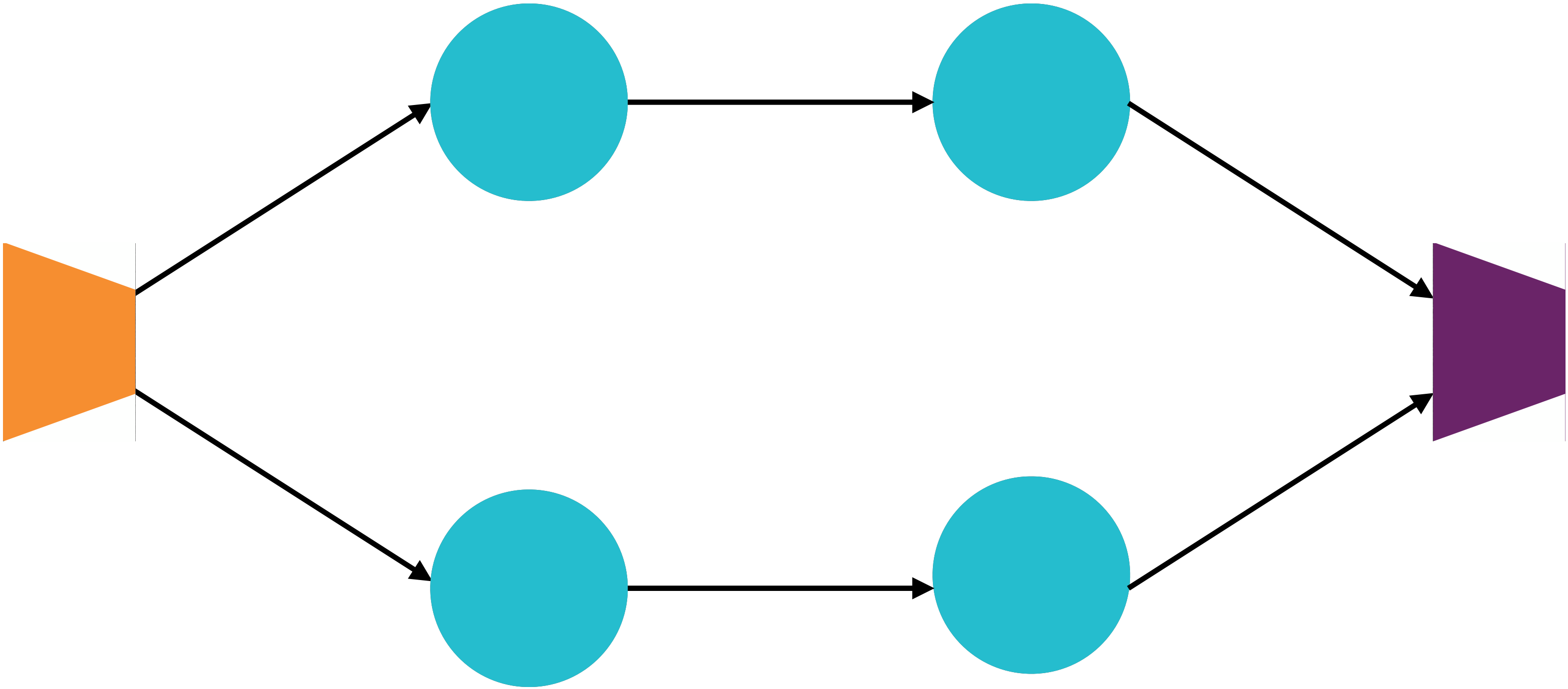
---



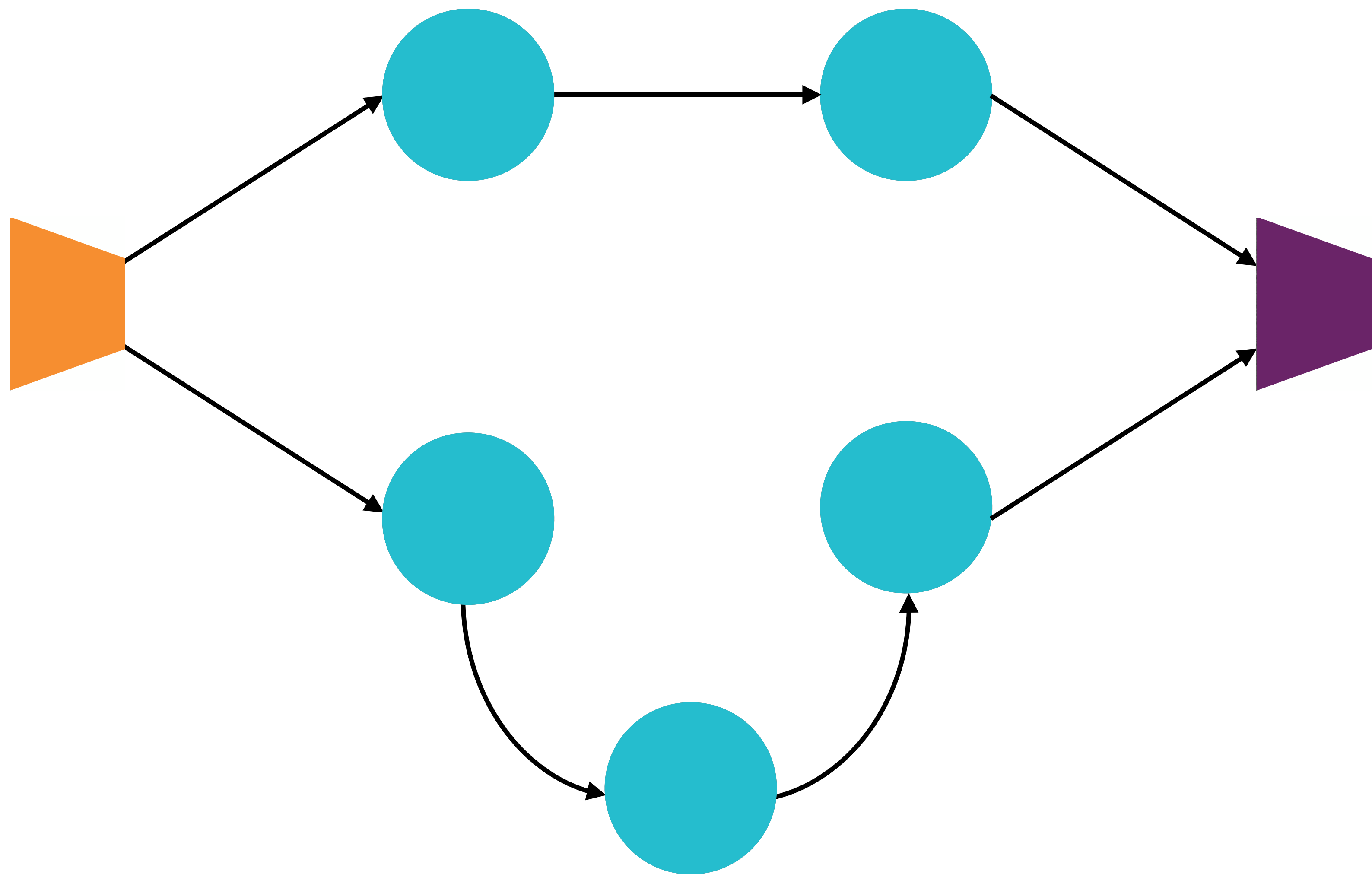
# ADD NEURON

---











# MUTATION OPERATORS

---

*and lots more...*

# EVOLUTIONARY ALGORITHM

---

- represent the candidate solution as a chromosome
- define the genetic operators for the chromosome
- **define a fitness function to measure the performance of the chromosome**
- chose the initial population size  $N$ , crossover probability  $P_c$  and mutation probability  $P_m$

# FITNESS FUNCTION

---

Fitness = performance of network on an actual problem



# EVOLUTIONARY ALGORITHM

---

- represent the candidate solution as a chromosome
- define the genetic operators for the chromosome
- define a fitness function to measure the performance of the chromosome
- **chose the initial population size  $N$ , crossover probability  $P_c$  and mutation probability  $P_m$**

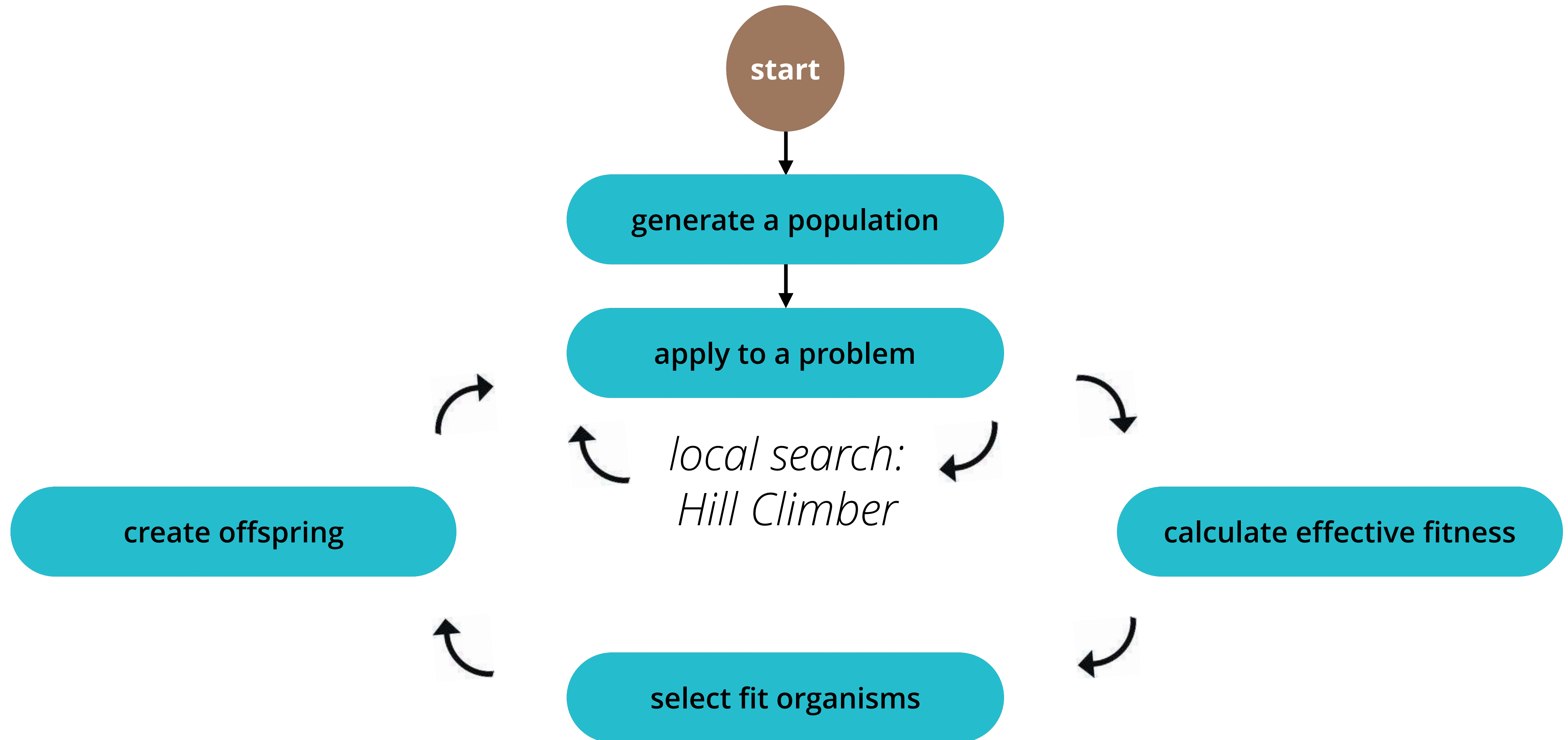
## RANDOM INTENSITY MUTATION

---

*Number of mutations = random( $1, \sqrt{\text{network size}}$ )*

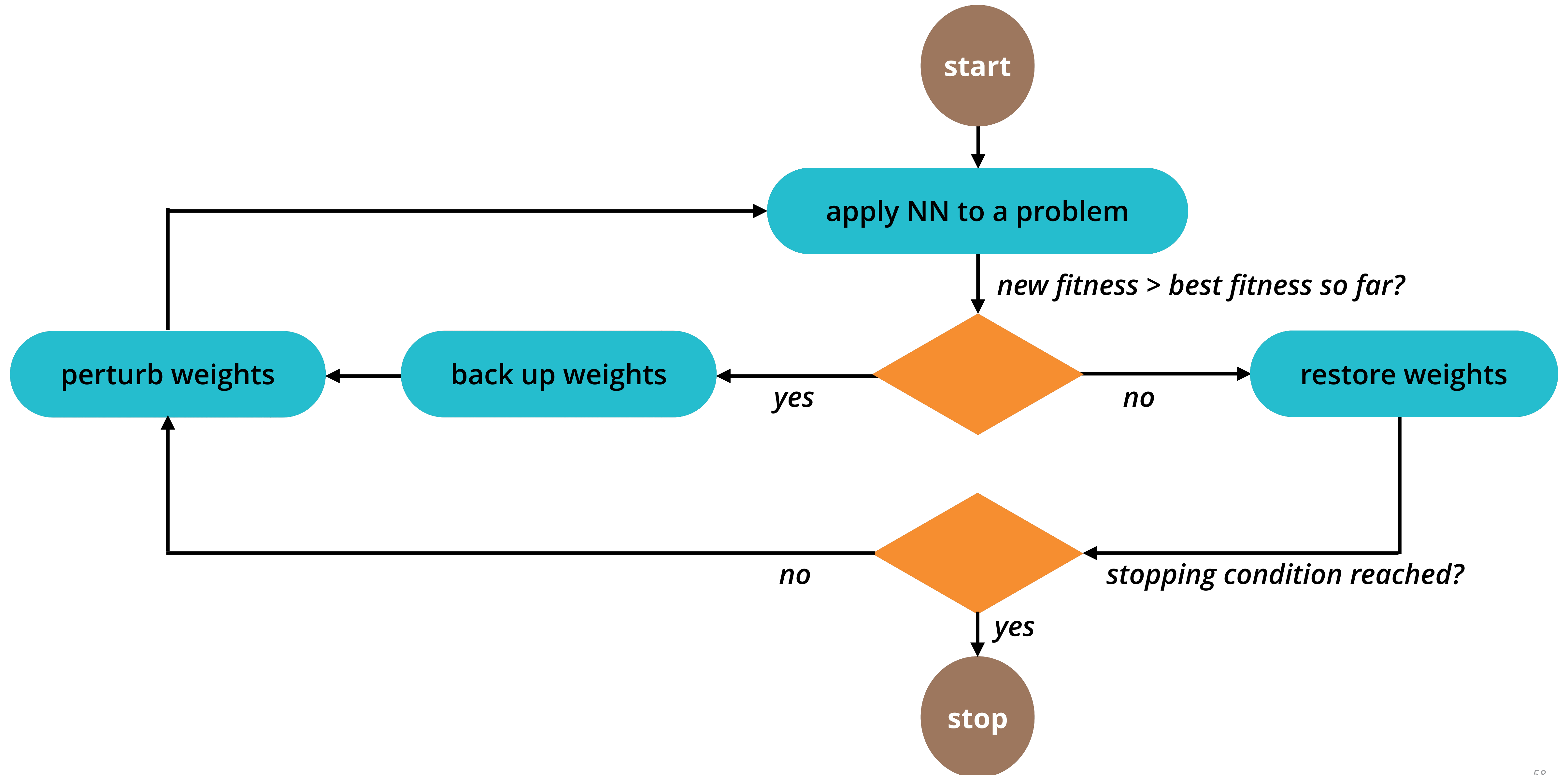
# MEMETIC ALGORITHM

---





# STOCHASTIC HILL CLIMBER (LOCAL SEARCH)



# MEET JOE ARMSTRONG

---

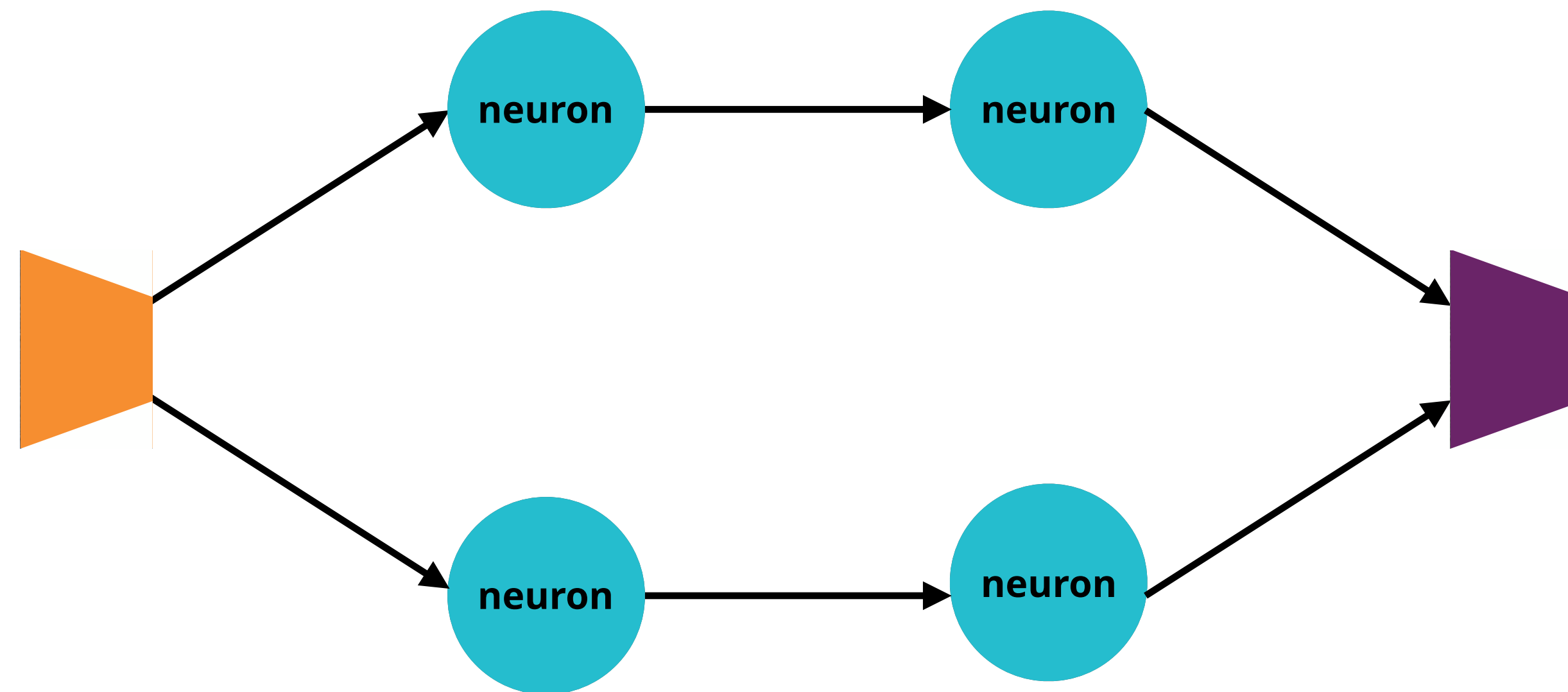




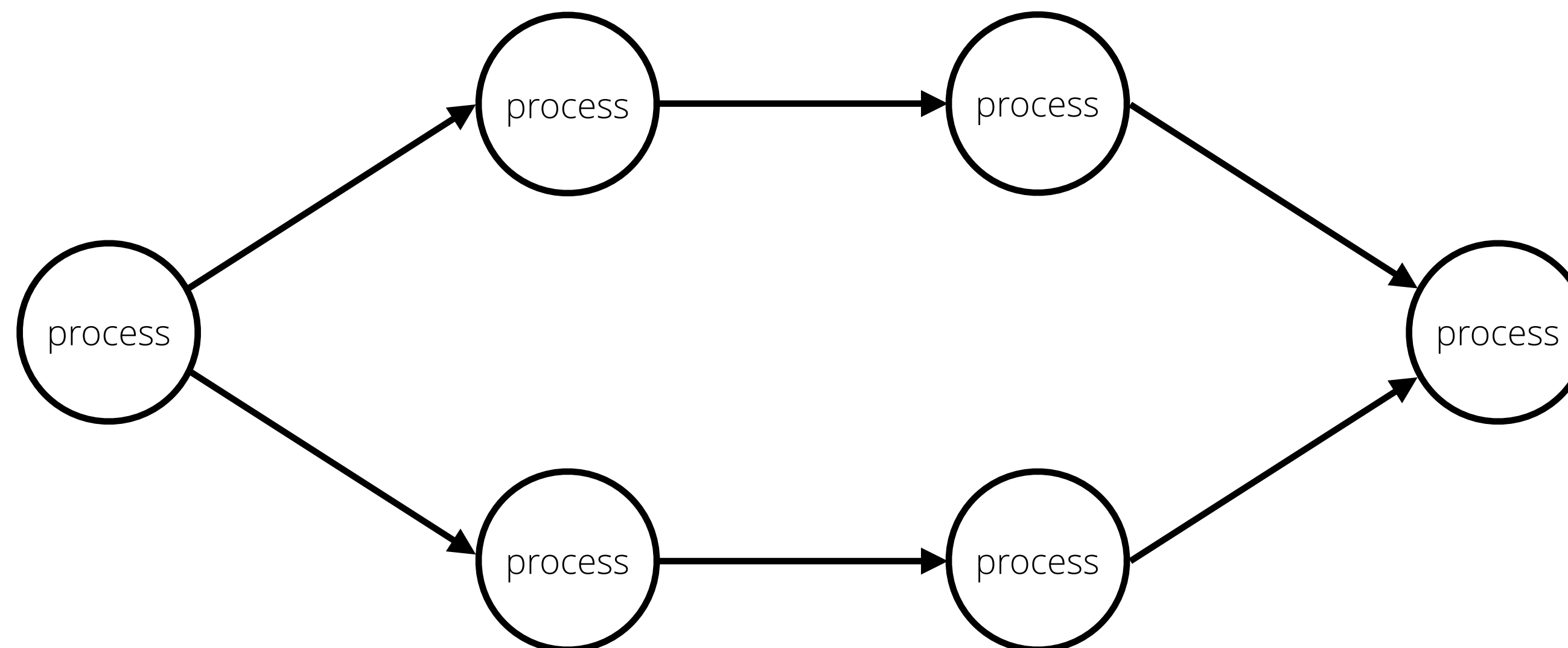
# 1:1 MAPPING

---

*genotype*

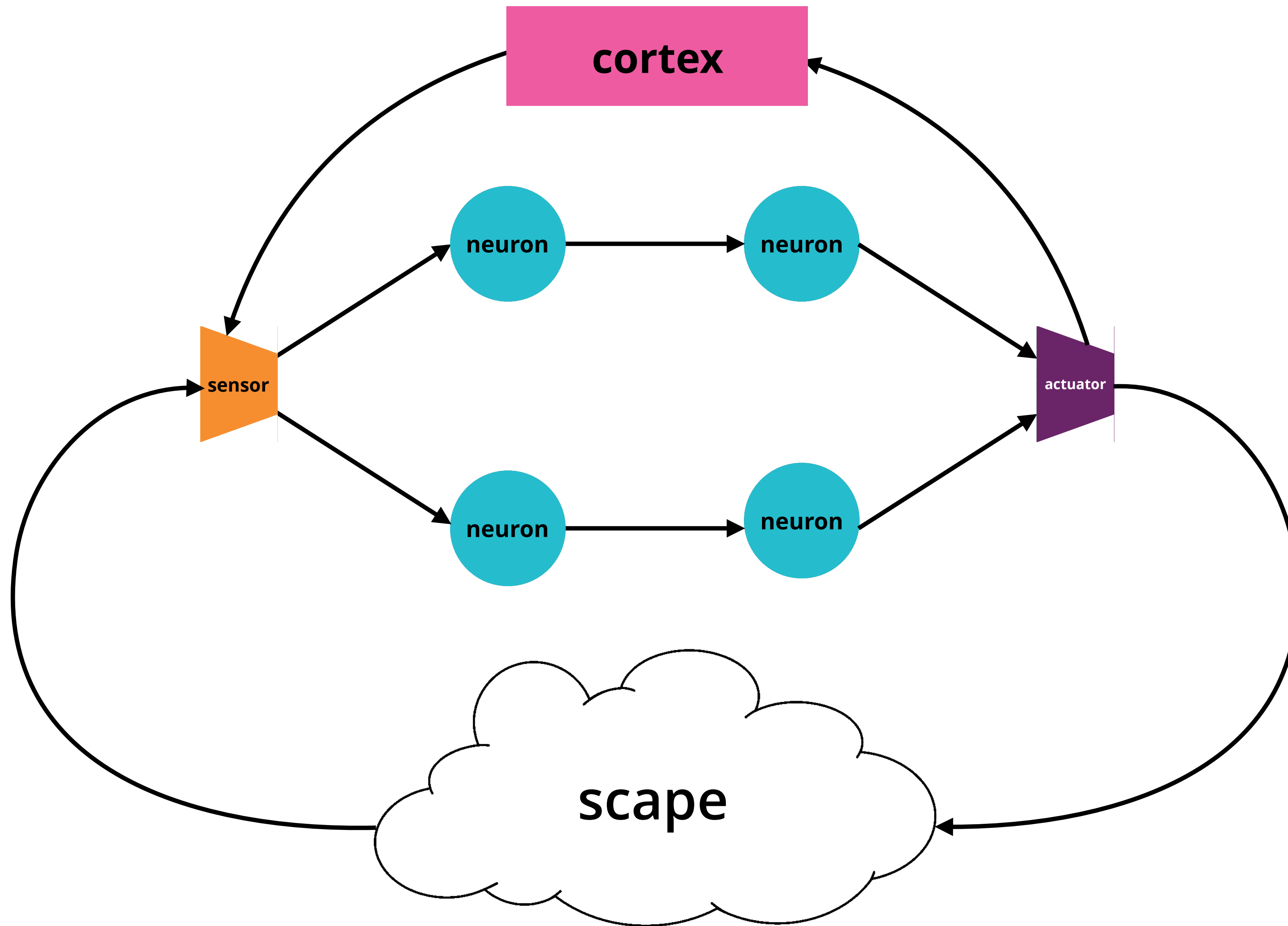


*phenotype /  
erlang*



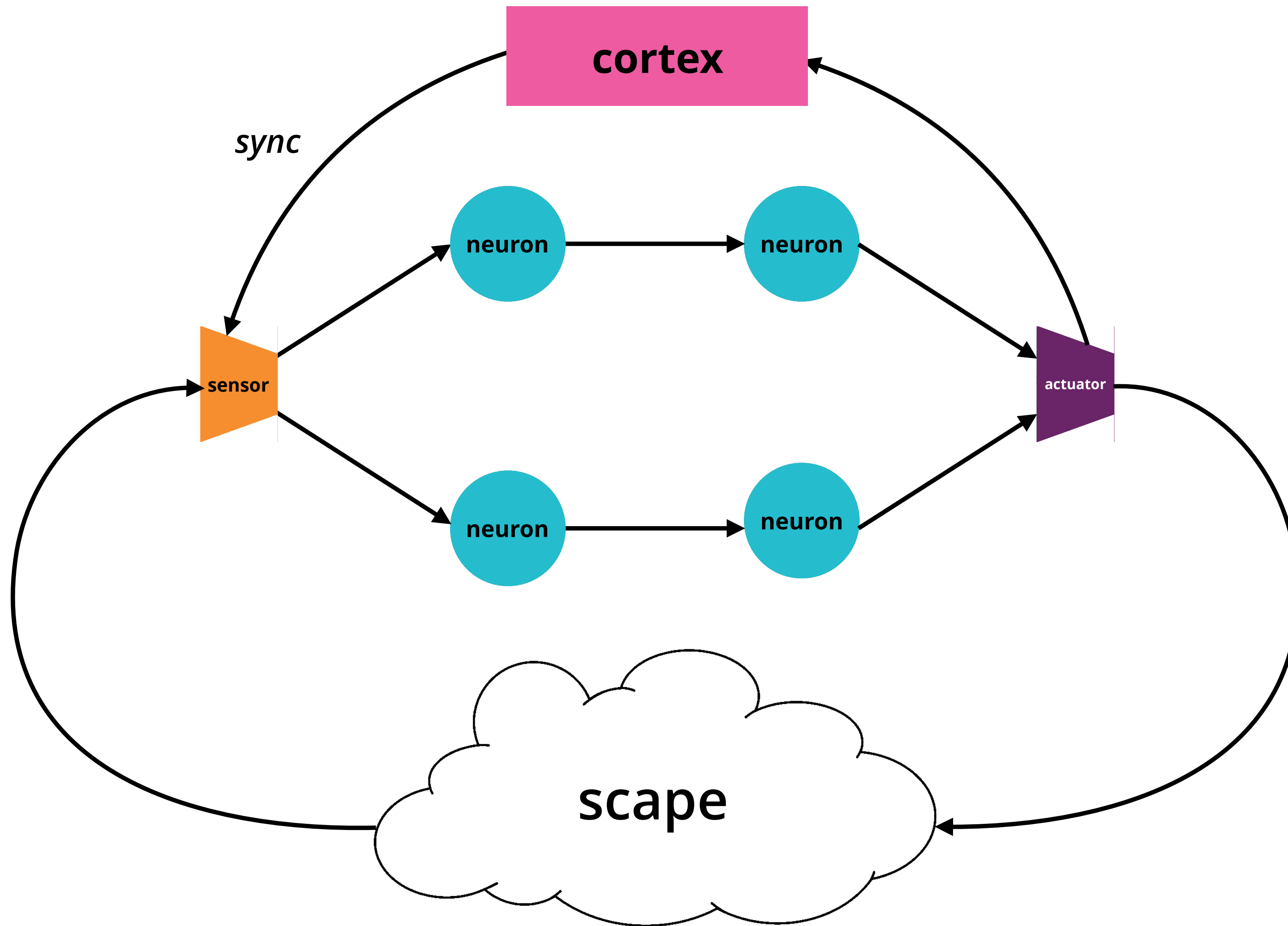
# THE NEURAL NETWORK

---



# THE NEURAL NETWORK

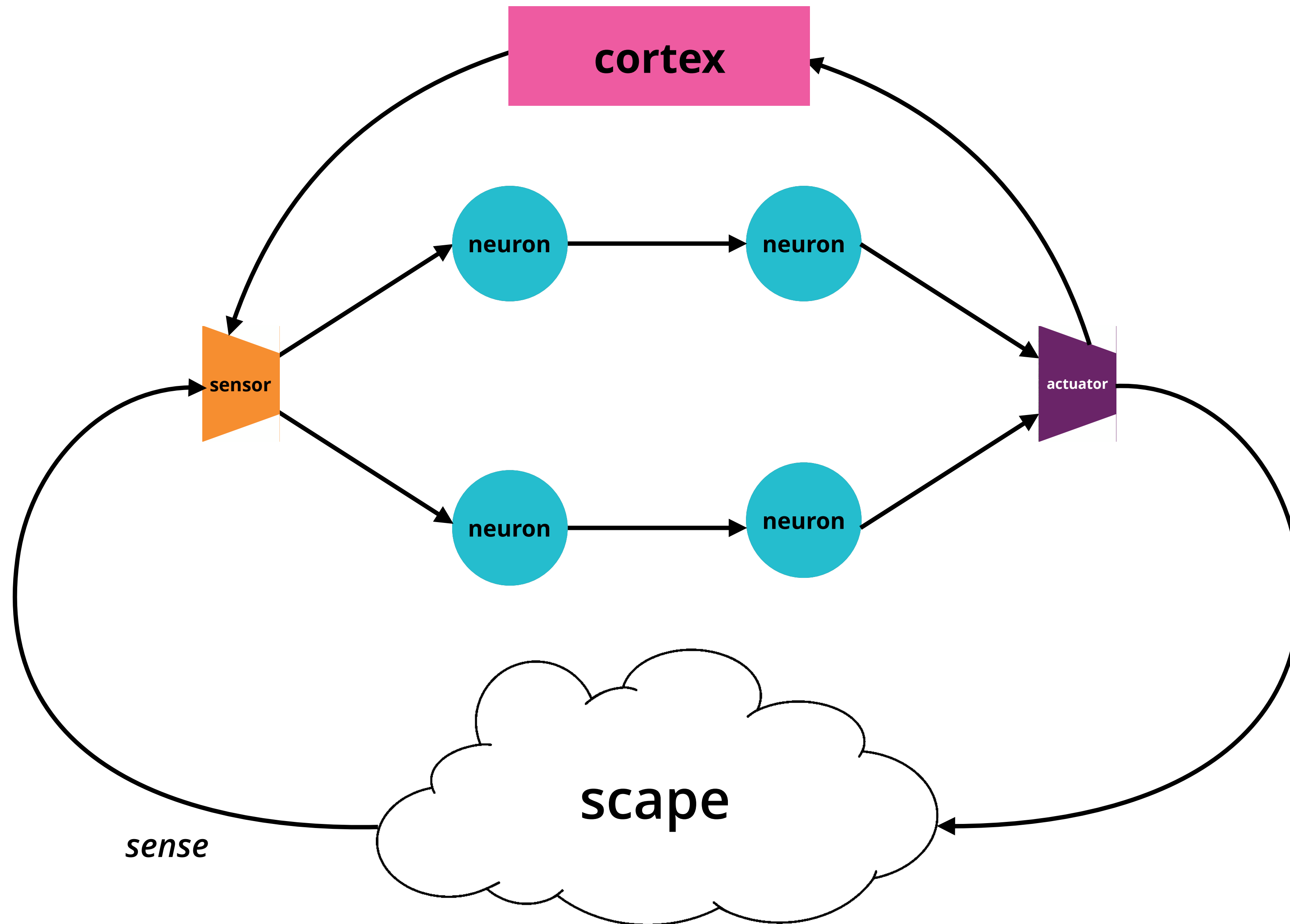
---





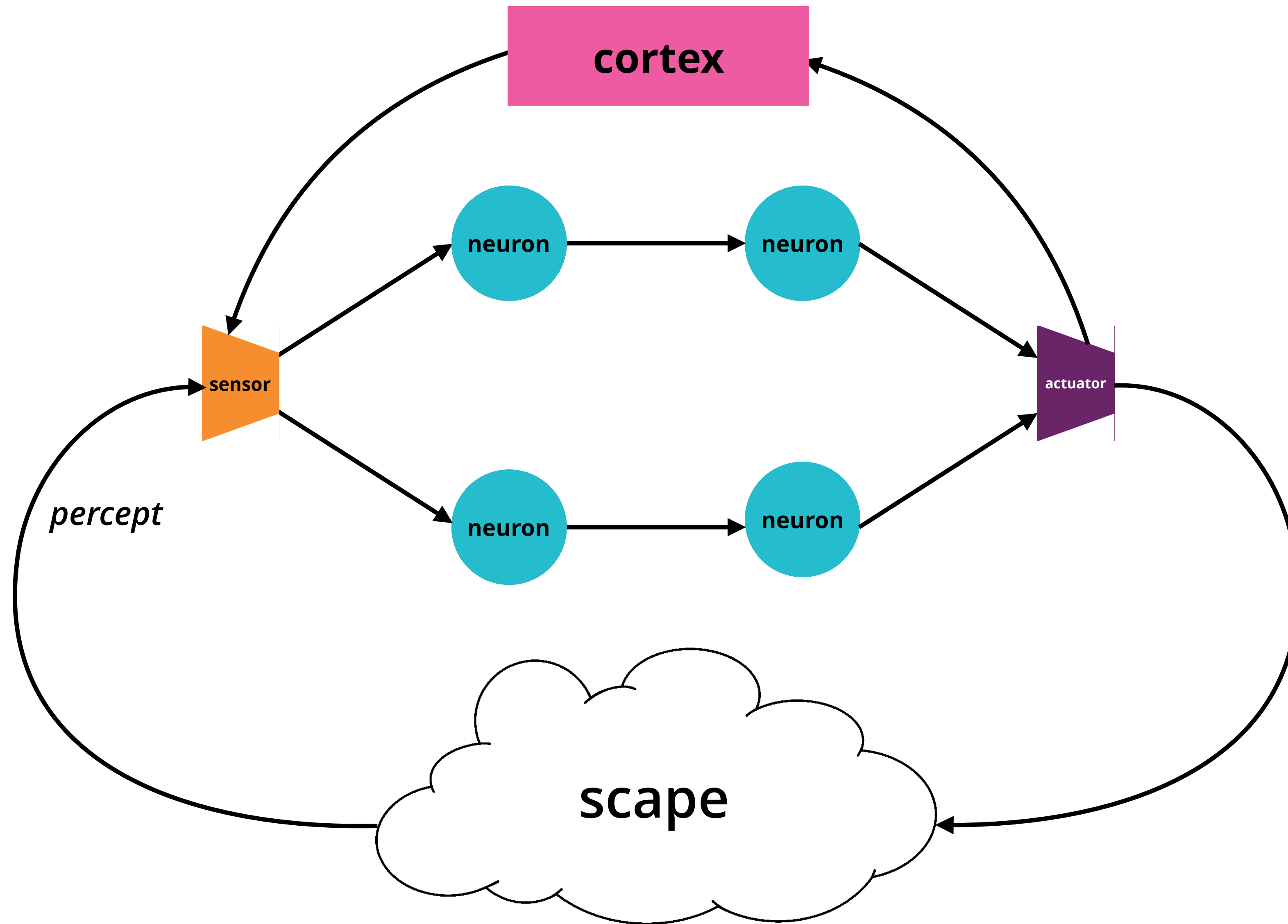
# THE NEURAL NETWORK

---



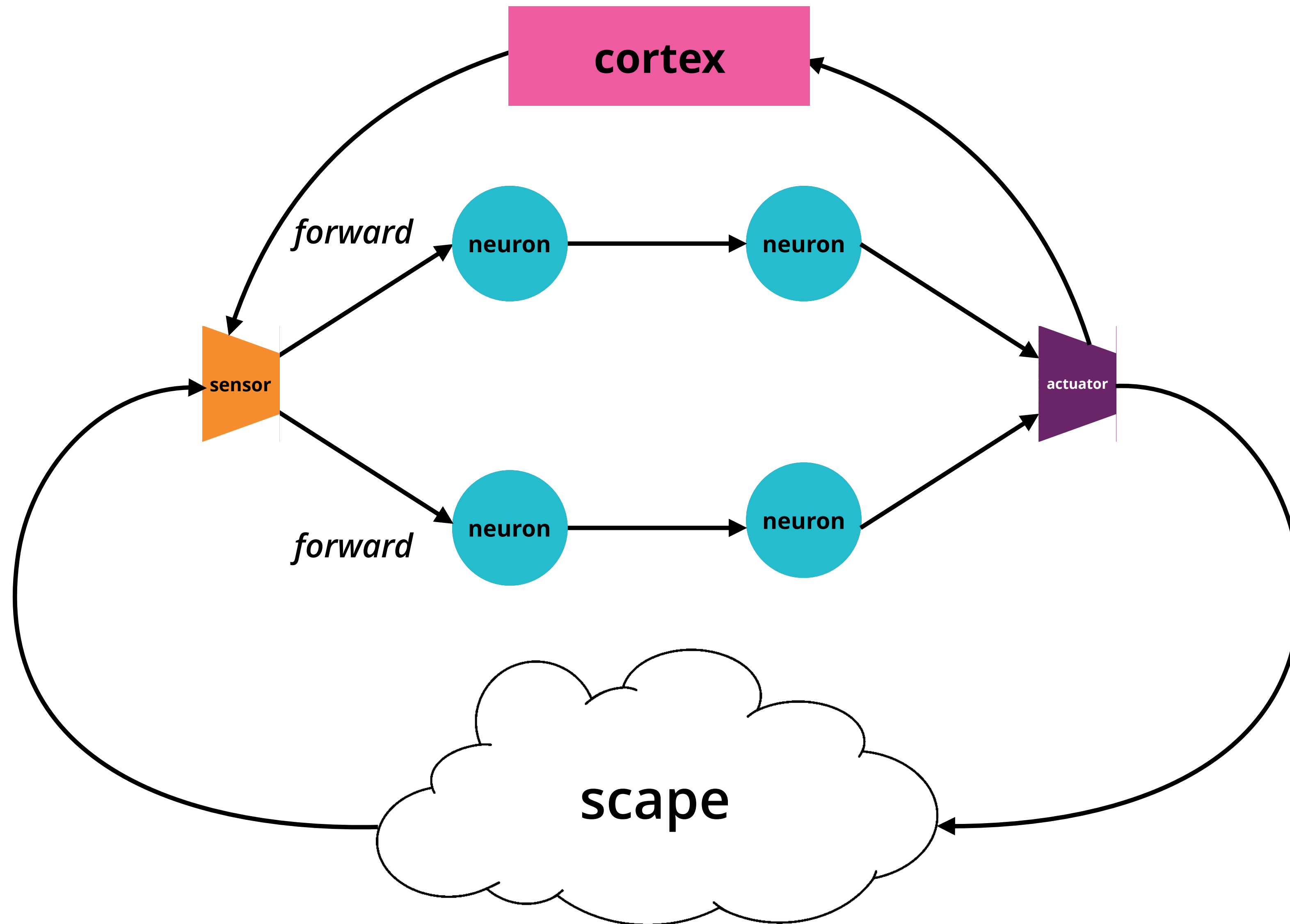
# THE NEURAL NETWORK

---



# THE NEURAL NETWORK

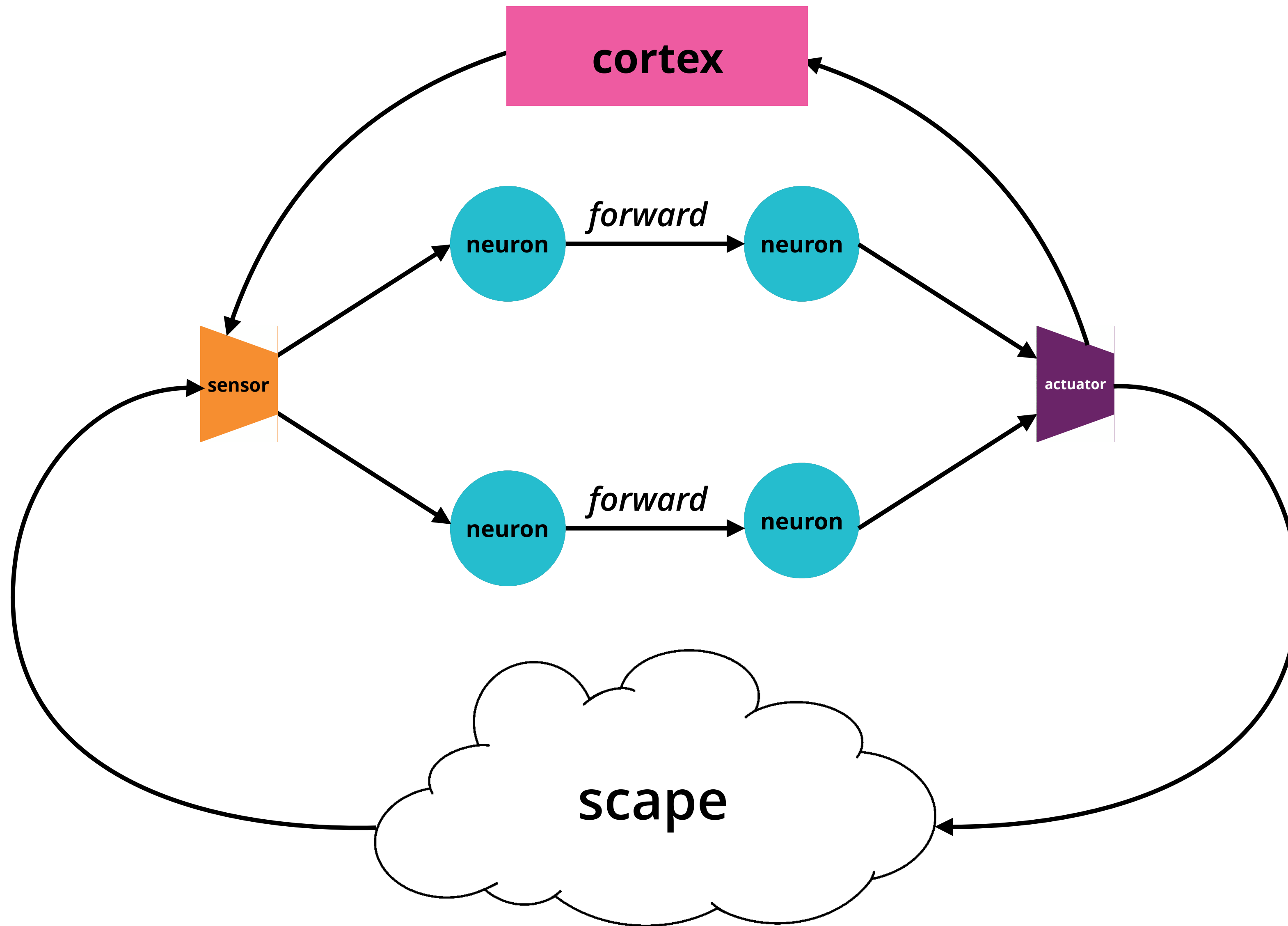
---





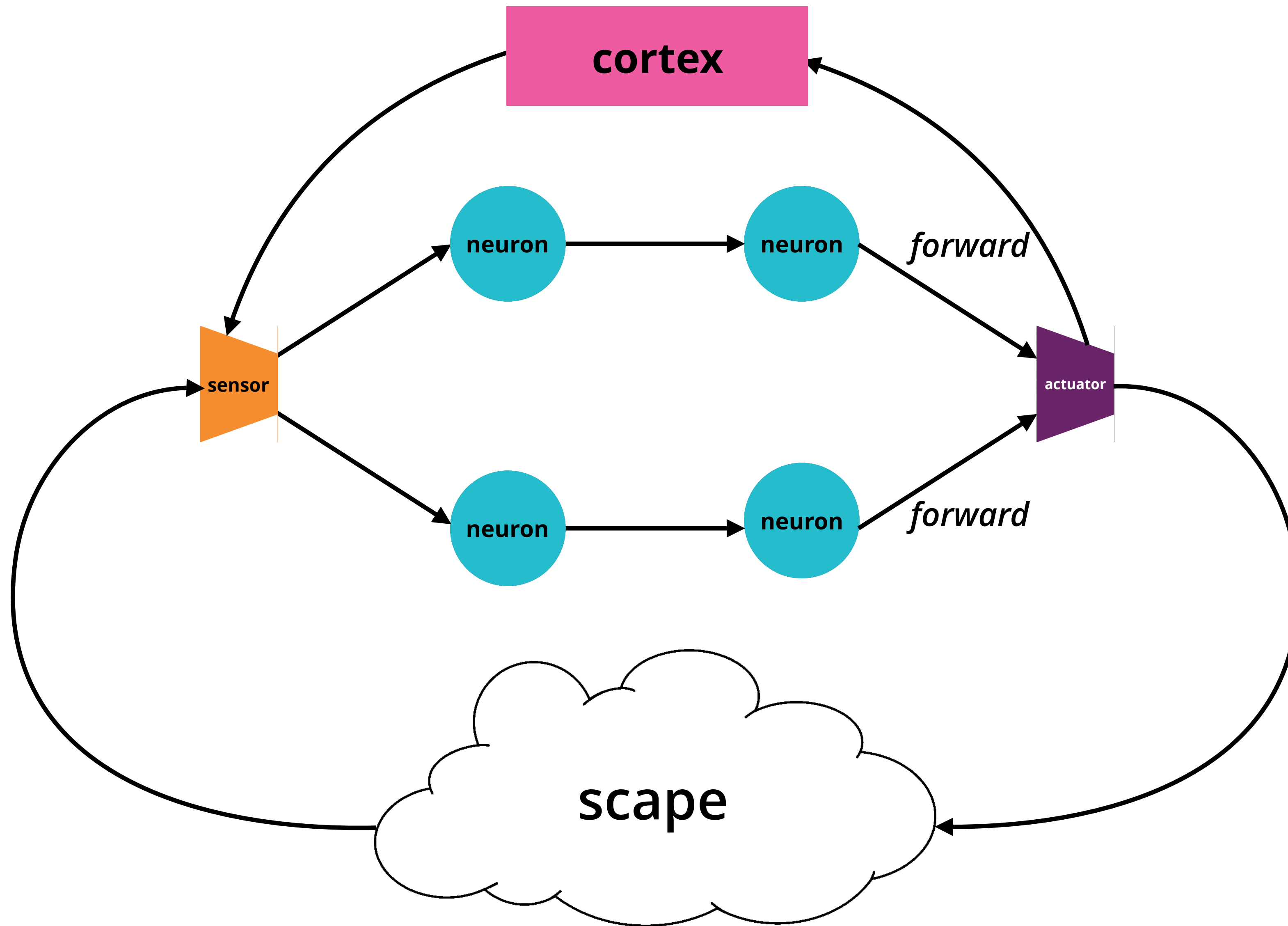
# THE NEURAL NETWORK

---



# THE NEURAL NETWORK

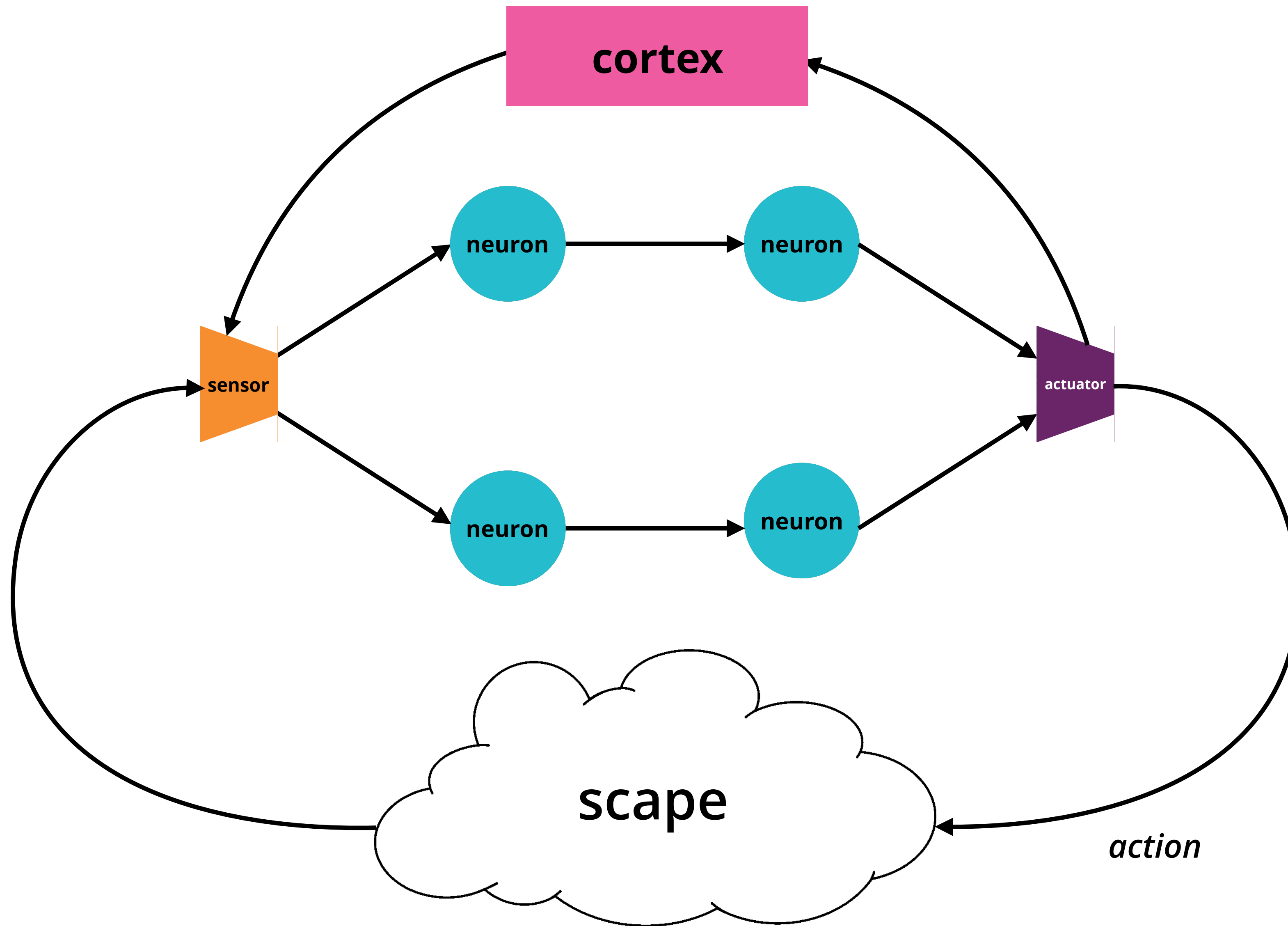
---





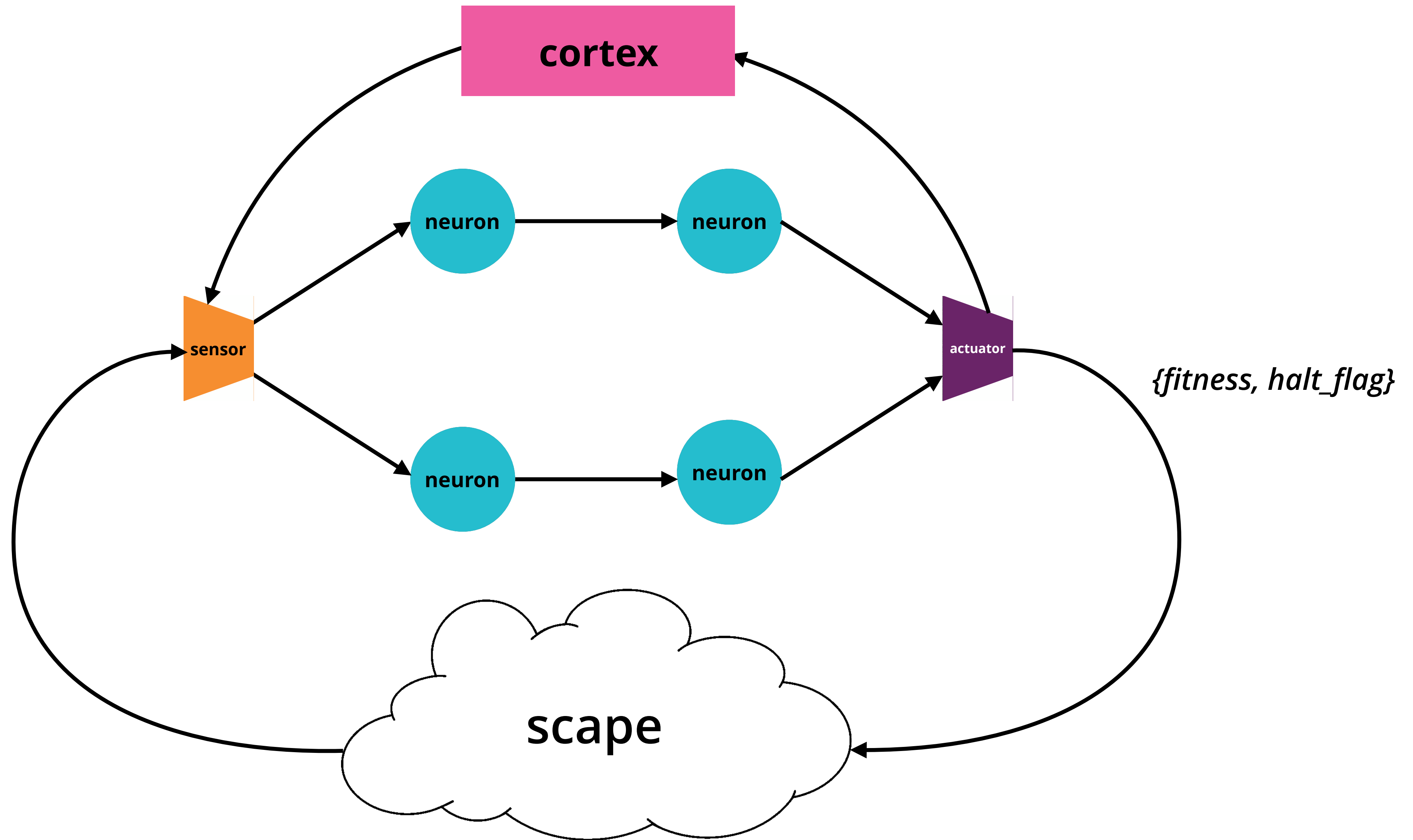
# THE NEURAL NETWORK

---



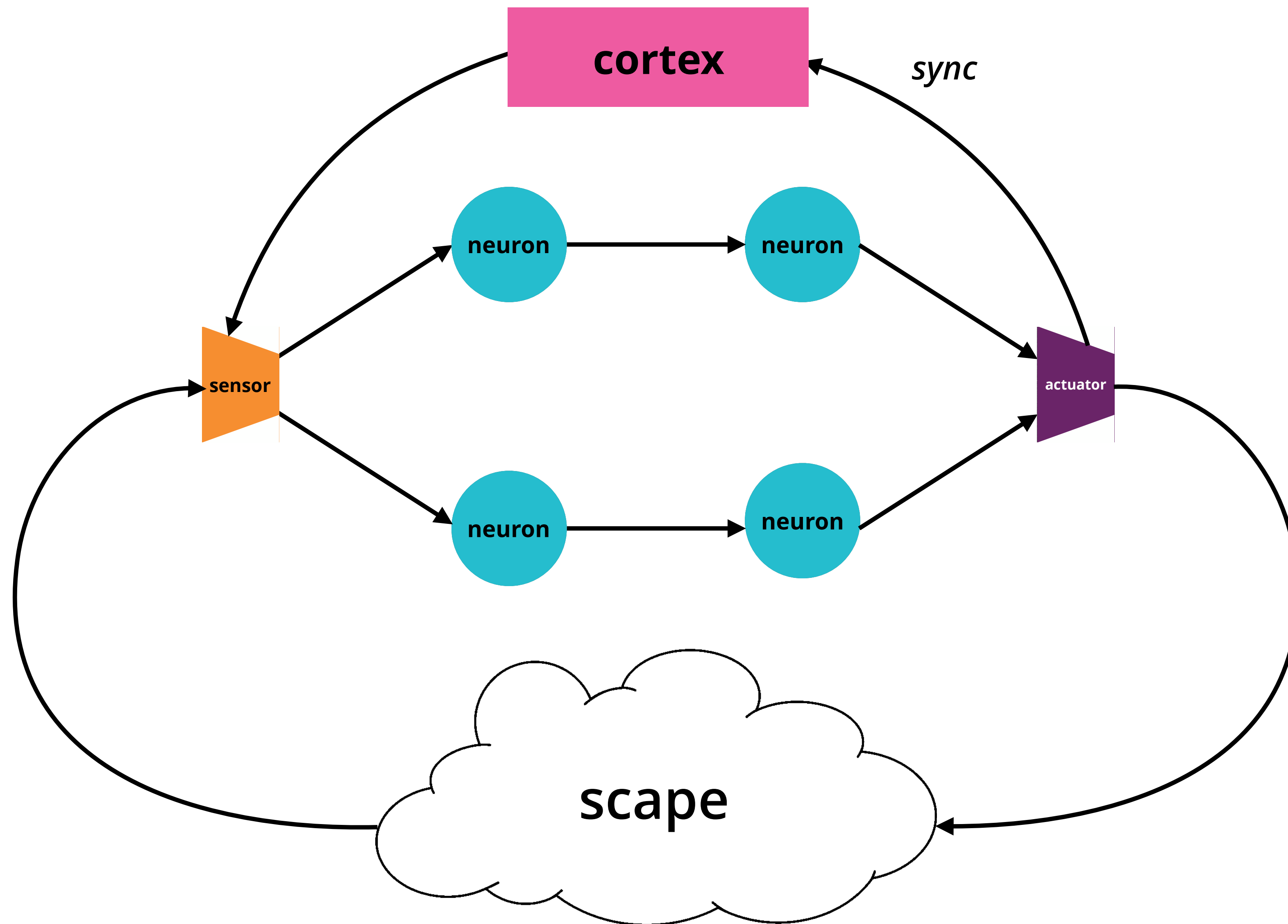
# THE NEURAL NETWORK

---



# THE NEURAL NETWORK

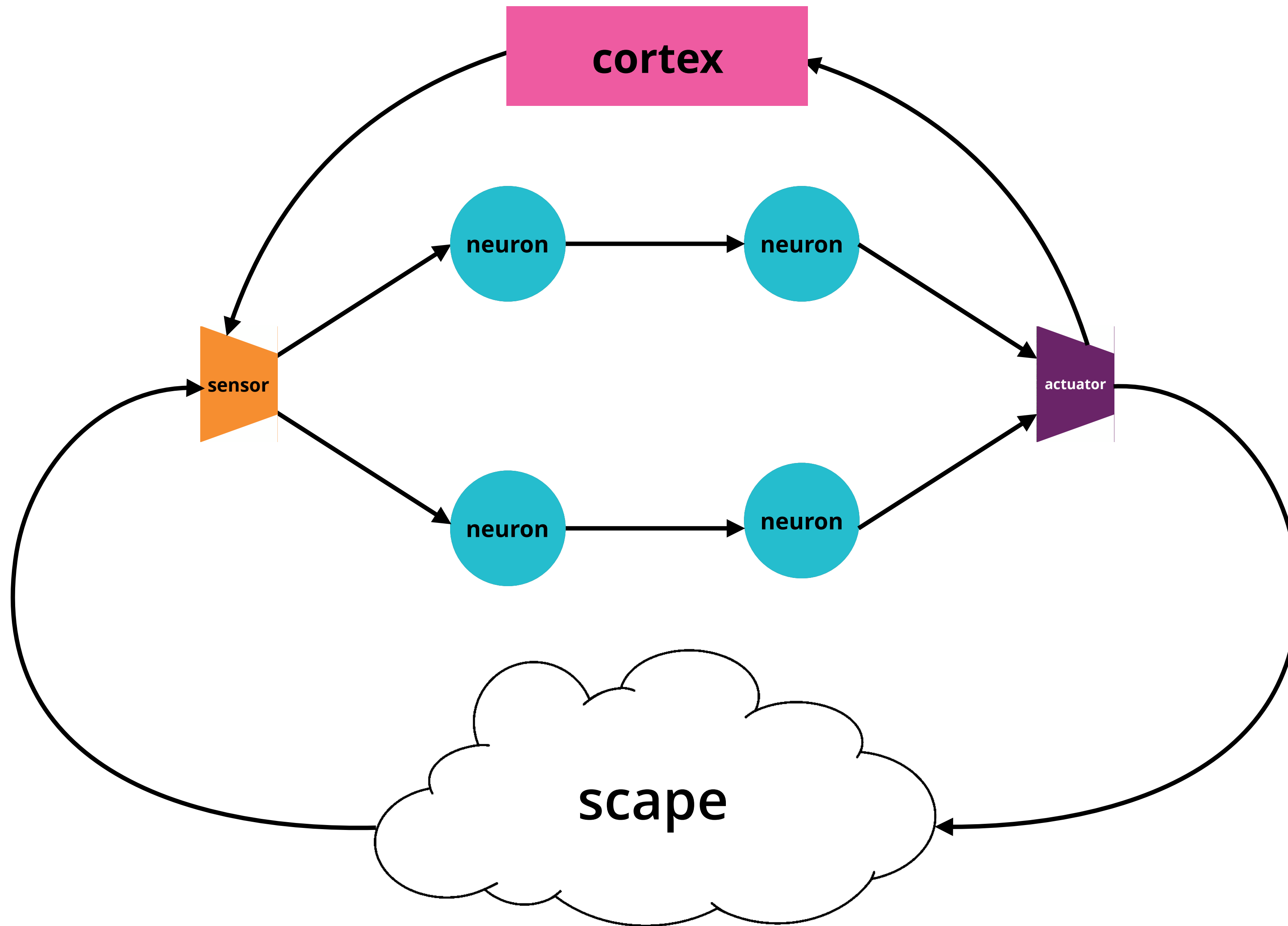
---





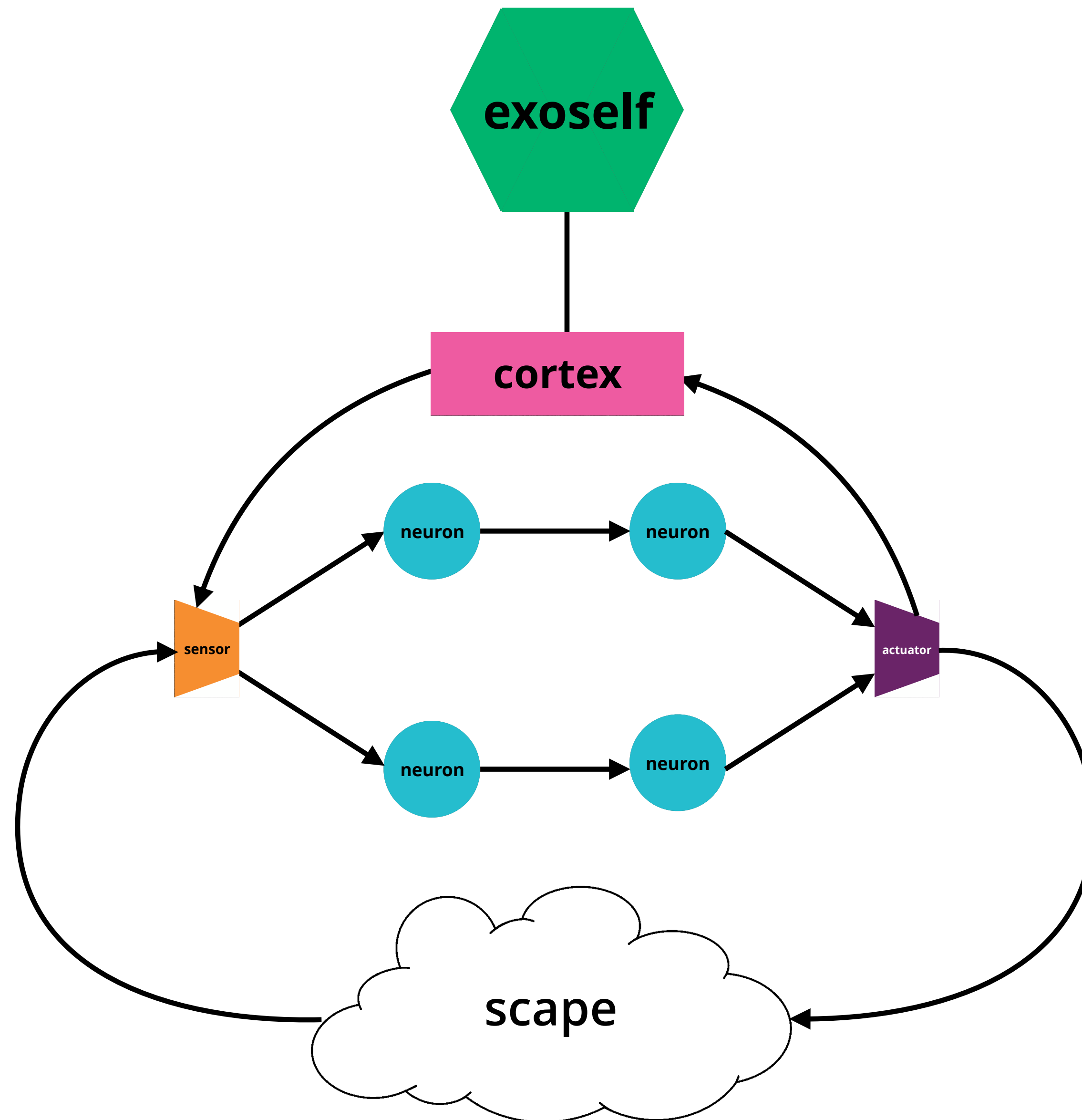
# THE NEURAL NETWORK

---



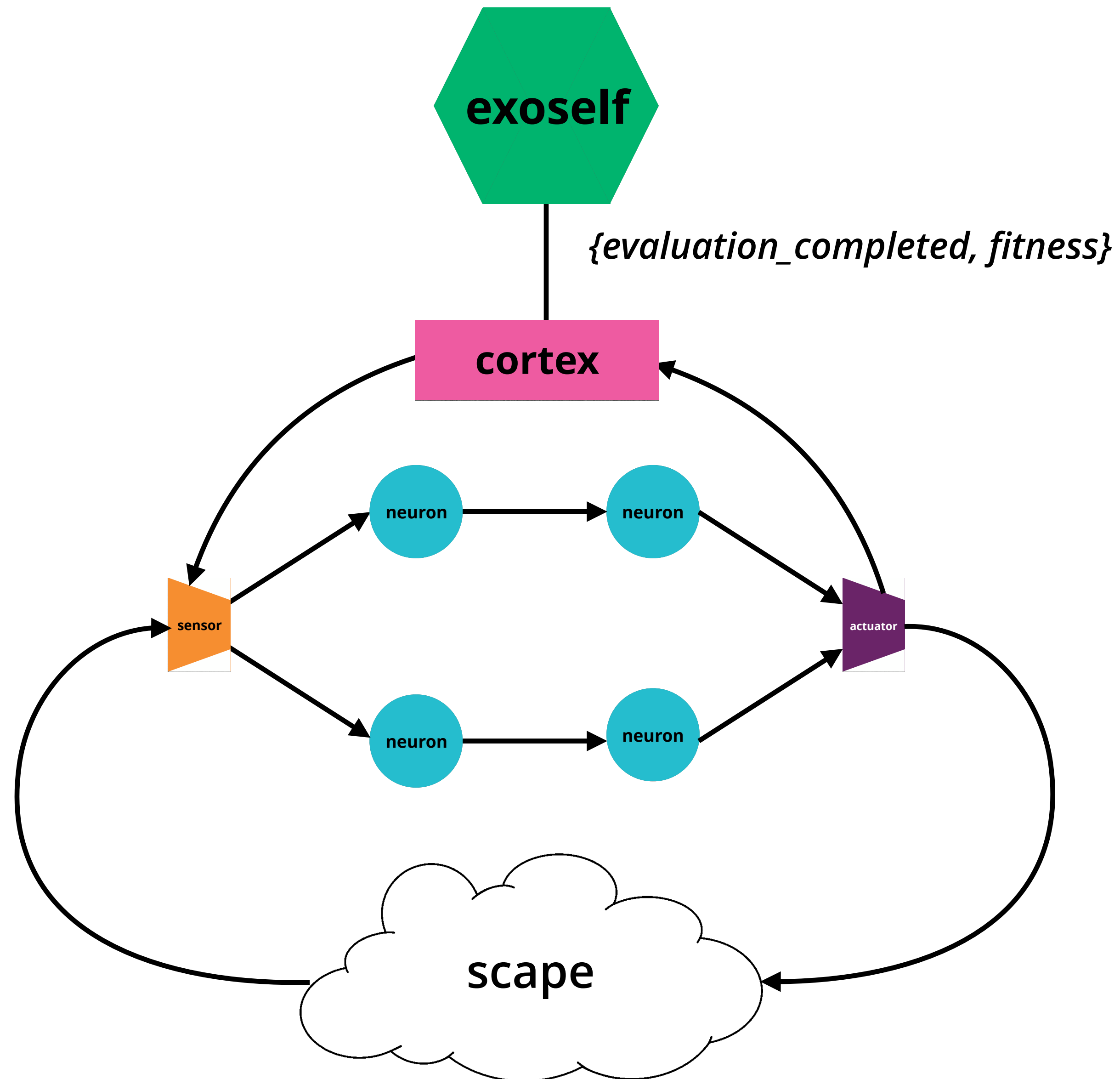
# THE EXOSELF

---



# THE EXOSELF

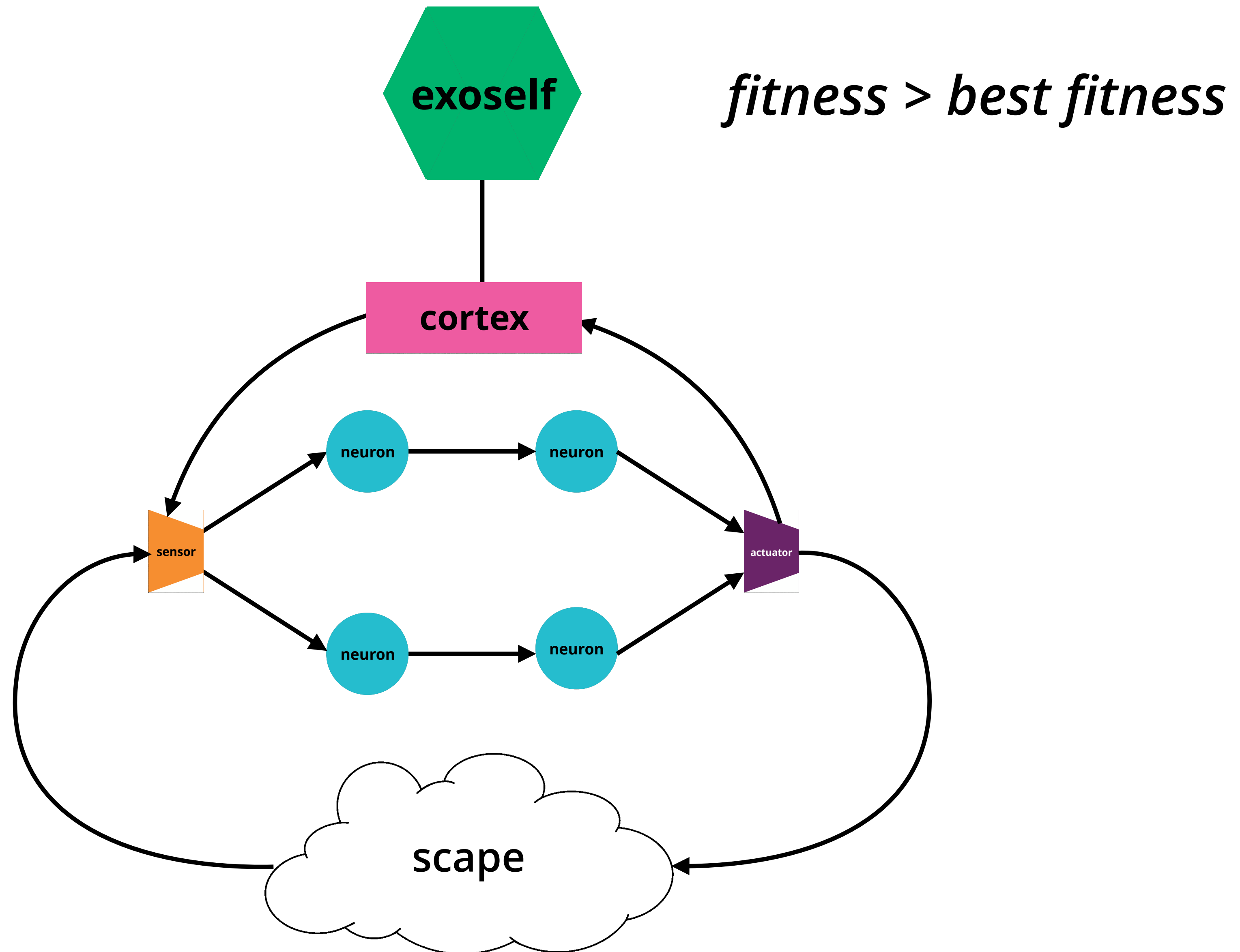
---





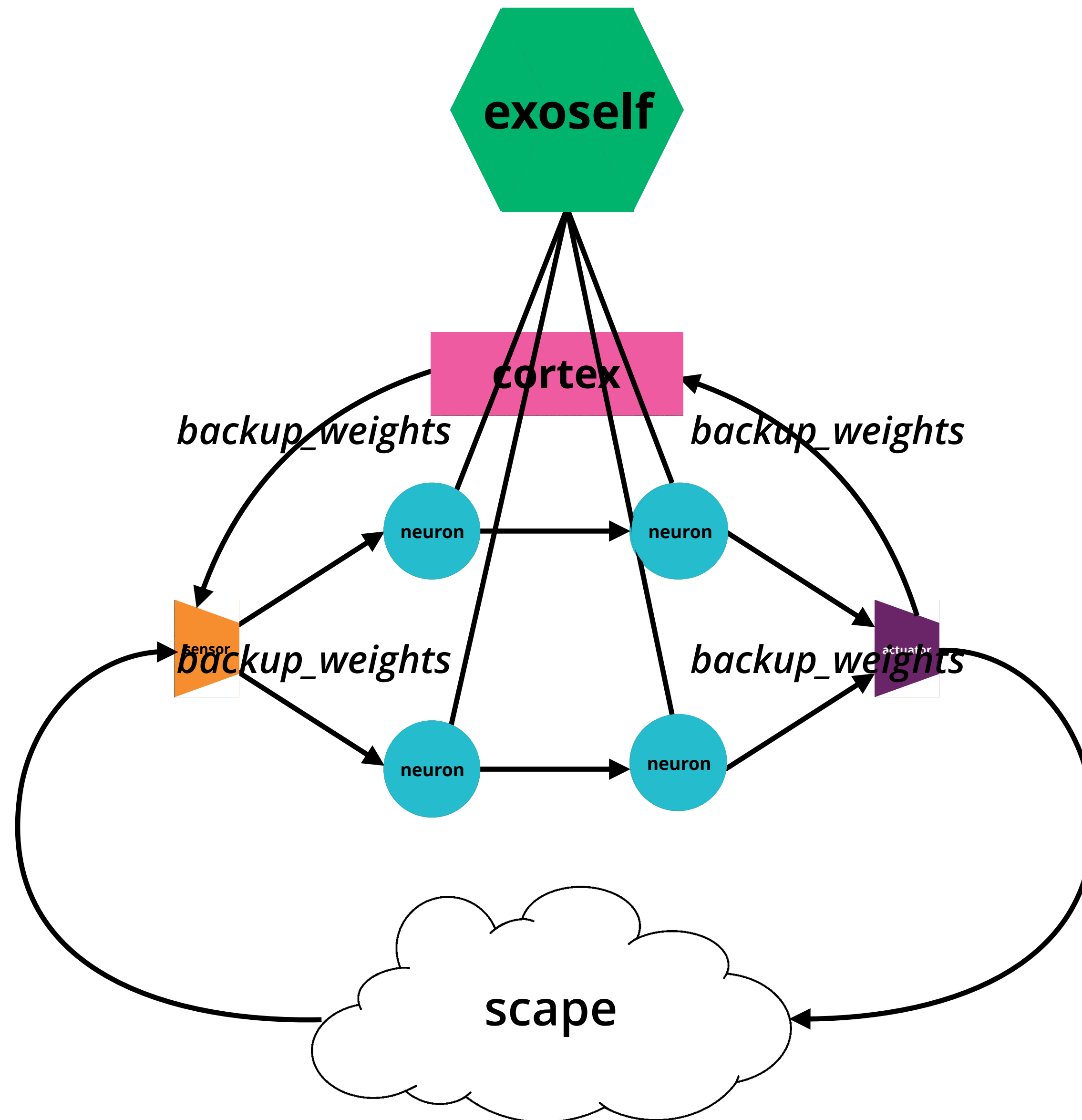
# THE EXOSELF

---



# THE EXOSELF

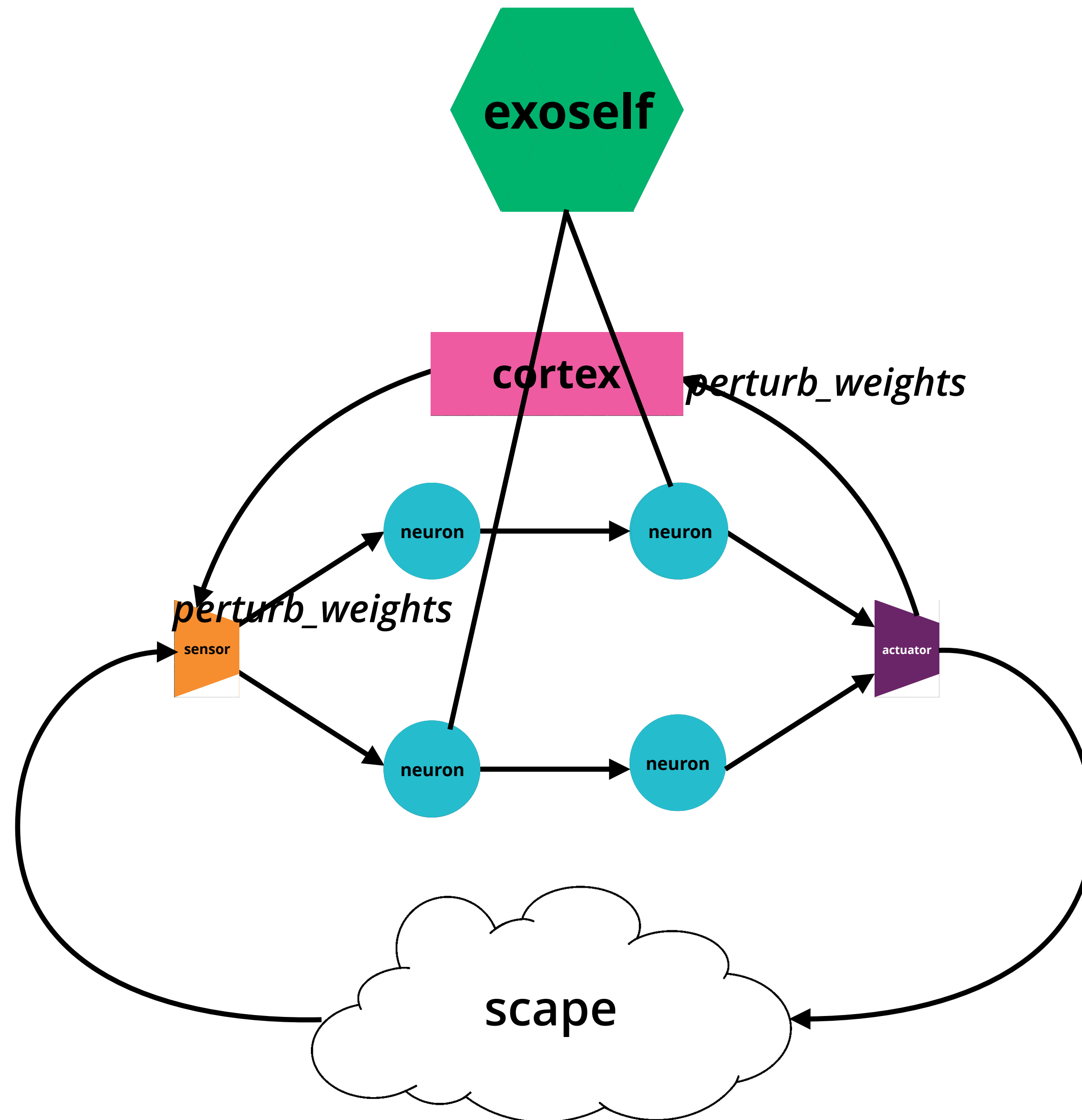
---





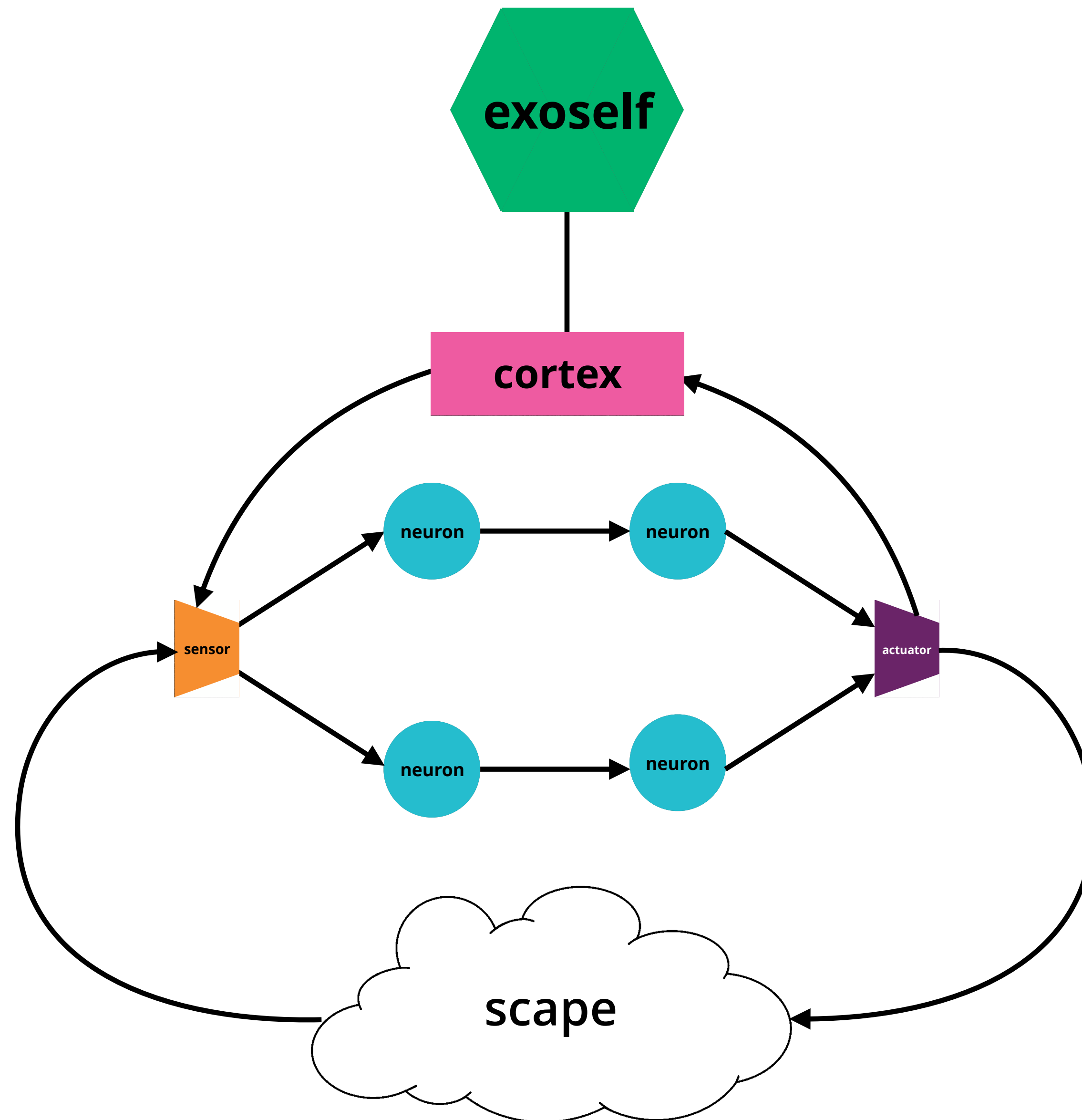
# THE EXOSELF

---



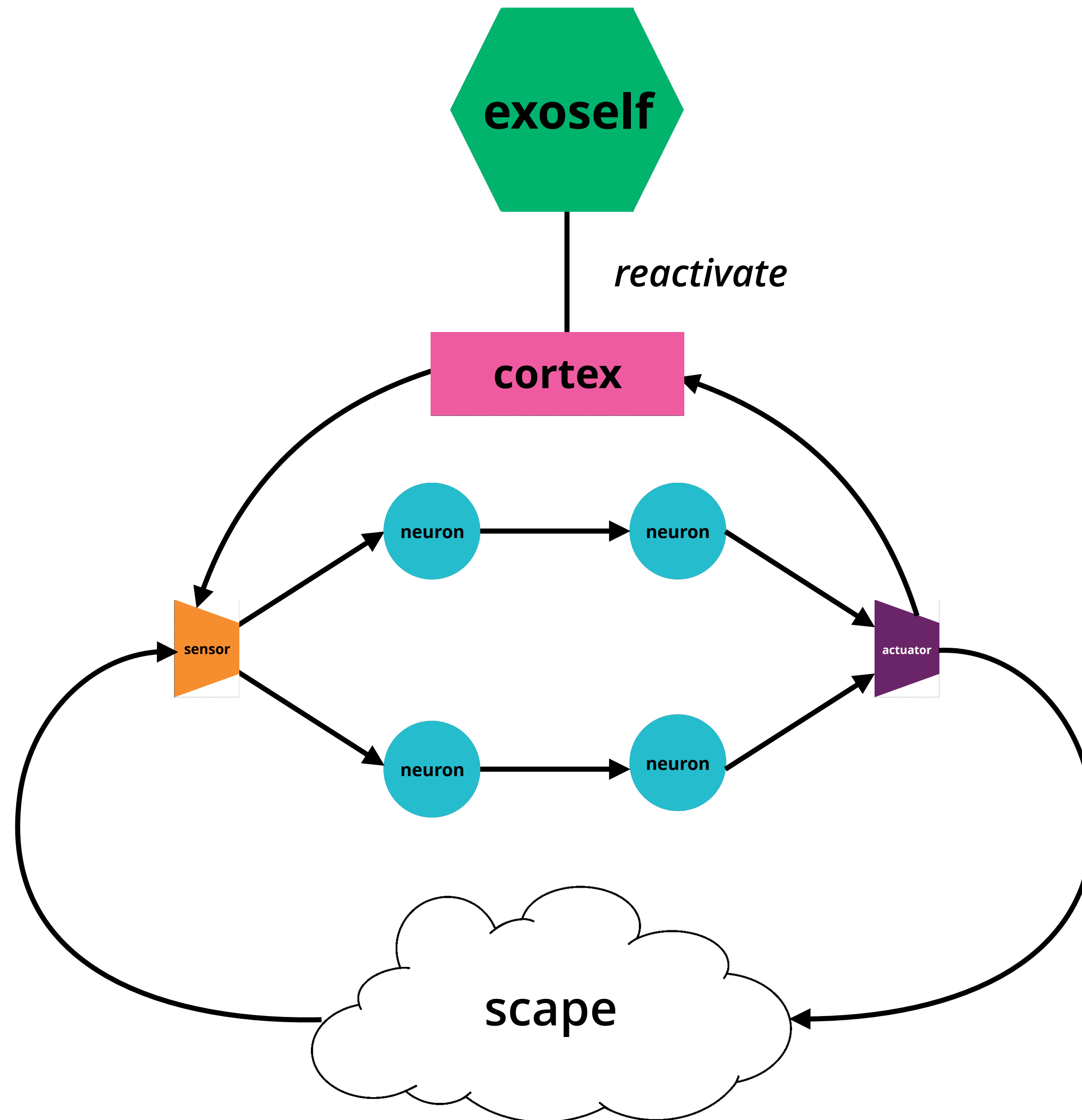
# THE EXOSELF

---



# THE EXOSELF

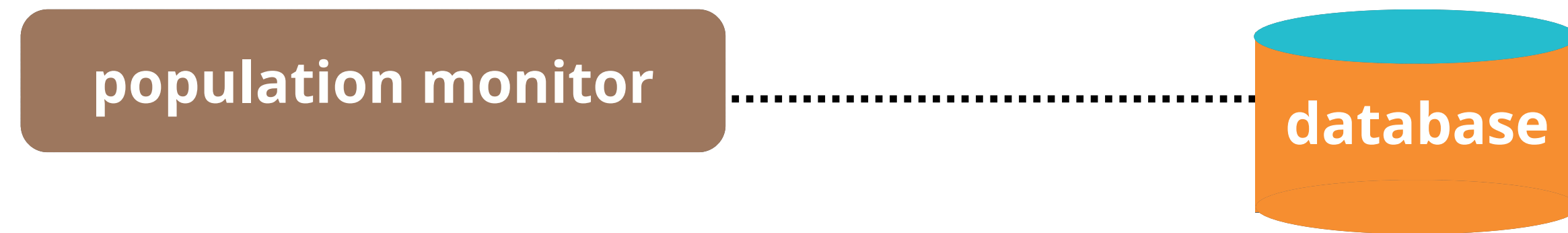
---



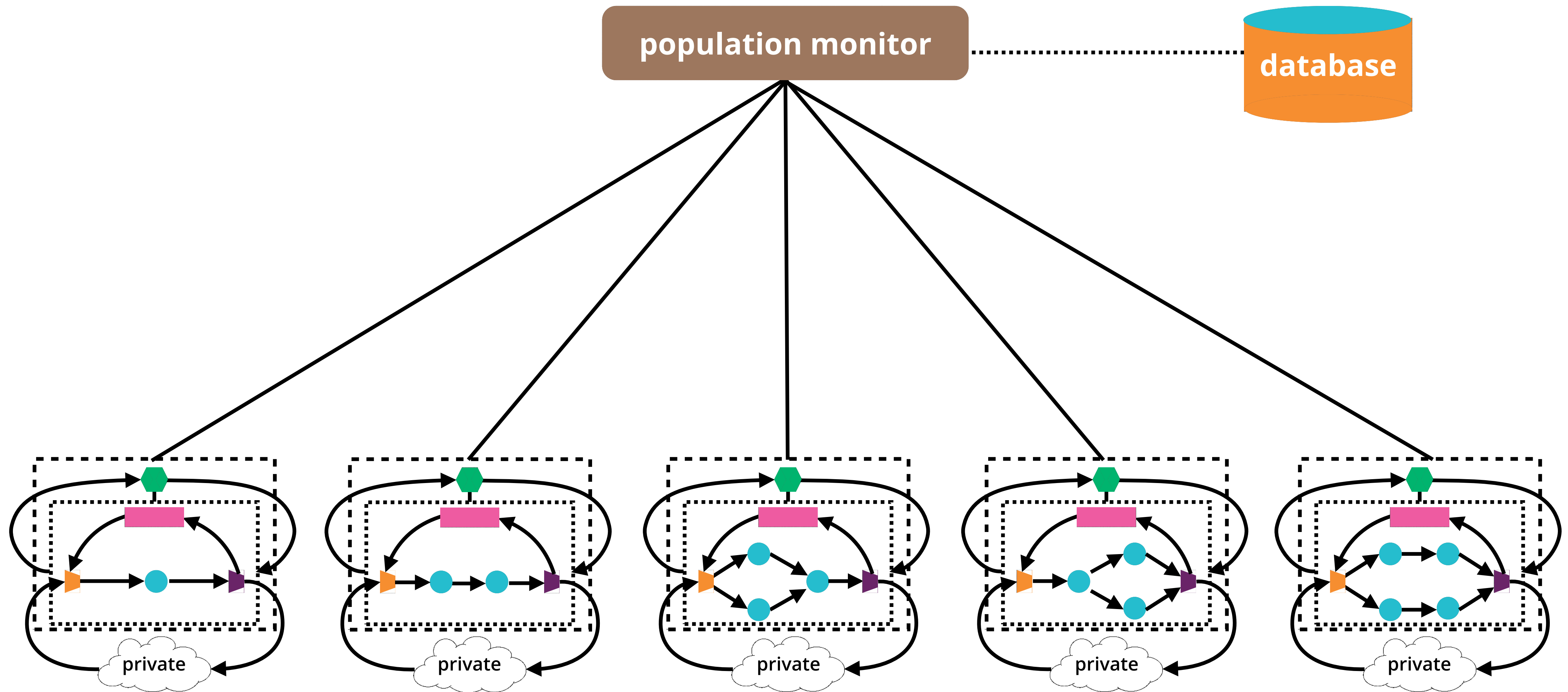


# THE POPULATION MONITOR

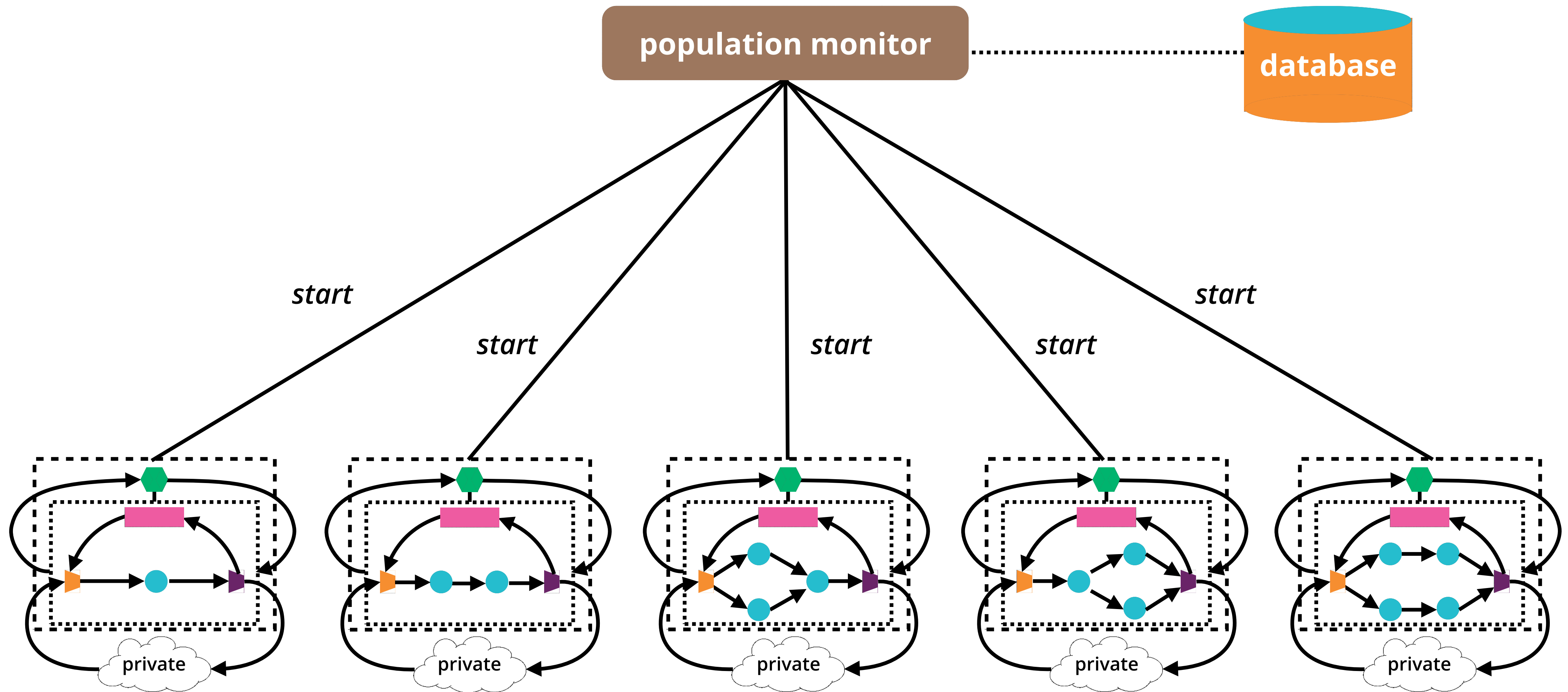
---



# THE POPULATION MONITOR

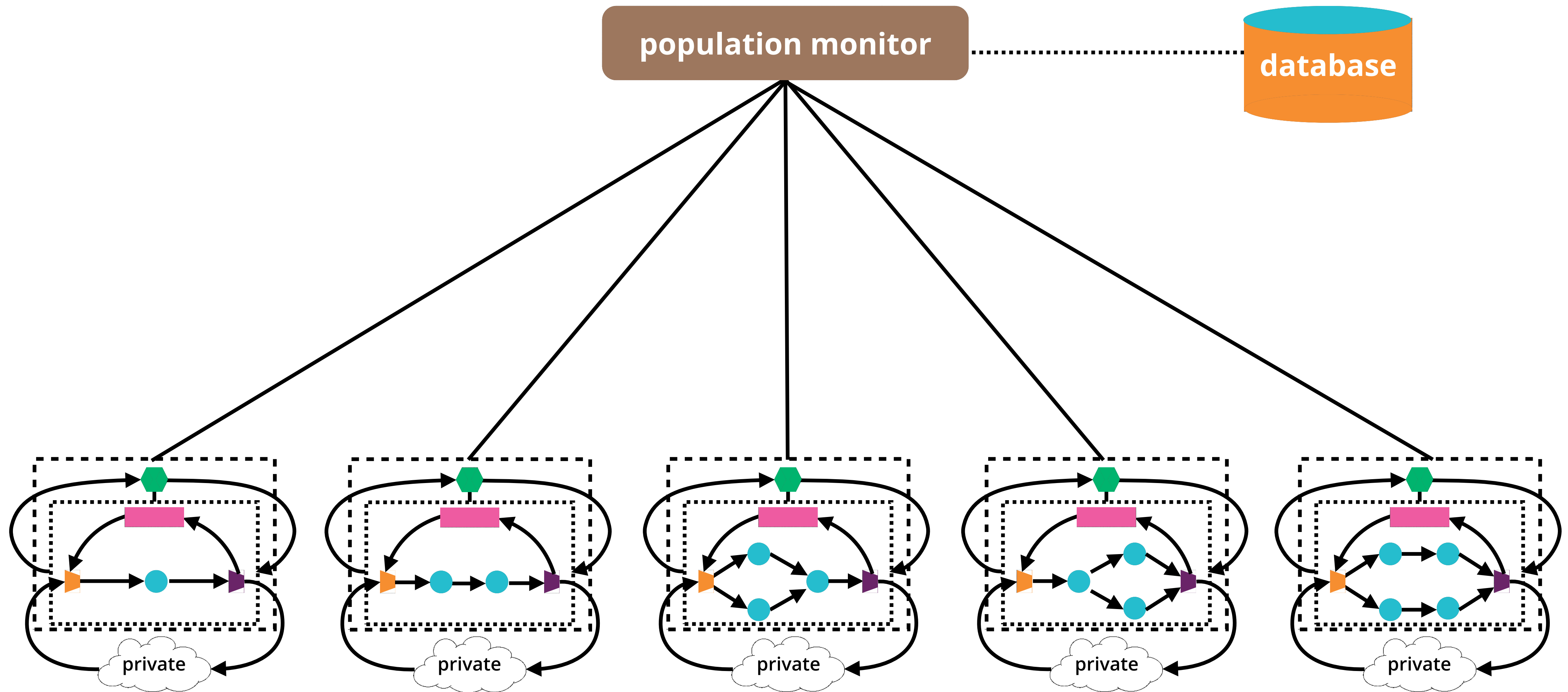


# THE POPULATION MONITOR

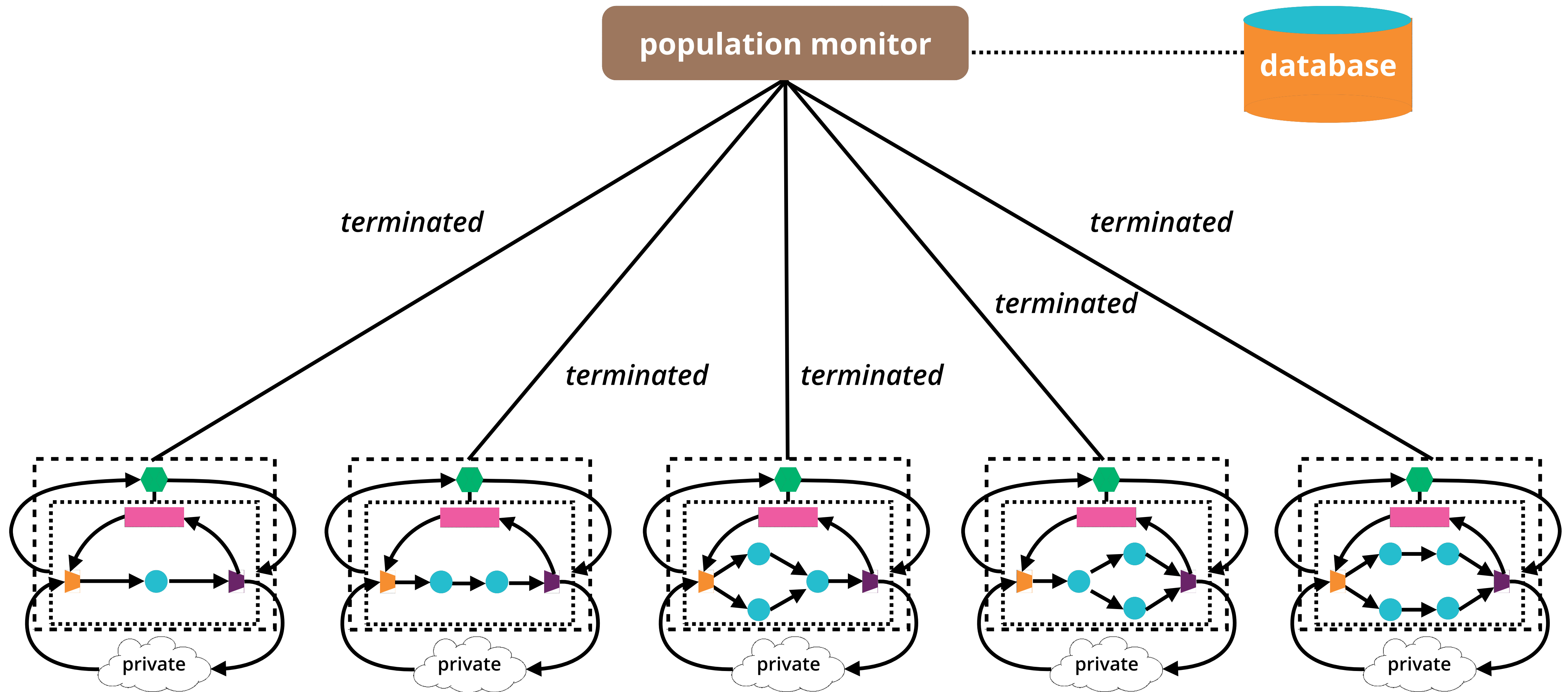




# THE POPULATION MONITOR



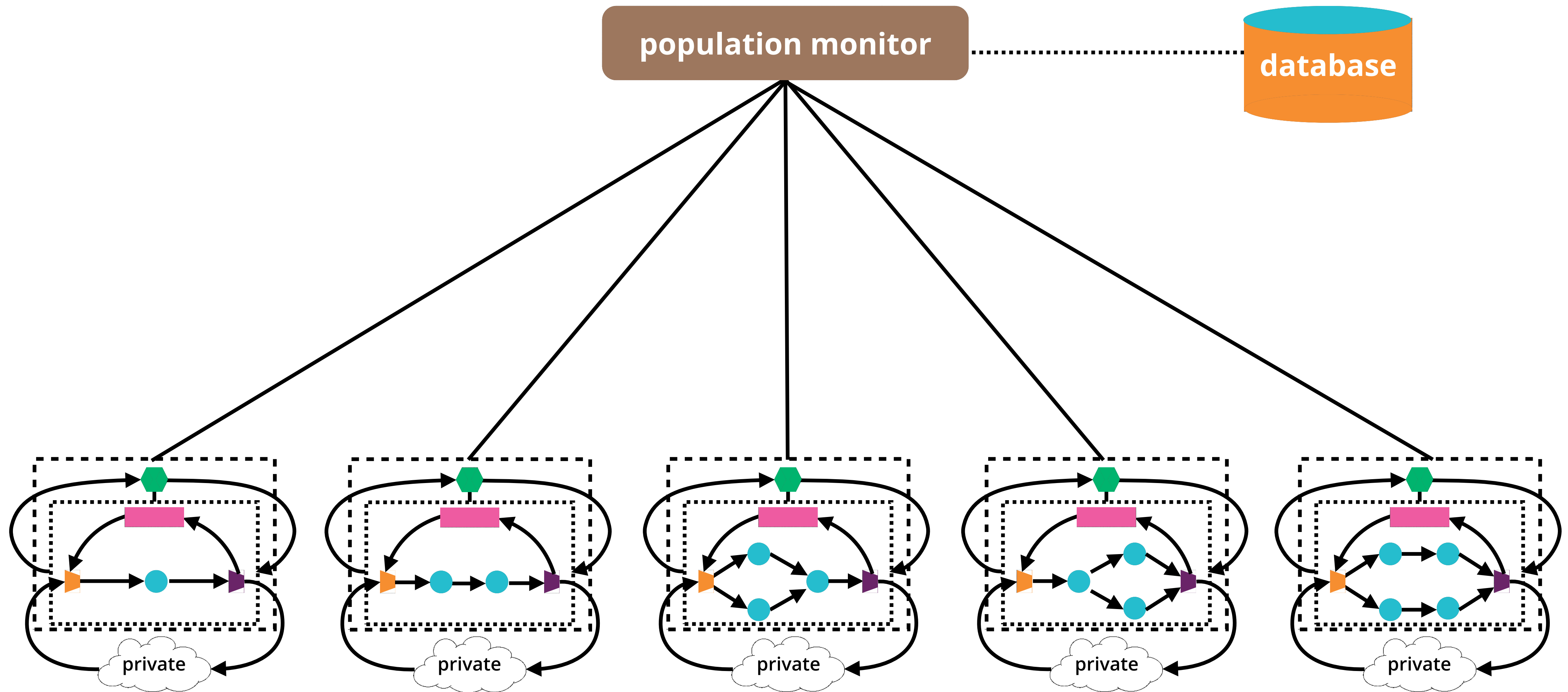
# THE POPULATION MONITOR





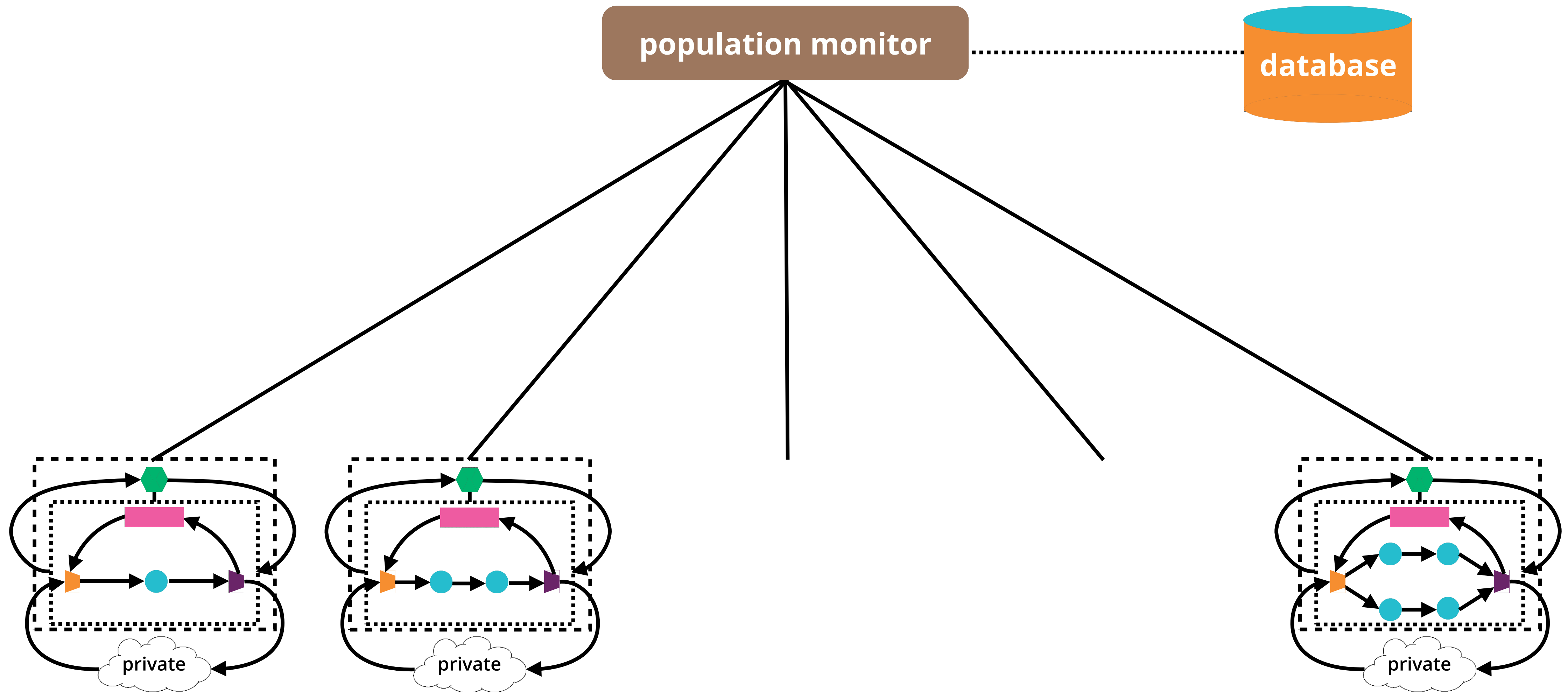
# THE POPULATION MONITOR

---



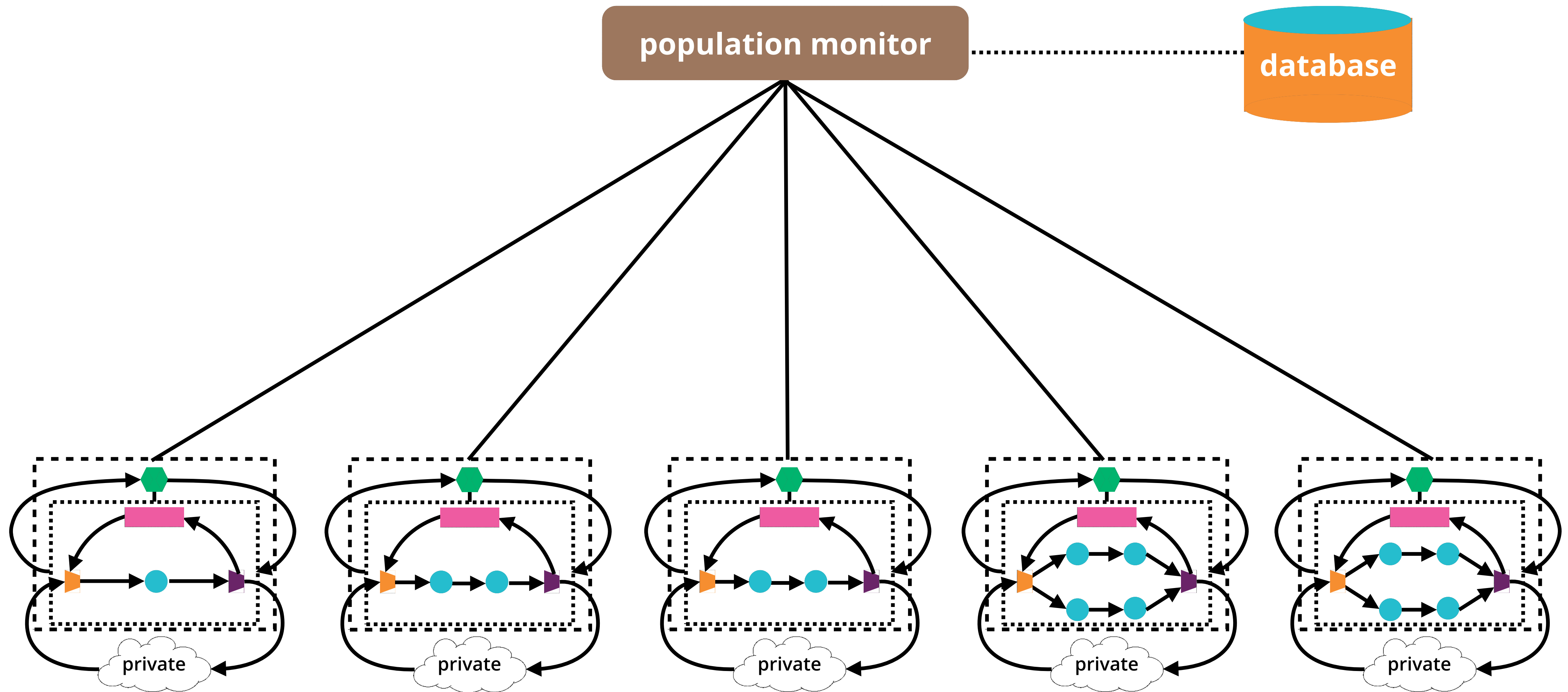
# THE POPULATION MONITOR

---



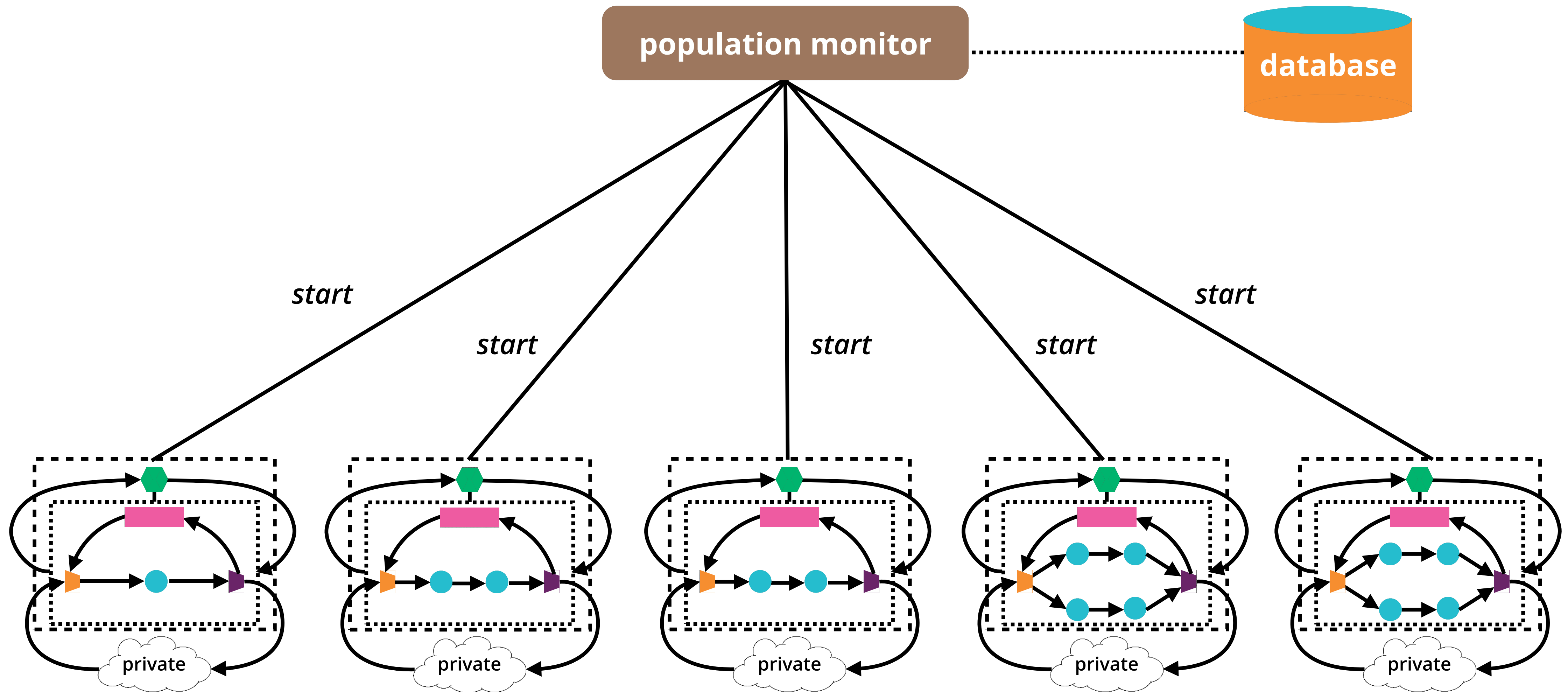
# THE POPULATION MONITOR

---





# THE POPULATION MONITOR



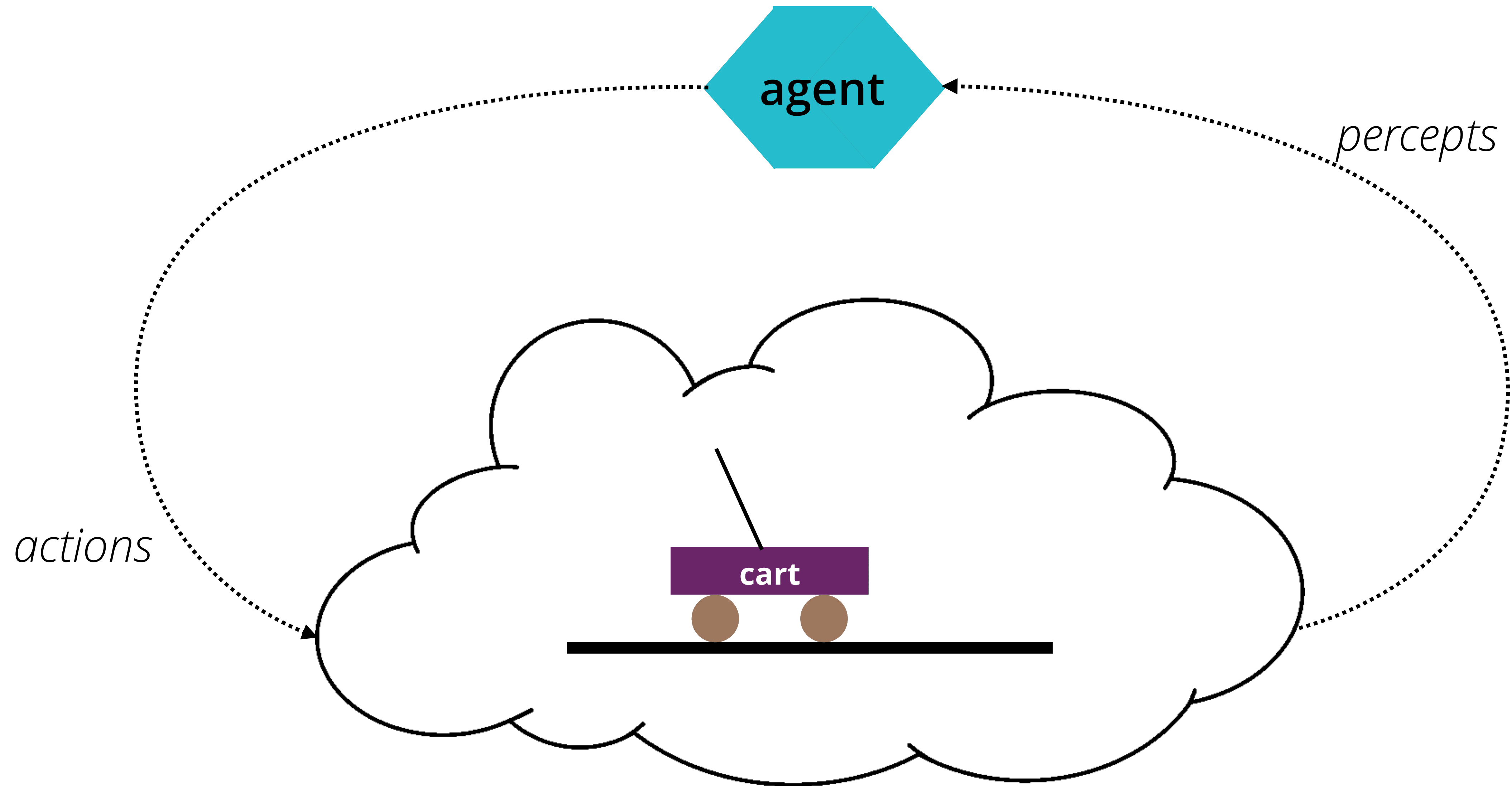
# THE DEVIL IS IN THE DETAILS

---

- recurrent connections
- newer generations get a higher chance for mutation
- neural plasticity
- public scapes and steady state evolution

# POLE BALANCING

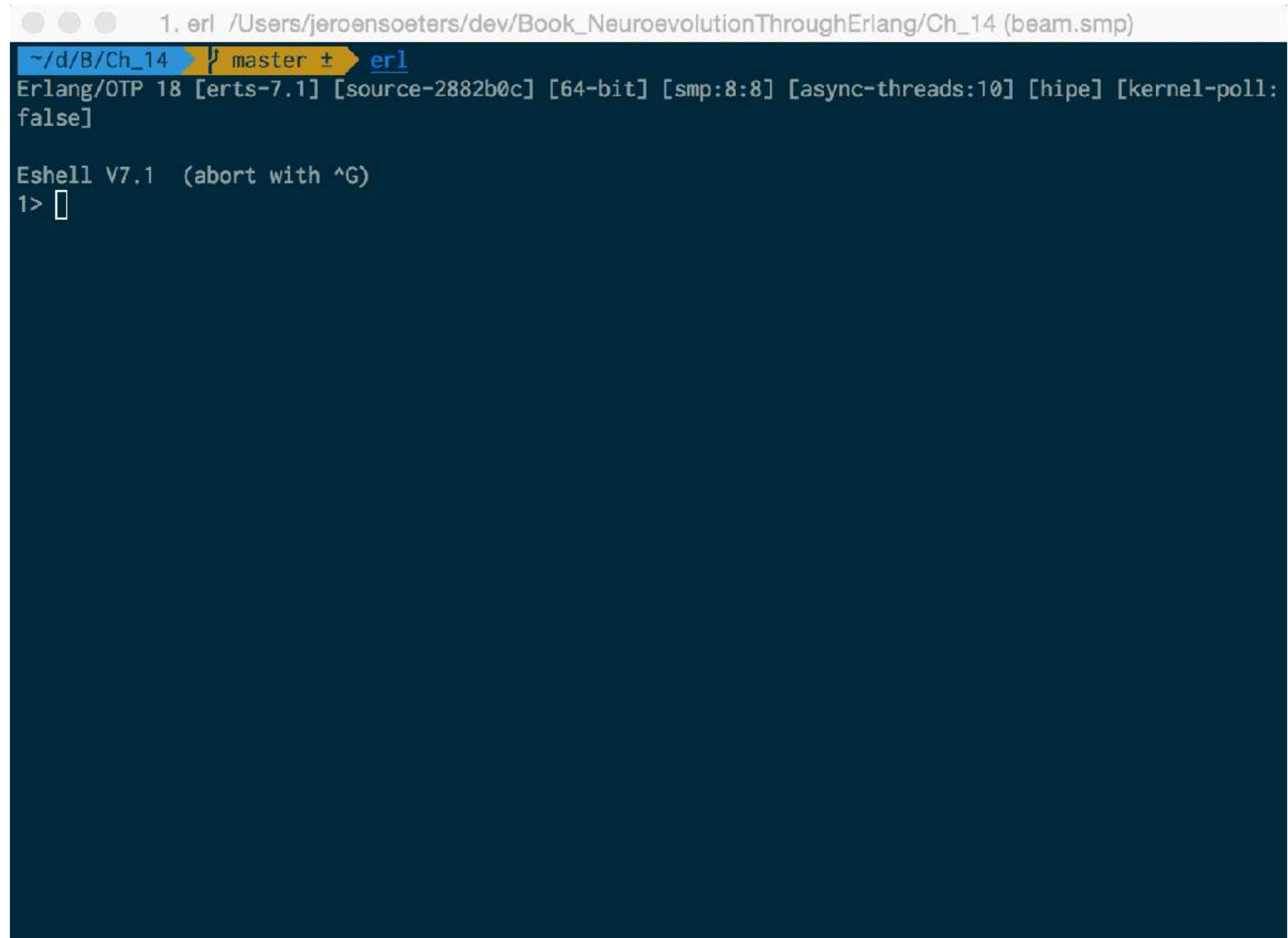
---





# DEMO

---

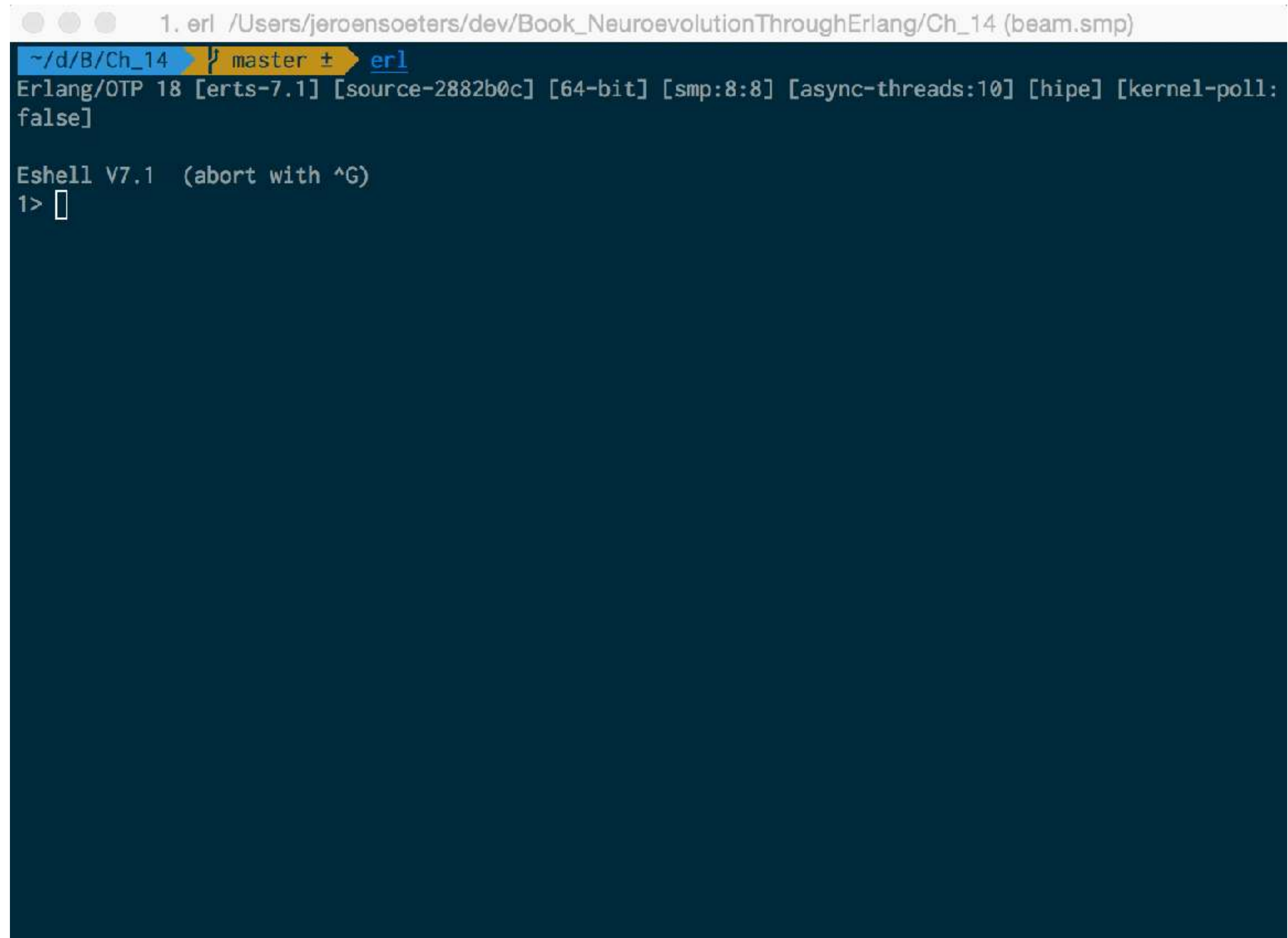


```
1. erl /Users/jeroensoeters/dev/Book_NeuroevolutionThroughErlang/Ch_14 (beam.smp)
~/d/B/Ch_14 master ± erl
Erlang/OTP 18 [erts-7.1] [source-2882b0c] [64-bit] [smp:8:8] [async-threads:10] [hipe] [kernel-poll:
false]

Eshell V7.1 (abort with ^G)
1> 
```

# DEMO

---

A terminal window with a dark blue background and light gray text. The title bar at the top shows three window control buttons and the text "1. erl /Users/jeroensoeters/dev/Book\_NeuroevolutionThroughErlang/Ch\_14 (beam.smp)". The terminal content shows the Erlang shell startup sequence: a prompt "~ /d/B/Ch\_14" followed by "master ±" and "erl". Below this, the system information is displayed: "Erlang/OTP 18 [erts-7.1] [source-2882b0c] [64-bit] [smp:8:8] [async-threads:10] [hipe] [kernel-poll:false]". Then, the Eshell version is shown: "Eshell V7.1 (abort with ^G)". Finally, the prompt "1>" is displayed with a cursor.

```
1. erl /Users/jeroensoeters/dev/Book_NeuroevolutionThroughErlang/Ch_14 (beam.smp)
~/d/B/Ch_14 master ± erl
Erlang/OTP 18 [erts-7.1] [source-2882b0c] [64-bit] [smp:8:8] [async-threads:10] [hipe] [kernel-poll:
false]

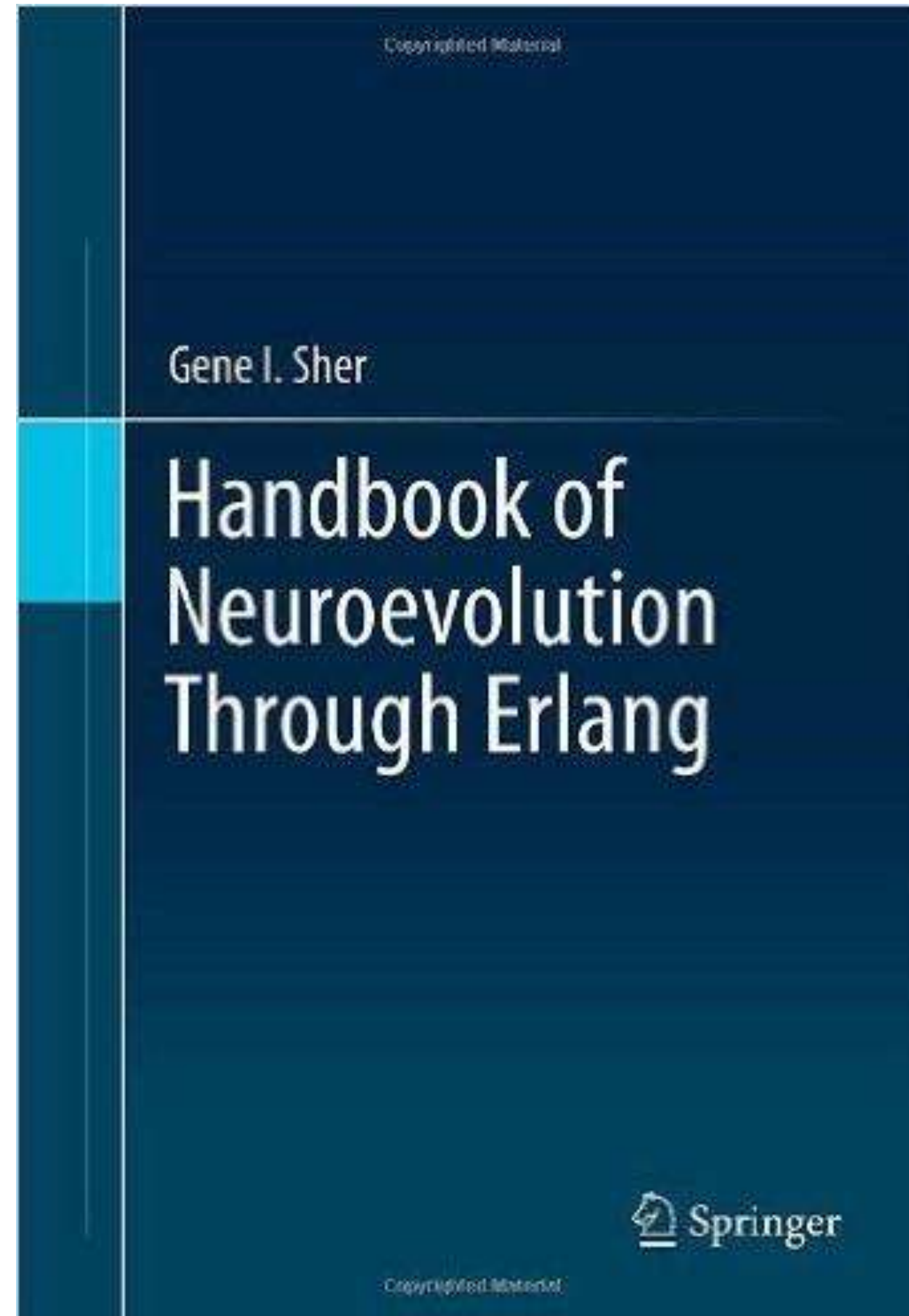
Eshell V7.1 (abort with ^G)
1> 
```



# BENCHMARK RESULTS

---

| Method         | Double-Pole/Partial Information W/O Damping | Double-Pole W/Damping |
|----------------|---------------------------------------------|-----------------------|
| RWG            | 415209                                      | 1232296               |
| SANE           | 262700                                      | 451612                |
| CNE            | 76906                                       | 87623                 |
| ESP            | 7374                                        | 26342                 |
| NEAT           | -                                           | 6929                  |
| CMA-ES*        | 3521*                                       | 6061*                 |
| CoSyNE*        | 1249*                                       | 3416*                 |
| <b>DXNN v1</b> | <b>2359</b>                                 | <b>2313</b>           |
| <b>DXNN v2</b> | <b>1289</b>                                 | <b>1830</b>           |



# THANK YOU

*Feel free to reach out at:*

*e. [wfhsoeters@gmail.com](mailto:wfhsoeters@gmail.com)*

*t. @JeroenSoeters*

**ThoughtWorks®**